MÔ HÌNH LOGISTIC REGRESSION

1. Từ Perceptron đến bài toán phân loại bằng xác suất

Ở bài trước, ta đã xây dựng **Perceptron**: mô hình tính điểm số $s = \mathbf{xw}$ rồi dùng hàm dấu để trả về một **nhãn cứng** ($\hat{y} = +1$ hoặc $\hat{y} = -1$). Perceptron là một bước tiến so với hồi quy tuyến tính vì nó cho ra nhãn thay vì một con số vô nghĩa, nhưng nó vẫn để lại nhiều câu hỏi chưa trả lời được. Mô hình nào cho biết nó **chắc chắn đến đâu**? Mô hình nào học **êm** ngay cả khi dữ liệu chồng lấn? Mô hình nào có thể đặt vào một mạng gồm nhiều lớp?

Mục tiêu của bài này là xây dựng **Logistic Regression**, mô hình phân loại tuyến tính cho ra **xác suất**, bằng cách tiếp tục tận dụng những gì đã học. Trong mục 1, ta trả lời ba câu hỏi:

- Perceptron còn hạn chế gì khi giải bài toán phân loại?
- Bài toán phân loại nhị phân được mô tả lại như thế nào khi ta muốn **dự đoán xác suất**?
- Ý tưởng nào cho phép biến điểm số tuyến tính thành một xác suất hợp lý?

Quy ước ký hiệu

Ta giữ nguyên quy ước ký hiệu của bài trước: chữ thường in nghiêng là đại lượng vô hướng, **chữ thường in đậm là vector**, **chữ hoa in đậm là ma trận**. Vector đặc trưng là **vector hàng**, nhờ đó các vector đặc trưng của n mẫu xếp chồng thành các hàng của ma trận đặc trưng.

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số mẫu, số đặc trưng	n, d	vô hướng	n quan sát, mỗi quan sát có d đặc trưng
Vector đặc trưng (hàng)	$\mathbf{x}_i$	$1 \times (d+1)$	Đặc trưng của mẫu thứ i, phần tử đầu tiên bằng 1
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$	Hàng thứ i của $\mathbf{X}$ là $\mathbf{x}_i$
Vector tham số (cột)	$\mathbf{w}$	$(d+1) \times 1$	Gồm hệ số chệch w_0 và các trọng số $w_1, \dots, w_d$
Vector nhãn thực (cột)	$\mathbf{y}$	$n \times 1$	Nhãn thực tế $y_i \in \{0,1\}$ của từng mẫu
Điểm số	s_i, $\mathbf{s}$	vô hướng, $n \times 1$	$s_i = \mathbf{x}_i\mathbf{w}$ và $\mathbf{s} = \mathbf{Xw}$
Xác suất dự đoán	p_i, $\mathbf{p}$	vô hướng, $n \times 1$	Xác suất mô hình gán cho việc mẫu i thuộc lớp dương
Nhãn dự đoán	$\hat{y}_i$, $\hat{\mathbf{y}}$	vô hướng, $n \times 1$	Nhãn mà mô hình dự đoán, $\hat{y}_i \in \{0,1\}$

Cụ thể, với mẫu thứ i ta có vector hàng và ma trận đặc trưng:

$$\mathbf{x}_i = [1 \quad x_{i1} \quad x_{i2} \quad \cdots \quad x_{id}], \qquad \mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1d} \\ 1 & x_{21} & \cdots & x_{2d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \cdots & x_{nd} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{bmatrix}$$

Vector tham số vẫn là vector cột:

$$\mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T$$

Một thay đổi quan trọng so với bài Perceptron: nhãn nay là $\{0,1\}$ chứ không phải $\{-1,+1\}$. Lý do sẽ rõ ở mục 1.2. Hai cách mã hóa chuyển qua lại dễ dàng: $y = \frac{y_\pm + 1}{2}$ và $y_\pm = 2y - 1$, trong đó $y_\pm$ là nhãn theo kiểu $\{-1,+1\}$ của bài trước.

1.1. Hạn chế của Perceptron khi giải quyết bài toán phân loại

Perceptron hoạt động tốt trong những điều kiện đơn giản (dữ liệu phân tách tuyến tính được), và nó đã dạy ta nhiều ý tưởng nền tảng: đường biên $\mathbf{xw} = 0$, margin $m = y\,s$, quy tắc cập nhật khi sai. Tuy nhiên, khi áp dụng vào thực tế, ta gặp bốn hạn chế sau.

Hạn chế 1: chỉ cho nhãn cứng, không cho biết độ tin cậy

Hãy quay lại ví dụ sinh viên đậu/rớt với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$, và xét hai sinh viên:

- **Sinh viên A** học 9 giờ, vắng 0 buổi: $\mathbf{x}_A = [1 \quad 9 \quad 0]$ nên $s_A = -2{,}5 + 9 - 0 = 6{,}5$.
- **Sinh viên B** học 5 giờ, vắng 2 buổi: $\mathbf{x}_B = [1 \quad 5 \quad 2]$ nên $s_B = -2{,}5 + 5 - 2 = 0{,}5$.

Cả hai đều có $s \geq 0$ nên Perceptron trả lời giống hệt nhau: **"đậu"**. Nhưng hai trường hợp khác nhau rất nhiều. Sinh viên A nằm sâu trong vùng "đậu", còn sinh viên B chỉ cách đường biên một chút, và chỉ cần học ít hơn một giờ hoặc vắng thêm một buổi là đã chuyển sang "rớt". Perceptron **không nói được** sự khác biệt đó.

Trong thực tế, ta thường cần biết mức độ chắc chắn để ra quyết định. Ví dụ, nhà trường muốn **cảnh báo học vụ sớm** cho mọi sinh viên có khả năng rớt trên 30%, kể cả khi mô hình vẫn nghiêng về "đậu". Với nhãn cứng, điều này không làm được.

Hạn chế 2: không có cách điều chỉnh ngưỡng theo hậu quả của từng loại sai

Ở mục 1.1 và 5.2 của bài Perceptron, ta đã thấy có hai loại sai (bỏ sót và báo nhầm) với hậu quả khác nhau tùy bài toán. Với Perceptron, ngưỡng phân chia **cố định** ở $s = 0$. Nếu bài toán đòi hỏi phát hiện thật nhiều ca dương (ví dụ tầm soát bệnh), ta không có cách nào "hạ ngưỡng" một cách có ý nghĩa, vì điểm số s không có đơn vị rõ ràng. Nếu mô hình cho **xác suất**, ta có thể nói rõ: "dự đoán dương khi xác suất vượt 20%".

Hạn chế 3: không học êm khi dữ liệu chồng lấn

Ở mục 5.2 của bài Perceptron, với dữ liệu chồng lấn, số mẫu bị sai của bộ $\mathbf{w}$ hiện tại **dao động mạnh** từ khoảng 25 đến hơn 100 trên 280 mẫu qua các epoch, và ta phải dùng thêm thuật toán túi để giữ lại bộ $\mathbf{w}$ tốt nhất. Nguyên nhân là hàm mất mát $\max(0, -m)$ chỉ quan tâm đến mẫu sai

và quy tắc cập nhật luôn có bước nhảy cố định $\pm\eta\mathbf{x}_i^T$, nên khi luôn còn mẫu sai, $\mathbf{w}$ không bao giờ yên.

Hạn chế 4: hàm mất mát và hàm kích hoạt không "mượt"

Có hai khía cạnh:

- **Mẫu đúng nhưng sát đường biên không được quan tâm.** Mất mát Perceptron bằng 0 ngay khi $m \geq 0$. Một mẫu có margin $m = 0{,}01$ và một mẫu có margin $m = 5$ đều bị coi là "ổn" như nhau, nên mô hình không có động lực đẩy mẫu sát biên ra xa. Hệ quả đã thấy ở mục 4.1 của bài Perceptron: thuật toán dừng ngay khi hết mẫu sai, và đường biên có thể sát mẫu.
- **Hàm dấu có đạo hàm bằng 0 hầu như mọi nơi** (mục 2.1 của bài Perceptron). Điều này khiến ta không thể dùng Gradient Descent trực tiếp trên đầu ra, và cũng không thể ghép nhiều Perceptron thành mạng nhiều lớp huấn luyện được.

Tóm tắt các hạn chế và điều ta cần

Hạn chế của Perceptron	Điều ta mong muốn
Chỉ cho nhãn cứng ± 1	Đầu ra là một **xác suất** thể hiện độ tin cậy
Ngưỡng cố định, không gắn với hậu quả	Có thể **chọn ngưỡng** theo bài toán (ví dụ cảnh báo khi $P > 30\%$)
Dao động khi dữ liệu chồng lấn	Hàm mất mát **trơn** để Gradient Descent hoạt động êm
Mẫu đúng nhưng sát biên không được quan tâm	Hàm mất mát vẫn **phạt nhẹ** mẫu chưa đủ chắc chắn
Hàm dấu không có đạo hàm hữu ích	Hàm kích hoạt **khả vi**

Ở các mục sau, ta sẽ thấy tất cả những mong muốn này được đáp ứng bằng cùng một thay đổi: **thay hàm dấu bằng một hàm cho ra xác suất**.

1.2. Mô tả bài toán phân loại nhị phân theo xác suất và quy ước nhãn

Từ "nhãn" sang "xác suất"

Trước đây, ta coi bài toán phân loại nhị phân là: với đặc trưng $\mathbf{x}$, **dự đoán nhãn** $\hat{y} \in \{-1, +1\}$. Bây giờ ta nhìn bài toán theo cách tinh tế hơn: với đặc trưng $\mathbf{x}$, **dự đoán xác suất** mẫu thuộc lớp dương:

$$p(\mathbf{x}) = P(y = 1 \mid \mathbf{x}) \in [0,1]$$

Cách nhìn này hợp lý vì **cùng một đặc trưng có thể cho ra kết quả khác nhau**. Trong thực tế, hai sinh viên có số giờ học và số buổi vắng hoàn toàn giống nhau vẫn có thể một người đậu, một người rớt, vì còn nhiều yếu tố ngoài hai đặc trưng này (sức khỏe, may rủi, độ khó của đề, …). Do đó, nhãn không hoàn toàn được xác định bởi $\mathbf{x}$, và mô tả trung thực nhất là nói về **khả năng**: nếu có rất nhiều sinh viên cùng đặc trưng $\mathbf{x}$ thì tỉ lệ đậu là bao nhiêu? Con số đó chính là $p(\mathbf{x})$.

Từ xác suất, ta lấy lại nhãn bằng một quy tắc đơn giản (ngưỡng 0,5):

$$\hat{y} = \begin{cases} 1 & \text{nếu } p(\mathbf{x}) \geq 0{,}5 \\ 0 & \text{nếu } p(\mathbf{x}) < 0{,}5 \end{cases}$$

Quy ước nhãn {0,1}

Vì nhãn bây giờ là kết quả của một phép thử có hai khả năng (đậu hoặc rớt) với xác suất đậu là p, ta dùng cách mã hóa chuẩn của xác suất: $y = 1$ nếu đậu và $y = 0$ nếu rớt. Khi đó **giá trị kỳ vọng** của nhãn chính là xác suất cần dự đoán:

$$\mathbb{E}[y \mid \mathbf{x}] = 1 \cdot p(\mathbf{x}) + 0 \cdot \big(1 - p(\mathbf{x})\big) = p(\mathbf{x})$$

Nhờ đó, một mô hình dự đoán xác suất p cũng chính là một mô hình dự đoán y **ở mức trung bình**, và các công thức ở các mục sau (hàm mất mát, gradient) sẽ viết rất gọn bằng nhãn $\{0,1\}$. Bảng chuyển đổi giữa hai cách mã hóa:

Ý nghĩa	Bài Perceptron ($y_{\pm}$)	Bài này (y)
Lớp dương (đậu)	$+1$	1
Lớp âm (rớt)	-1	0
Công thức chuyển	$y_{\pm} = 2y - 1$	$y = \dfrac{y_{\pm} + 1}{2}$

Mô tả bài toán

Giả sử ta có bộ dữ liệu huấn luyện gồm n quan sát:

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}, \qquad y_i \in \{0,1\}$$

Bài toán phân loại nhị phân theo xác suất là: **tìm một mô hình cho ra xác suất** $p_i = P(y = 1 \mid \mathbf{x}_i)$ **từ đặc trưng** $\mathbf{x}_i$ **sao cho xác suất này phù hợp với dữ liệu quan sát được**, nghĩa là p_i cao với các mẫu có $y_i = 1$ và thấp với các mẫu có $y_i = 0$. Từ p_i, ta lấy nhãn dự đoán $\hat{y}_i$ bằng ngưỡng 0,5.

Ví dụ xuyên suốt: dự đoán sinh viên đậu hay rớt

Ta tiếp tục dùng ví dụ 6 sinh viên của bài Perceptron, với hai đặc trưng x_1 là số giờ tự học mỗi tuần và x_2 là số buổi vắng, nay mã hóa nhãn là $y = 1$ (đậu) và $y = 0$ (rớt):

Sinh viên	x_1 (giờ tự học)	x_2 (buổi vắng)	y (kết quả)
1	2	4	0 (rớt)
2	3	3	0 (rớt)
3	4	5	0 (rớt)
4	6	1	1 (đậu)
5	8	2	1 (đậu)
6	9	0	1 (đậu)

Ở đây $n = 6$ và $d = 2$. Ma trận đặc trưng và vector nhãn là

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 1 \\ 1 & 8 & 2 \\ 1 & 9 & 0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

So với bài Perceptron, ma trận **X** không đổi, chỉ có vector nhãn **y** đổi từ ± 1 sang $0/1$.

Ví dụ thực tế hơn: xác suất đậu theo số giờ học

Để thấy xác suất là một khái niệm tự nhiên, ta dùng bộ dữ liệu mô phỏng 400 sinh viên ở mục 5.2 của bài Perceptron (đậu/rớt, hai lớp chồng lấn nhau) và chỉ xét đặc trưng **số giờ tự học** x_1. Chia sinh viên thành các nhóm theo số giờ học và đếm tỉ lệ đậu trong từng nhóm:

Nhóm giờ học x_1	Số sinh viên	Số người đậu	Tỉ lệ đậu
$[0,2)$	39	0	0%
$[2,4)$	68	2	2,9%
$[4,6)$	96	33	34,4%
$[6,8)$	93	69	74,2%
$[8,10)$	79	71	89,9%
≥ 10	25	25	100%

Có hai điều đáng chú ý. Thứ nhất, **không có một "ngưỡng giờ học" nào tách hẳn đậu khỏi rớt**: ở nhóm $[4,6)$ giờ, khoảng một phần ba sinh viên đậu, còn hai phần ba rớt. Thứ hai, tỉ lệ đậu **tăng dần và chuyển từ gần** 0 **lên gần** 1 **một cách trơn tru**, không nhảy vọt. Hai điều này cho thấy mô hình tự nhiên nhất cho dữ liệu này là một hàm xác suất tăng dần theo số giờ học, chứ không phải một hàm nhãn cứng.

1.3. Ý tưởng: biến điểm số tuyến tính thành xác suất

Thử dùng thẳng hồi quy tuyến tính trên nhãn $0/1$

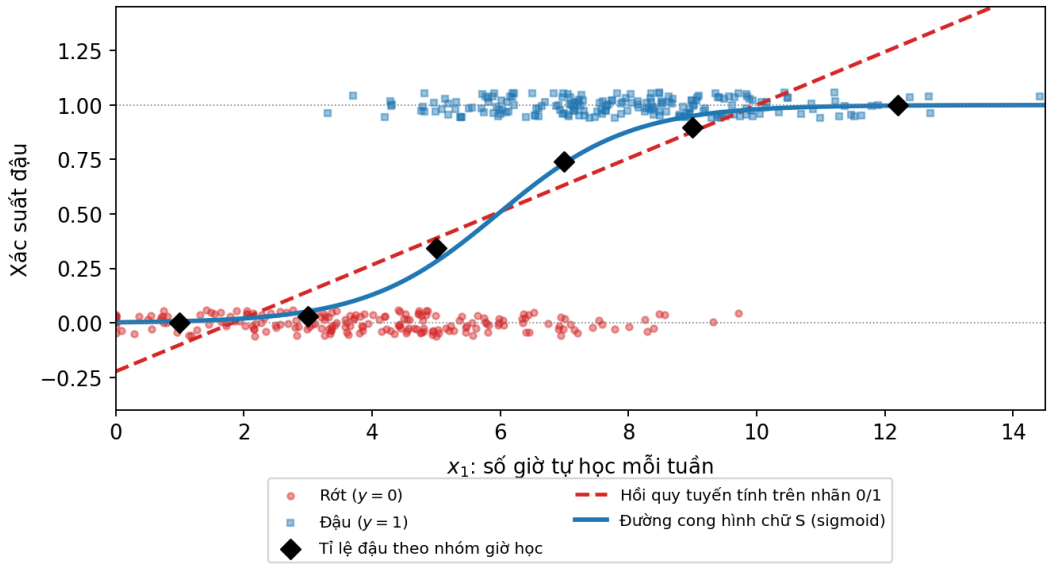
Một ý tưởng đơn giản: coi nhãn 0 và 1 là hai con số và dùng hồi quy tuyến tính để dự đoán, rồi hiểu kết quả $s = \mathbf{xw}$ là xác suất. Ta thử trên dữ liệu "tỉ lệ đậu theo số giờ học" ở trên. Hồi quy tuyến tính trên nhãn $0/1$ cho

$$p = -0{,}222 + 0{,}122\, x_1$$

(đường đứt đỏ trong Hình 1). Ta thấy ngay hai điều không ổn:

- Với $x_1 = 0$ giờ, mô hình cho "xác suất" $p = -0{,}222 < 0$.
- Với $x_1 = 12$ giờ, mô hình cho "xác suất" $p = -0{,}222 + 0{,}122 \cdot 12 \approx 1{,}244 > 1$.

Xác suất không thể âm hoặc lớn hơn 1. Ngoài ra, đường thẳng không bám sát hình dạng thật của dữ liệu: tỉ lệ đậu nằm sát 0 ở bên trái và sát 1 ở bên phải, còn ở giữa thì tăng nhanh, tạo nên một đường cong hình chữ **S**.



Hình 1. Xác suất đậu theo số giờ tự học. Các điểm nhỏ là sinh viên (đậu ở trên, rớt ở dưới, đã rung nhẹ cho dễ nhìn), hình thoi đen là tỉ lệ đậu theo từng nhóm giờ học. Đường thẳng đứt (hồi quy tuyến tính) vượt ra ngoài $[0,1]$*; đường cong hình chữ S bám sát tỉ lệ đậu thực tế.*

Vấn đề đã gặp ở bài trước (mục 1.2 của bài Perceptron): hồi quy tuyến tính cho ra giá trị **không bị chặn**, còn xác suất bắt buộc nằm trong $[0,1]$. Ta cần một cách "nén" điểm số về khoảng này.

Tách làm hai bước: điểm số tuyến tính, rồi nén thành xác suất

Ý tưởng chính của bài này gồm hai bước:

1. **Giữ nguyên điểm số tuyến tính** $s = \mathbf{xw}$. Điểm số có thể là mọi số thực: $s \in (-\infty, +\infty)$. Điểm số lớn dương nghĩa là "rất có khả năng đậu", rất âm nghĩa là "rất có khả năng rớt".
2. **Đưa điểm số qua một hàm "nén"** σ để thu được xác suất $p = \sigma(s) \in (0,1)$.

Hàm σ cần có các tính chất hợp lý: **tăng** theo s (điểm số cao thì xác suất cao), tiến tới 0 khi $s \to -\infty$ và tiến tới 1 khi $s \to +\infty$, bằng 0,5 tại $s = 0$ (không nghiêng về bên nào), và **trơn** (có đạo hàm ở mọi điểm) để dùng Gradient Descent. Câu hỏi là: hàm nào?

Gợi ý từ "odds" và "log-odds"

Có một cách nhìn tự nhiên dẫn đến đúng hàm cần tìm. Thay vì xác suất p, người ta thường dùng **odds** (tỉ số cược, hay "cơ hội"), là tỉ lệ giữa khả năng xảy ra và khả năng không xảy ra:

$$\text{odds} = \frac{p}{1-p}$$

Ví dụ, $p = 0{,}8$ nghĩa là khả năng đậu gấp 4 lần khả năng rớt, tức odds $= 4$ ("4 trên 1"). Khi $p = 0{,}5$ thì odds $= 1$ ("ngang ngửa"), và khi $p = 0{,}2$ thì odds $= 0{,}25$.

Odds nhận giá trị trong $(0, +\infty)$, vẫn chưa phủ hết trục số thực. Nếu lấy **logarit** của odds, ta được **log-odds** (còn gọi là **logit**), nhận giá trị trên toàn bộ $(-\infty, +\infty)$, đúng bằng miền giá trị của điểm số:

$$\text{logit}(p) = \ln \frac{p}{1-p} \in (-\infty, +\infty)$$

Xác suất p	Odds $\frac{p}{1-p}$	Log-odds $\ln \frac{p}{1-p}$
0,1	0,111	−2,197
0,2	0,25	−1,386
0,5	1	0
0,8	4	1,386
0,9	9	2,197

Để ý tính **đối xứng**: p và $1 - p$ cho log-odds trái dấu nhau. Nhờ tính chất này, log-odds là một thang đo tự nhiên cho "nghiêng về lớp nào và nghiêng bao nhiêu".

Đây chính là chỗ để gắn điểm số tuyến tính vào. Ta đưa ra **giả thiết của Logistic Regression**: *log-odds của xác suất đậu là một hàm tuyến tính của các đặc trưng*:

$$\ln \frac{p}{1-p} = \mathbf{xw} = s$$

Giải ra xác suất: hàm sigmoid

Từ giả thiết trên, ta giải để tìm p theo s. Lấy hàm mũ hai vế, rồi biến đổi:

$$\frac{p}{1-p} = e^s \quad \Rightarrow \quad p = e^s(1-p) \quad \Rightarrow \quad p\,(1+e^s) = e^s \quad \Rightarrow \quad p = \frac{e^s}{1+e^s}$$

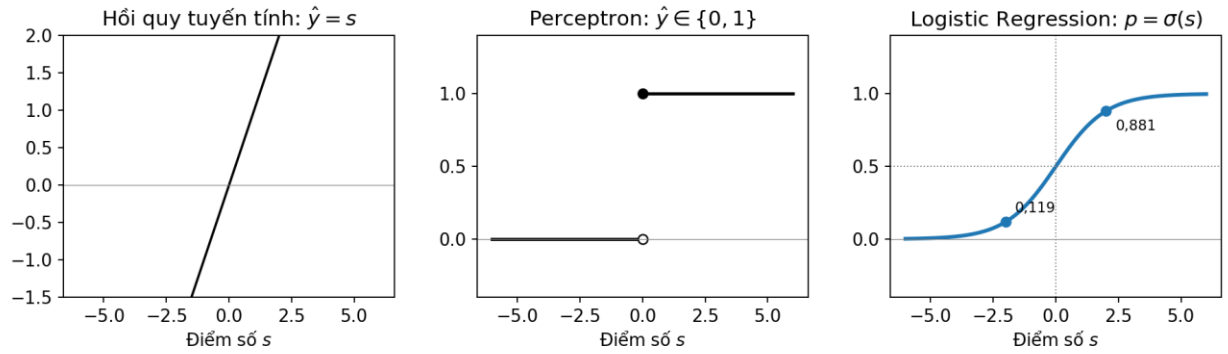
Chia cả tử và mẫu cho e^s, ta được dạng quen thuộc:

$$p = \sigma(s) = \frac{1}{1+e^{-s}}$$

Hàm σ này được gọi là **hàm sigmoid** (hay **hàm logistic**, vì thế mô hình mang tên Logistic Regression). Vậy mô hình của ta là

$$p = P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{xw}) = \frac{1}{1+e^{-\mathbf{xw}}}$$

Hình 2 vẽ ba hàm kích hoạt đặt cạnh nhau. Hồi quy tuyến tính giữ nguyên điểm số, Perceptron "bẻ" điểm số về 0 hoặc 1 tại $s = 0$, còn sigmoid chuyển **mượt** từ 0 lên 1.



Hình 2. Ba cách xử lý cùng một điểm số $s = \boldsymbol{xw}$*. Hồi quy tuyến tính giữ nguyên (không bị chặn), Perceptron cho nhãn* $0/1$ *(nhảy đột ngột tại* $s = 0$*), Logistic Regression cho xác suất trong* $(0,1)$ *(chuyển mượt).*

Có thể tóm gọn: **Logistic Regression = hồi quy tuyến tính (tính điểm số) + hàm sigmoid (đưa ra xác suất)**, song song với công thức "Perceptron = hồi quy tuyến tính + hàm dấu" ở bài trước.

Một số giá trị của sigmoid nên nhớ:

Điểm số s	-4	-2	-1	0	$0,5$	1	2	4
$\sigma(s)$	$0,018$	$0,119$	$0,269$	$0,5$	$0,622$	$0,731$	$0,881$	$0,982$

Và tính chất đối xứng $\sigma(-s) = 1 - \sigma(s)$, thể hiện qua các cặp $\sigma(2) = 0,881$ và $\sigma(-2) = 0,119$.

Điều gì được giữ lại, điều gì thay đổi so với Perceptron?

Giữ lại. Điểm số tuyến tính $s = \mathbf{xw}$ vẫn nguyên vẹn, và **đường biên quyết định vẫn là siêu phẳng** $\mathbf{xw} = 0$. Thật vậy, vì σ là hàm tăng và $\sigma(0) = 0,5$ nên

$$p \geq 0,5 \Leftrightarrow \sigma(s) \geq \sigma(0) \Leftrightarrow s \geq 0$$

Do đó quy tắc ngưỡng $0,5$ cho cùng một phân chia không gian đặc trưng như quy tắc "dấu của s" của Perceptron. Mọi hiểu biết hình học ở bài trước (vector pháp tuyến $\tilde{\mathbf{w}}$, hệ số chệch w_0 làm tịnh tiến, khoảng cách $|s|/\|\tilde{\mathbf{w}}\|$ đến đường biên) vẫn đúng.

Thay đổi. Đầu ra không còn là nhãn mà là **xác suất**, nên ta có thêm thông tin về độ tin cậy. Đồng thời, vì đầu ra đã thay đổi, **hàm mất mát cũng phải thay đổi**: ta không thể dùng mất mát $\max(0, -m)$ của Perceptron (vốn thiết kế cho nhãn cứng), cũng không nên dùng MSE. Ta sẽ xây dựng hàm mất mát phù hợp với xác suất ở mục 3.

Ví dụ số với dữ liệu sinh viên

Dùng lại $\mathbf{w} = [-2,5 \quad 1 \quad -1]^T$ của bài Perceptron. Điểm số và xác suất của cả 6 sinh viên được tính đồng thời bằng

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} -4{,}5 \\ -2{,}5 \\ -3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix}, \qquad \mathbf{p} = \sigma(\mathbf{s}) = \begin{bmatrix} \sigma(-4{,}5) \\ \sigma(-2{,}5) \\ \sigma(-3{,}5) \\ \sigma(2{,}5) \\ \sigma(3{,}5) \\ \sigma(6{,}5) \end{bmatrix} \approx \begin{bmatrix} 0{,}011 \\ 0{,}076 \\ 0{,}029 \\ 0{,}924 \\ 0{,}971 \\ 0{,}998 \end{bmatrix}$$

trong đó σ được áp dụng lên **từng phần tử** của $\mathbf{s}$. Bảng kết quả:

Sinh viên	s	$p = \sigma(s)$	$\hat{y}$ (ngưỡng 0,5)	y
1	$-4{,}5$	0,011	0	0
2	$-2{,}5$	0,076	0	0
3	$-3{,}5$	0,029	0	0
4	2,5	0,924	1	1
5	3,5	0,971	1	1
6	6,5	0,998	1	1

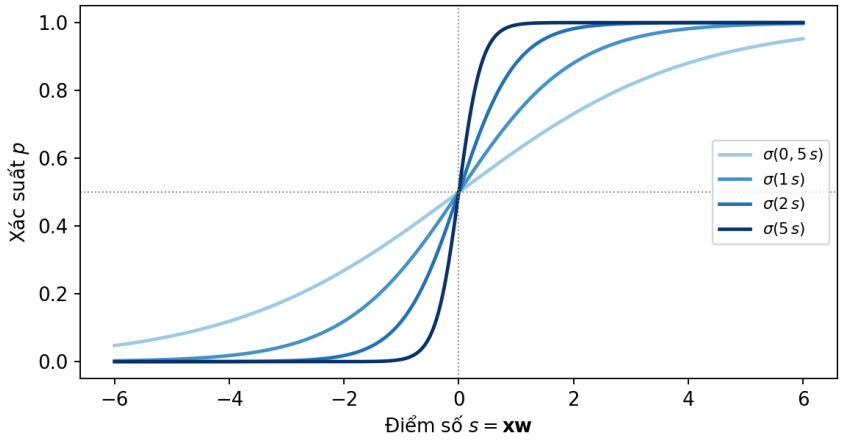
Nhãn dự đoán trùng với Perceptron ở mọi mẫu, nhưng giờ ta có thêm cột xác suất. Quay lại hai sinh viên ở mục 1.1: sinh viên A ($s_A = 6{,}5$) có $p = 0{,}998$ (gần như chắc chắn đậu), còn sinh viên B ($s_B = 0{,}5$) chỉ có $p = \sigma(0{,}5) \approx 0{,}622$ (nghiêng về đậu nhưng còn nhiều rủi ro). Perceptron cho cùng câu trả lời "đậu", còn Logistic Regression phân biệt được hai trường hợp.

Điểm khác biệt quan trọng: độ lớn của $\mathbf{w}$ bây giờ có ý nghĩa

Ở bài Perceptron (mục 2.3), ta đã thấy $\mathbf{w}$ và $c\,\mathbf{w}$ (với $c > 0$) cho **cùng một bộ phân loại**, vì nhân điểm số với một số dương không đổi dấu của nó. Với Logistic Regression, đường biên và nhãn dự đoán vẫn không đổi, nhưng **xác suất thì thay đổi**. Ví dụ với sinh viên B ($s = 0{,}5$):

Tham số	Điểm số s	Xác suất $\sigma(s)$
$0{,}5\,\mathbf{w}$	0,25	0,562
$\mathbf{w}$	0,5	0,622
$2\,\mathbf{w}$	1	0,731

Hình 3 cho thấy hiện tượng này: nhân điểm số với một hệ số lớn làm đường sigmoid **dốc hơn** (mô hình "quả quyết" hơn, xác suất nhanh chóng tiến về 0 hoặc 1 khi rời khỏi đường biên), còn hệ số nhỏ làm đường sigmoid **thoải hơn** (mô hình "dè dặt" hơn). Điều này có nghĩa là độ lớn của $\mathbf{w}$ điều khiển **độ tự tin** của mô hình, và việc chọn đúng độ lớn là một phần của quá trình huấn luyện, điều mà Perceptron hoàn toàn không phải quan tâm.



Hình 3. Đồ thị $\sigma(c\,s)$ với các hệ số c khác nhau. Mọi đường đều đi qua điểm $(0;\,0{,}5)$ nên đường biên không đổi, nhưng c càng lớn thì xác suất chuyển càng nhanh từ 0 lên 1.

> **Lưu ý.** Xác suất mà mô hình đưa ra chỉ đáng tin khi giả thiết "log-odds là hàm tuyến tính của đặc trưng" tương đối phù hợp với dữ liệu. Ta sẽ có cách kiểm tra chất lượng xác suất ở mục 5.

Câu hỏi dẫn dắt

Ta đã có mô hình $p = \sigma(\mathbf{xw})$, nhưng $\mathbf{w}$ vẫn chưa biết. Với Perceptron, ta chỉnh $\mathbf{w}$ khi gặp mẫu sai. Với một mô hình cho ra **xác suất**, "sai" không còn là "đúng/sai" mà là mức độ phù hợp giữa xác suất dự đoán và nhãn thật: nếu mẫu thực sự đậu mà mô hình chỉ gán $p = 0{,}1$ thì "sai nhiều", còn nếu gán $p = 0{,}9$ thì "sai ít". Ta cần một hàm mất mát đo được mức độ phù hợp này.

> **Câu hỏi: hàm mất mát nào đo tốt sự phù hợp giữa xác suất dự đoán và nhãn thật, và làm sao tìm $\mathbf{w}$ để nó nhỏ nhất?**

Câu trả lời dựa trên nguyên lý **hợp lý cực đại (maximum likelihood)** và là nội dung của mục 3. Trước đó, mục 2 sẽ mô tả chi tiết mô hình Logistic Regression.

Tóm tắt mục 1

- Perceptron có bốn hạn chế: chỉ cho **nhãn cứng** (không biết độ tin cậy), **ngưỡng cố định**, **dao động** khi dữ liệu chồng lấn, và hàm mất mát cùng hàm dấu **không mượt**.
- Bài toán được mô tả lại theo xác suất: dự đoán $p(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$ với nhãn $y \in \{0,1\}$. Khi đó $\mathbb{E}[y \mid \mathbf{x}] = p(\mathbf{x})$, và nhãn dự đoán lấy theo ngưỡng $0{,}5$.
- Hồi quy tuyến tính trên nhãn $0/1$ cho "xác suất" **ngoài** $[0,1]$ và không bám sát hình dạng chữ S của dữ liệu, nên cần một hàm nén.
- Giả thiết của Logistic Regression: **log-odds là hàm tuyến tính của đặc trưng**, $\ln\frac{p}{1-p} = \mathbf{xw}$. Giải ra ta được $p = \sigma(\mathbf{xw}) = \frac{1}{1+e^{-\mathbf{xw}}}$.
- Có thể nhớ gọn: **Logistic Regression = hồi quy tuyến tính + hàm sigmoid**.
- **Đường biên vẫn là $\mathbf{xw} = 0$**. Điểm khác là mô hình cho thêm độ tin cậy, và độ lớn của $\mathbf{w}$ nay điều khiển độ dốc của xác suất.

- Vì đầu ra đã đổi, hàm mất mát cũng phải đổi, và ta sẽ tìm nó bằng nguyên lý hợp lý cực đại ở mục 3.

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ và một sinh viên có $\mathbf{x} = [1 \quad 6 \quad 3]$, hãy tính điểm số s, xác suất $p = \sigma(s)$ và nhãn dự đoán $\hat{y}$.
2. Một mô hình cho xác suất đậu của một sinh viên là $p = 0{,}75$. Hãy tính odds và log-odds. Làm tương tự với $p = 0{,}25$ và nhận xét.
3. Nếu log-odds của một sinh viên là -2 thì xác suất đậu bằng bao nhiêu?
4. Giải thích vì sao việc thay hàm dấu bằng hàm sigmoid không làm thay đổi đường biên quyết định.
5. Với cùng đường biên, so sánh xác suất mô hình gán cho một mẫu có $s = 0{,}5$ khi dùng $\mathbf{w}$ và khi dùng $2\mathbf{w}$. Điều này nói lên điều gì về vai trò của độ lớn của $\mathbf{w}$?
6. Chuyển vector nhãn $\mathbf{y}_{\pm} = [-1 \quad -1 \quad -1 \quad +1 \quad +1 \quad +1]^T$ của bài Perceptron sang nhãn $\{0{,}1\}$ bằng công thức $y = \frac{y_{\pm}+1}{2}$.

Gợi ý đáp án

1. $s = -2{,}5 + 6 - 3 = 0{,}5$, $p = \sigma(0{,}5) \approx 0{,}622$, và vì $p \geq 0{,}5$ nên $\hat{y} = 1$ (dự đoán đậu).
2. Với $p = 0{,}75$: odds $= \frac{0{,}75}{0{,}25} = 3$ và log-odds $= \ln 3 \approx 1{,}099$. Với $p = 0{,}25$: odds $= \frac{0{,}25}{0{,}75} = \frac{1}{3}$ và log-odds $= \ln\frac{1}{3} \approx -1{,}099$. Hai log-odds **trái dấu, cùng độ lớn**, thể hiện tính đối xứng giữa p và $1 - p$.
3. $p = \sigma(-2) = \frac{1}{1+e^2} \approx 0{,}119$.
4. Vì σ là hàm tăng và $\sigma(0) = 0{,}5$, nên $\sigma(s) \geq 0{,}5 \Leftrightarrow s \geq 0$. Do đó ngưỡng $p \geq 0{,}5$ tương đương $\mathbf{xw} \geq 0$, tức là cùng một phân chia không gian đặc trưng với quy tắc dấu của Perceptron.
5. Với $\mathbf{w}$: $s = 0{,}5$ nên $p \approx 0{,}622$. Với $2\mathbf{w}$: $s = 1$ nên $p = \sigma(1) \approx 0{,}731$. Đường biên và nhãn dự đoán không đổi nhưng xác suất đổi, nên **độ lớn của w điều khiển độ tự tin** của mô hình (khác với Perceptron, nơi chỉ hướng của $\mathbf{w}$ mới quan trọng).
6. $y = \frac{y_{\pm}+1}{2}$ cho $\mathbf{y} = [0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1]^T$.

2. Mô hình Logistic Regression

Ở mục 1, ta đã đi đến ý tưởng: giữ điểm số tuyến tính $s = \mathbf{xw}$ rồi đưa nó qua hàm sigmoid để thu được xác suất. Mục này biến ý tưởng đó thành một mô hình hoàn chỉnh, gọi là **Logistic Regression** (hồi quy logistic). Dù mang tên "hồi quy", đây là một mô hình **phân loại**: nó hồi quy ra xác suất, rồi từ xác suất suy ra nhãn.

Trong mục này, ta trả lời ba câu hỏi:

- Logistic Regression nhận đầu vào, xử lý và đưa ra xác suất như thế nào?
- Mô hình được viết gọn bằng vector và ma trận ra sao?
- Đường biên quyết định ở đâu, và các trọng số w_j có ý nghĩa gì khi mô hình trả về xác suất?

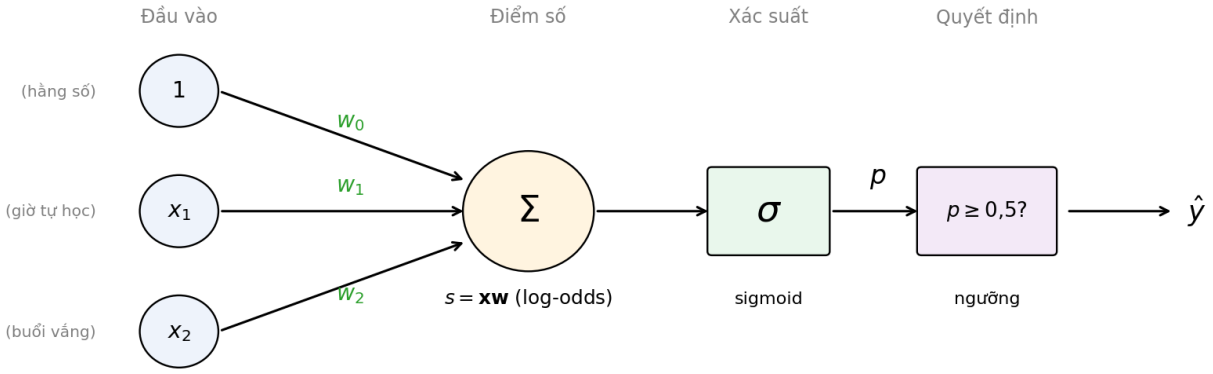
Ta tiếp tục dùng quy ước ký hiệu ở mục 1: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng** với phần tử đầu tiên bằng 1, vector tham số $\mathbf{w}$ là **vector cột**, và nhãn $y_i, \hat{y}_i \in \{0,1\}$.

2.1. Ý tưởng hoạt động của Logistic Regression

Có thể hình dung Logistic Regression như một "máy ước lượng khả năng" gồm **bốn bước**:

1. **Nhận đặc trưng.** Mô hình nhận vào $x_1, x_2, \dots, x_d$ của một mẫu.
2. **Tính điểm số.** Mỗi đặc trưng được nhân với trọng số rồi cộng lại, kèm hệ số chệch. Kết quả $s = \mathbf{xw}$ là một số thực bất kỳ, và như ta đã thấy ở mục 1.3, đó chính là **log-odds** của xác suất cần tìm.
3. **Tính xác suất.** Điểm số được "nén" bằng hàm sigmoid thành xác suất $p = \sigma(s) \in (0,1)$.
4. **Ra quyết định (nếu cần).** Nếu bài toán đòi hỏi một nhãn, ta so xác suất với một ngưỡng (mặc định 0,5) để được $\hat{y} \in \{0,1\}$.

So với Perceptron, quy trình chỉ khác ở bước 3: thay vì cắt thẳng điểm số thành nhãn, mô hình **giữ lại xác suất** và chỉ cắt thành nhãn ở bước cuối khi thật sự cần. Hình 4 minh họa quy trình này với ví dụ sinh viên đậu/rớt ($d = 2$).



Hình 4. Cấu trúc của Logistic Regression với hai đặc trưng. Các đầu vào được nhân với trọng số và cộng lại thành điểm số s (chính là log-odds), hàm sigmoid chuyển s thành xác suất p, và việc so sánh p với ngưỡng cho ra nhãn $\hat{y}$.

2.2. Hàm tổng hợp có trọng số, hệ số chệch và log-odds

Với một mẫu có vector đặc trưng $\mathbf{x} = [1 \quad x_1 \quad \cdots \quad x_d]$, mô hình tính **điểm số** là tổng có trọng số của các đặc trưng cộng với hệ số chệch, hoàn toàn giống Perceptron và hồi quy tuyến tính:

$$s = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d = w_0 + \sum_{j=1}^{d} w_j\, x_j = \mathbf{xw}, \qquad \mathbf{w} = [w_0 \quad w_1 \quad \cdots \quad w_d]^T$$

Điểm khác biệt nằm ở **cách diễn giải** điểm số: theo giả thiết ở mục 1.3, đó là **log-odds** của việc mẫu thuộc lớp dương:

$$s = \ln\frac{p}{1-p}$$

Cách diễn giải này giúp ta hiểu từng thành phần của s:

- **Hệ số chệch** w_0 là log-odds khi mọi đặc trưng bằng 0. Nói cách khác, $\sigma(w_0)$ là **xác suất nền** của mô hình. Với $w_0 = -2{,}5$ trong ví dụ sinh viên, một sinh viên học 0 giờ và vắng 0 buổi có xác suất đậu $\sigma(-2{,}5) \approx 0{,}076$, tức khoảng 7,6%.
- **Trọng số** w_j cho biết đặc trưng x_j đóng góp bao nhiêu vào log-odds: mỗi khi x_j tăng 1 đơn vị, log-odds tăng w_j. Nếu $w_j > 0$, đặc trưng đó làm tăng khả năng thuộc lớp dương, nếu $w_j < 0$ thì làm giảm, và nếu $w_j = 0$ thì đặc trưng không có ảnh hưởng.

Vì log-odds bằng s, ta có thể viết lại **odds** theo tích của các thừa số:

$$\frac{p}{1-p} = e^s = e^{w_0} \cdot e^{w_1x_1} \cdot e^{w_2x_2} \cdots e^{w_dx_d}$$

Nhận xét này rất hữu ích. Các đặc trưng **cộng** vào log-odds nhưng **nhân** vào odds. Mỗi đặc trưng đóng góp một thừa số độc lập $e^{w_jx_j}$ vào tỉ lệ cược, và ta sẽ khai thác điều này ở mục 2.5.

Ý nghĩa trong ví dụ sinh viên. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ ta có

$$s = -2{,}5 + 1 \cdot x_1 - 1 \cdot x_2$$

Mỗi giờ tự học làm log-odds đậu tăng 1, mỗi buổi vắng làm log-odds đậu giảm 1, và mô hình "bắt đầu" từ log-odds $-2{,}5$ (xác suất nền chỉ 7,6%). Sinh viên phải tích lũy đủ giờ học để vượt qua mức khởi điểm âm này thì xác suất đậu mới vượt 50%.

2.3. Hàm sigmoid, quy tắc dự đoán xác suất và nhãn

Hàm sigmoid và các tính chất

Để chuyển log-odds s thành xác suất p, mô hình dùng **hàm sigmoid** (hay hàm logistic):

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

Hình 5a vẽ đồ thị của σ. Ta tóm tắt các tính chất quan trọng, kèm chứng minh ngắn:

Tính chất	Biểu thức	Giải thích
Miền giá trị	$0 < \sigma(s) < 1$	Vì $e^{-s} > 0$ nên mẫu số > 1
Giá trị tại 0	$\sigma(0) = 0{,}5$	$e^0 = 1$ nên $\sigma(0) = \frac{1}{2}$
Giới hạn	$\sigma(s) \to 1$ khi $s \to +\infty$; $\sigma(s) \to 0$ khi $s \to -\infty$	$e^{-s} \to 0$ hoặc $e^{-s} \to +\infty$
Đơn điệu tăng	$\sigma'(s) > 0$	Xem chứng minh bên dưới
Đối xứng	$\sigma(-s) = 1 - \sigma(s)$	Xem chứng minh bên dưới
Hàm ngược	$\sigma^{-1}(p) = \ln\frac{p}{1-p}$	Chính là logit, như ở mục 1.3

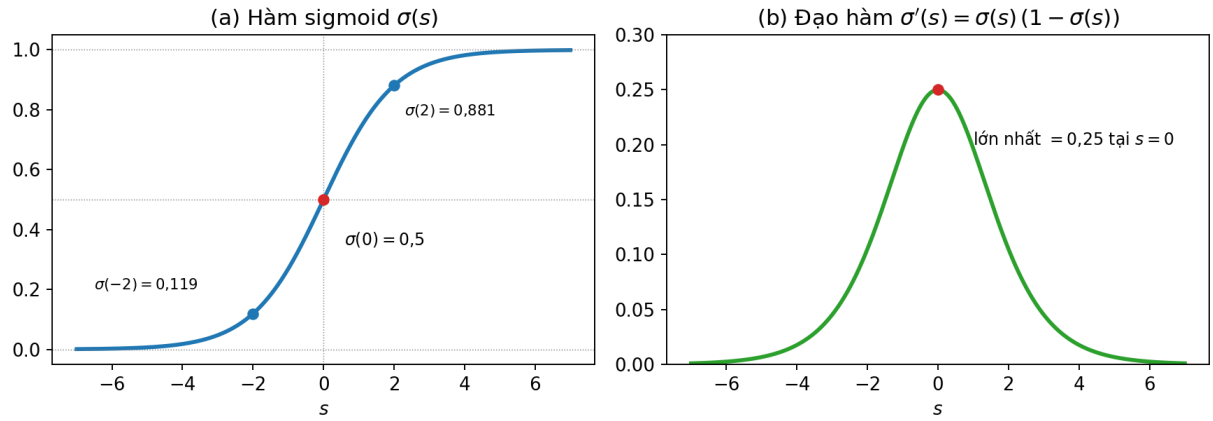
Tính chất đối xứng. Biến đổi $\sigma(-s)$:

$$\sigma(-s) = \frac{1}{1+e^{s}} = \frac{e^{-s}}{e^{-s}+1} = \frac{(1+e^{-s})-1}{1+e^{-s}} = 1 - \frac{1}{1+e^{-s}} = 1 - \sigma(s)$$

Đạo hàm. Viết $\sigma(s) = (1+e^{-s})^{-1}$ và dùng quy tắc đạo hàm hàm hợp:

$$\sigma'(s) = \frac{e^{-s}}{(1+e^{-s})^2} = \frac{1}{1+e^{-s}} \cdot \frac{e^{-s}}{1+e^{-s}} = \sigma(s)\left(1 - \sigma(s)\right)$$

Trong biến đổi cuối ta dùng $\frac{e^{-s}}{1+e^{-s}} = 1 - \frac{1}{1+e^{-s}} = 1 - \sigma(s)$. Kết quả $\sigma'(s) = \sigma(s)\big(1-\sigma(s)\big)$ cho thấy đạo hàm luôn dương, và vì tích $\sigma(1-\sigma)$ đạt lớn nhất khi $\sigma = 0{,}5$, đạo hàm lớn nhất bằng $0{,}25$ tại $s = 0$ (Hình 5b). Công thức rất gọn này sẽ được dùng liên tục ở mục 3 khi tính gradient.



Hình 5. (a) Hàm sigmoid $\sigma(s)$: đi qua $(0;\ 0{,}5)$, đối xứng qua điểm đó và bão hòa dần về 0 và 1. (b) Đạo hàm $\sigma'(s) = \sigma(s)\big(1 - \sigma(s)\big)$: lớn nhất tại $s = 0$ và tiến về 0 khi $|s|$ lớn.

Hình 5b cho thấy một đặc điểm đáng nhớ: khi $|s|$ lớn (mẫu nằm xa đường biên), đạo hàm gần bằng 0, tức là xác suất gần như **không đổi** dù điểm số thay đổi thêm. Ngược lại, quanh đường biên ($s \approx 0$), xác suất thay đổi nhanh nhất.

Xác suất của cả hai lớp

Mô hình cho xác suất của lớp dương, và vì xác suất của hai lớp cộng lại bằng 1, ta có luôn xác suất của lớp âm, nhờ tính đối xứng của sigmoid:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{xw}), \qquad P(y = 0 \mid \mathbf{x}) = 1 - \sigma(\mathbf{xw}) = \sigma(-\mathbf{xw})$$

Hai trường hợp này có thể gộp thành **một công thức duy nhất** cho xác suất mà mô hình gán cho nhãn thật $y \in \{0{,}1\}$:

$$P(y \mid \mathbf{x}) = p^{\,y}\,(1 - p)^{1-y}, \qquad p = \sigma(\mathbf{xw})$$

Thật vậy, khi $y = 1$ biểu thức bằng $p^1(1 - p)^0 = p$, còn khi $y = 0$ nó bằng $p^0(1 - p)^1 = 1 - p$. Công thức gọn này (phân phối Bernoulli) là nền tảng của hàm mất mát ở mục 3.

Quy tắc dự đoán nhãn

Từ xác suất, nhãn dự đoán được lấy bằng cách so với ngưỡng 0,5:

$$\hat{y} = \begin{cases} 1 & \text{nếu } p \geq 0{,}5 \\ 0 & \text{nếu } p < 0{,}5 \end{cases}$$

Vì $p = \sigma(s)$ là hàm tăng và $\sigma(0) = 0{,}5$, quy tắc này tương đương với quy tắc dựa vào dấu của điểm số:

$$\hat{y} = 1 \Leftrightarrow p \geq 0{,}5 \Leftrightarrow s = \mathbf{xw} \geq 0$$

Ta thống nhất với bài Perceptron quy ước **điểm nằm đúng trên đường biên** ($p = 0{,}5$, $s = 0$) được gán nhãn 1.

Ví dụ. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$:

- Sinh viên học 7 giờ, vắng 3 buổi, $\mathbf{x} = [1 \quad 7 \quad 3]$: $s = -2{,}5 + 7 - 3 = 1{,}5$, nên $p = \sigma(1{,}5) \approx 0{,}818$ và $\hat{y} = 1$.
- Sinh viên học 3 giờ, vắng 4 buổi, $\mathbf{x} = [1 \quad 3 \quad 4]$: $s = -3{,}5$, nên $p = \sigma(-3{,}5) \approx 0{,}029$ và $\hat{y} = 0$.
- Sinh viên học 4,5 giờ, vắng 2 buổi, $\mathbf{x} = [1 \quad 4{,}5 \quad 2]$: $s = 0$, nên $p = 0{,}5$ và theo quy ước $\hat{y} = 1$.

Ngưỡng quyết định tùy chọn

Một lợi thế lớn của việc có xác suất là ta **không bị buộc dùng ngưỡng** 0,5. Với ngưỡng $\tau \in (0,1)$ bất kỳ, ta dự đoán lớp dương khi $p \geq \tau$. Vì σ là hàm tăng, điều này tương đương với việc so điểm số với **logit của ngưỡng**:

$$\hat{y} = 1 \Leftrightarrow p \geq \tau \Leftrightarrow s \geq \ln\frac{\tau}{1-\tau}$$

Ngưỡng τ	0,2	0,3	0,5	0,7	0,8
Điểm số tối thiểu $\ln\frac{\tau}{1-\tau}$	$-1{,}386$	$-0{,}847$	0	0,847	1,386

Quay lại tình huống ở mục 1.1: nhà trường muốn **cảnh báo học vụ** cho mọi sinh viên có khả năng rớt trên 30%, tức là khi $P(\text{đậu}) < 0{,}7$. Theo bảng, điều đó tương đương $s < 0{,}847$. Áp dụng cho 6 sinh viên của ví dụ (với xác suất đậu đã tính ở mục 1.3):

Sinh viên	p (đậu)	$\hat{y}$ với ngưỡng 0,5	Có cảnh báo không? ($p < 0{,}7$)
1	0,011	0	Có
2	0,076	0	Có
3	0,029	0	Có
4	0,924	1	Không
5	0,971	1	Không
6	0,998	1	Không

Cả 6 sinh viên đều đã nằm xa ngưỡng nên kết quả không khác nhau ở đây. Nhưng với sinh viên B ở mục 1.1 ($s_B = 0{,}5$, $p \approx 0{,}622$), mô hình dự đoán "đậu" theo ngưỡng 0,5, song vì $p < 0{,}7$ nên vẫn **bị cảnh báo**. Điều mà Perceptron không làm được thì Logistic Regression làm được nhờ có xác suất. Ta sẽ phân tích kỹ hơn tác động của việc đổi ngưỡng lên precision và recall ở mục 5.3.

2.4. Biểu diễn vector và ma trận của mô hình

Một mẫu

Với một mẫu, mô hình gồm hai bước: nhân vector hàng $\mathbf{x}_i$ với vector cột $\mathbf{w}$ để được điểm số, rồi đưa qua sigmoid để được xác suất:

$$s_i = \mathbf{x}_i \mathbf{w}, \qquad p_i = \sigma(s_i), \qquad \hat{y}_i = \begin{cases} 1 & \text{nếu } p_i \geq 0{,}5 \\ 0 & \text{nếu } p_i < 0{,}5 \end{cases}$$

Về kích thước, $\mathbf{x}_i$ có kích thước $1 \times (d+1)$ và $\mathbf{w}$ có kích thước $(d+1) \times 1$ nên tích $\mathbf{x}_i \mathbf{w}$ là một **số**.

Toàn bộ n mẫu cùng lúc

Vì mỗi hàng của ma trận đặc trưng $\mathbf{X}$ chính là một vector $\mathbf{x}_i$, một phép nhân ma trận cho điểm số của **tất cả các mẫu**, và ta áp dụng sigmoid **lên từng phần tử**:

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} \mathbf{x}_1 \mathbf{w} \\ \mathbf{x}_2 \mathbf{w} \\ \vdots \\ \mathbf{x}_n \mathbf{w} \end{bmatrix} = \begin{bmatrix} s_1 \\ s_2 \\ \vdots \\ s_n \end{bmatrix}, \qquad \mathbf{p} = \sigma(\mathbf{X}\mathbf{w}) = \begin{bmatrix} \sigma(s_1) \\ \sigma(s_2) \\ \vdots \\ \sigma(s_n) \end{bmatrix} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}$$

Vector nhãn dự đoán $\hat{\mathbf{y}}$ có các phần tử $\hat{y}_i = 1$ nếu $p_i \geq 0{,}5$ và $\hat{y}_i = 0$ nếu ngược lại. Cách viết này có ba lợi ích: công thức ngắn gọn, thống nhất với hồi quy tuyến tính và Perceptron (chỉ khác hàm kích hoạt), và khi lập trình ta tính cho cả tập dữ liệu bằng một phép nhân ma trận. Bảng kích thước:

Đại lượng	Biểu thức	Kích thước
Ma trận đặc trưng	$\mathbf{X}$	$n \times (d+1)$
Vector tham số	$\mathbf{w}$	$(d+1) \times 1$
Vector điểm số (log-odds)	$\mathbf{s} = \mathbf{X}\mathbf{w}$	$n \times 1$
Vector xác suất	$\mathbf{p} = \sigma(\mathbf{s})$	$n \times 1$
Vector nhãn dự đoán	$\hat{\mathbf{y}}$	$n \times 1$
Vector nhãn thực	$\mathbf{y}$	$n \times 1$

Ví dụ số với dữ liệu sinh viên

Quay lại $\mathbf{X}$ và $\mathbf{y}$ ở mục 1, với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$:

$$\mathbf{s} = \mathbf{X}\mathbf{w} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 1 \\ 1 & 8 & 2 \\ 1 & 9 & 0 \end{bmatrix} \begin{bmatrix} -2{,}5 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} -4{,}5 \\ -2{,}5 \\ -3{,}5 \\ 2{,}5 \\ 3{,}5 \\ 6{,}5 \end{bmatrix} \Rightarrow \mathbf{p} \approx \begin{bmatrix} 0{,}011 \\ 0{,}076 \\ 0{,}029 \\ 0{,}924 \\ 0{,}971 \\ 0{,}998 \end{bmatrix} \Rightarrow \hat{\mathbf{y}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \mathbf{y}$$

Nhãn dự đoán trùng nhãn thực ở cả 6 mẫu, và ta có thêm vector xác suất $\mathbf{p}$.

Xác suất mà mô hình gán cho nhãn thật

Có một đại lượng sẽ trở thành trung tâm của mục 3: **xác suất mô hình gán cho nhãn thật** của từng mẫu, $q_i = P(y_i \mid \mathbf{x}_i) = p_i^{y_i}(1-p_i)^{1-y_i}$. Với $y_i = 1$, $q_i = p_i$; với $y_i = 0$, $q_i = 1 - p_i$:

$$\mathbf{q} \approx \begin{bmatrix} 1-0{,}011 \\ 1-0{,}076 \\ 1-0{,}029 \\ 0{,}924 \\ 0{,}971 \\ 0{,}998 \end{bmatrix} \approx \begin{bmatrix} 0{,}989 \\ 0{,}924 \\ 0{,}971 \\ 0{,}924 \\ 0{,}971 \\ 0{,}998 \end{bmatrix}$$

Mô hình tốt khi mọi q_i đều gần 1 (mô hình gán xác suất cao cho điều đã xảy ra). Nếu coi các mẫu độc lập, xác suất mô hình gán cho **toàn bộ tập nhãn quan sát được** là tích của các q_i:

$$\prod_{i=1}^{6} q_i \approx 0{,}989 \cdot 0{,}924 \cdot 0{,}971 \cdot 0{,}924 \cdot 0{,}971 \cdot 0{,}998 \approx 0{,}795$$

Tích này càng gần 1 thì mô hình càng phù hợp với dữ liệu. Chọn **w** để tích này **lớn nhất** chính là nguyên lý **hợp lý cực đại**, nội dung của mục 3.

2.5. Đường biên quyết định và ý nghĩa của trọng số

Đường biên quyết định

Mô hình dự đoán $\hat{y} = 1$ khi $p \geq 0{,}5$, tức là khi $\mathbf{xw} \geq 0$. Do đó ranh giới giữa hai nhãn là tập hợp các điểm có $p = 0{,}5$, hay điểm số bằng đúng 0:

$$\{\mathbf{x}: \mathbf{xw} = 0\} \quad \Leftrightarrow \quad w_0 + w_1 x_1 + \cdots + w_d x_d = 0$$

Đây **giống hệt** đường biên của Perceptron, và mọi kết quả hình học của bài trước đều còn đúng: đường biên là một **siêu phẳng** (đường thẳng khi $d = 2$), vector $\tilde{\mathbf{w}} = [w_1 \quad \cdots \quad w_d]^T$ là **vector pháp tuyến** chỉ về phía xác suất tăng, hệ số chệch w_0 làm đường biên **tịnh tiến**, và các trọng số làm đường biên **xoay**. Với $d = 2$ và $w_2 \neq 0$:

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2} x_1$$

Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$, đường biên là $x_2 = x_1 - 2{,}5$. Vì đường biên vẫn là siêu phẳng, Logistic Regression vẫn là **bộ phân loại tuyến tính**, và nó không phân loại được những bài toán như XOR (ta sẽ quay lại ở mục 4.5).

Các đường đồng mức xác suất

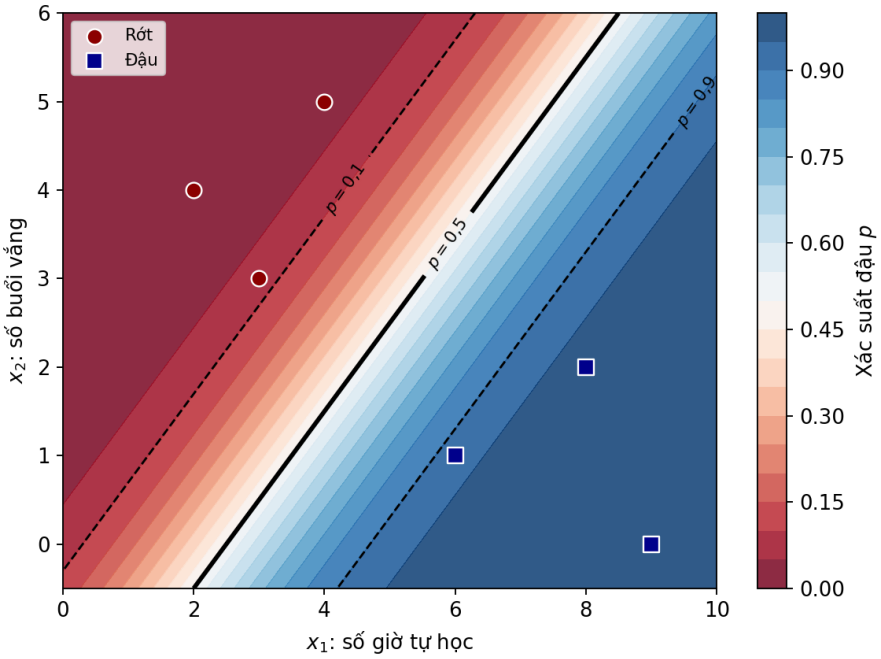
Điều **mới** so với Perceptron là mô hình còn cho biết xác suất ở mọi điểm, không chỉ phía nào của đường biên. Tập hợp các điểm có cùng xác suất $p = c$ thỏa mãn

$$\sigma(\mathbf{xw}) = c \Leftrightarrow \mathbf{xw} = \ln\frac{c}{1-c}$$

Đây là một siêu phẳng **song song** với đường biên (cùng vector pháp tuyến $\tilde{\mathbf{w}}$, chỉ khác hằng số bên phải). Như vậy, không gian đặc trưng được chia thành các "lớp" xác suất song song nhau, xác suất tăng dần từ phía rớt sang phía đậu (Hình 6). Với **w** ở ví dụ, đường $p = 0{,}9$ ứng với $s =$

$\ln 9 \approx 2{,}197$ và đường $p = 0{,}1$ ứng với $s = -2{,}197$. Khoảng cách từ đường biên đến mỗi đường đó là

$$\frac{|s|}{\| \tilde{\mathbf{w}} \|} = \frac{2{,}197}{\sqrt{1^2 + (-1)^2}} = \frac{2{,}197}{\sqrt{2}} \approx 1{,}55$$



Hình 6. Bản đồ xác suất đậu của mô hình $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ *trên mặt phẳng đặc trưng. Đường liền là đường biên* $p = 0{,}5$*, hai đường đứt là* $p = 0{,}1$ *và* $p = 0{,}9$*. Xác suất chuyển mượt từ vùng rớt (đỏ) sang vùng đậu (xanh), nhanh nhất quanh đường biên.*

Nếu $\| \tilde{\mathbf{w}} \|$ lớn, các đường đồng mức nằm **sát nhau** (xác suất chuyển nhanh, mô hình "quả quyết"). Nếu $\| \tilde{\mathbf{w}} \|$ nhỏ, chúng nằm **thưa** (xác suất chuyển chậm, mô hình "dè dặt"). Đây chính là điều ta đã thấy ở Hình 3 của mục 1, nay nhìn dưới dạng hình học.

Ý nghĩa của trọng số: odds ratio

Ở Perceptron, trọng số w_j chỉ cho biết đặc trưng đẩy điểm số lên hay xuống. Với Logistic Regression, ta có một cách diễn giải **định lượng** rất đẹp. Giả sử đặc trưng x_j tăng thêm 1 đơn vị, các đặc trưng khác giữ nguyên. Khi đó điểm số tăng thêm w_j, và odds được nhân với

$$\frac{\text{odds sau}}{\text{odds trước}} = \frac{e^{\,s+w_j}}{e^{\,s}} = e^{\,w_j}$$

Đại lượng e^{w_j} được gọi là **tỉ số odds (odds ratio)** của đặc trưng x_j. Cách đọc:

- $w_j > 0$: $e^{w_j} > 1$, odds **tăng** theo hệ số e^{w_j}.
- $w_j < 0$: $e^{w_j} < 1$, odds **giảm** theo hệ số e^{w_j}.
- $w_j = 0$: $e^{w_j} = 1$, odds không đổi.

Ví dụ. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$:

- Mỗi giờ tự học thêm nhân odds đậu với $e^1 \approx 2{,}718$, tức là **gần gấp ba**.
- Mỗi buổi vắng thêm nhân odds đậu với $e^{-1} \approx 0{,}368$, tức là odds **giảm khoảng** 63%.

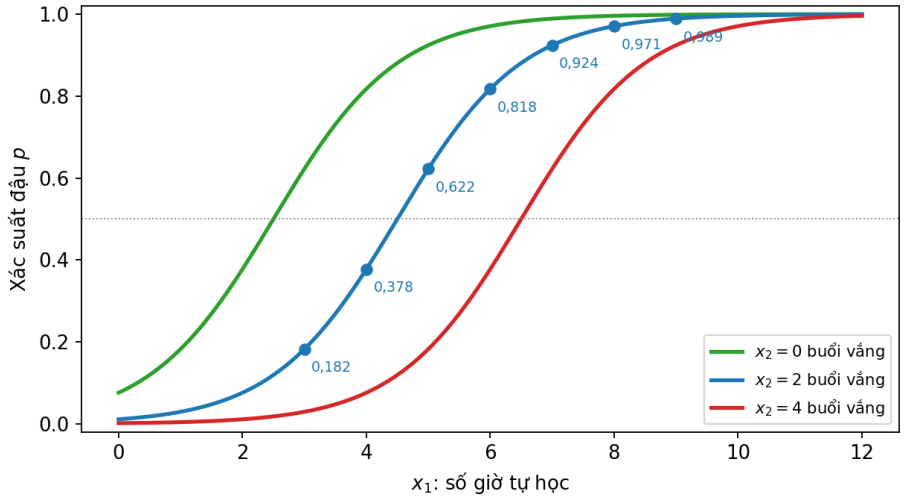
Ta kiểm chứng với sinh viên B ($x_1 = 5$, $x_2 = 2$, $s = 0{,}5$, odds $= e^{0{,}5} \approx 1{,}649$, $p \approx 0{,}622$). Nếu sinh viên này học thêm 1 giờ ($x_1 = 6$): $s = 1{,}5$, odds $= e^{1{,}5} \approx 4{,}482 = 1{,}649 \times 2{,}718$ và $p \approx 0{,}818$. Nếu thay vào đó vắng thêm 1 buổi ($x_2 = 3$): $s = -0{,}5$, odds $= e^{-0{,}5} \approx 0{,}607 = 1{,}649 \times 0{,}368$ và $p \approx 0{,}378$.

Odds ratio không đổi, nhưng xác suất thì thay đổi

Cần cẩn thận một điểm hay bị hiểu nhầm: **odds** (và log-odds) thay đổi theo hệ số **cố định** khi x_j tăng 1, nhưng **xác suất** thì **không**. Bảng sau cố định $x_2 = 2$ và cho x_1 tăng dần:

x_1	3	4	5	6	7	8	9
$s = x_1 - 4{,}5$	$-1{,}5$	$-0{,}5$	0,5	1,5	2,5	3,5	4,5
Odds $= e^s$	0,223	0,607	1,649	4,482	12,18	33,12	90,02
Xác suất p	0,182	0,378	0,622	0,818	0,924	0,971	0,989
Tăng của p so với cột trước		+0,196	+0,244	+0,196	+0,106	+0,047	+0,018

Mỗi bước, odds luôn được nhân thêm 2,718 (hàng "Odds" là một cấp số nhân), nhưng mức tăng của xác suất lại **thay đổi mạnh**: lớn nhất ở quanh $p = 0{,}5$ rồi nhỏ dần khi p tiến sát 1 (Hình 7). Điều này giải thích vì sao một giờ học thêm có tác dụng rất lớn với sinh viên đang ở mức trung bình nhưng gần như không đổi gì với sinh viên đã gần chắc chắn đậu.



Hình 7. Xác suất đậu theo số giờ tự học với số buổi vắng cố định. Ba đường ứng với $x_2 = 0{,}2{,}4$ là cùng một đường cong sigmoid tịnh tiến theo x_1. Các số trên đường $x_2 = 2$ là xác suất tại từng giá trị nguyên của x_1.

Hiện tượng này được tính bằng đạo hàm. Vì $p = \sigma(s)$ và $\sigma' = \sigma(1-\sigma)$, theo quy tắc đạo hàm hàm hợp:

$$\frac{\partial p}{\partial x_j} = \sigma'(s) \cdot \frac{\partial s}{\partial x_j} = p\,(1-p)\,w_j$$

Mức tăng của xác suất khi x_j tăng một lượng nhỏ **phụ thuộc vào chính** p: nó lớn nhất khi $p = 0{,}5$, bằng $\frac{w_j}{4}$, và gần bằng 0 khi p tiến sát 0 hoặc 1. Với $w_1 = 1$, mỗi giờ học thêm làm xác suất tăng tối đa khoảng $0{,}25$, sát với mức tăng lớn nhất $+0{,}244$ trong bảng.

Một vài lưu ý khi diễn giải trọng số

- **Dấu** của w_j cho biết hướng ảnh hưởng, **độ lớn** cho biết cường độ qua e^{w_j}. Đây là lợi thế diễn giải mà Perceptron không có.
- Đơn vị của w_j gắn với đơn vị của x_j (w_1 tính theo giờ, w_2 theo buổi vắng), nên **chỉ so sánh độ lớn các trọng số một cách công bằng khi các đặc trưng cùng thang đo** (ví dụ sau khi chuẩn hóa, mục 4.2).
- Nhân đôi $\mathbf{w}$ không đổi đường biên nhưng đổi odds ratio từ e^{w_j} thành e^{2w_j}, tức là "tác động" của đặc trưng mạnh hơn. Đó là lý do độ lớn của $\mathbf{w}$ có ý nghĩa với Logistic Regression.

So sánh nhanh với Perceptron

	Perceptron	Logistic Regression
Điểm số	$s = \mathbf{xw}$	$s = \mathbf{xw}$ (giống hệt, là log-odds)
Hàm kích hoạt	Hàm dấu / hàm bước	Sigmoid $\sigma(s)$
Đầu ra	Nhãn cứng	Xác suất $p \in (0,1)$, từ đó suy ra nhãn
Đường biên	$\mathbf{xw} = 0$	$\mathbf{xw} = 0$ (tức $p = 0{,}5$), cùng với các đường đồng mức $p = c$ song song
Độ lớn của w	Không ảnh hưởng đến bộ phân loại	Điều khiển độ tự tin của xác suất
Ý nghĩa của w_j	Hướng ảnh hưởng	Hướng và cường độ: e^{w_j} là odds ratio
Đạo hàm của hàm kích hoạt	0 hầu như mọi nơi	$\sigma(1-\sigma) > 0$ ở mọi nơi

Tóm tắt mục 2

- Logistic Regression tính **điểm số** $s = \mathbf{xw}$, được hiểu là **log-odds**, rồi dùng **hàm sigmoid** để thu được xác suất $p = \sigma(\mathbf{xw}) = \frac{1}{1+e^{-\mathbf{xw}}}$. Nhãn dự đoán lấy từ ngưỡng: $\hat{y} = 1$ khi $p \geq 0{,}5$, tương đương $\mathbf{xw} \geq 0$.
- Có thể nhớ gọn: **Logistic Regression = hồi quy tuyến tính + hàm sigmoid**.
- Sigmoid có các tính chất $\sigma(0) = 0{,}5$, $\sigma(-s) = 1 - \sigma(s)$ và $\sigma'(s) = \sigma(s)\big(1-\sigma(s)\big)$. Xác suất của nhãn thật là $p^y(1-p)^{1-y}$.

- Ngưỡng có thể chọn tùy ý: $\hat{y} = 1 \Leftrightarrow \mathbf{xw} \geq \ln\frac{\tau}{1-\tau}$.
- Với toàn bộ dữ liệu: $\mathbf{p} = \sigma(\mathbf{Xw})$, trong đó σ được áp dụng lên từng phần tử.
- **Đường biên** là siêu phẳng $\mathbf{xw} = 0$ như Perceptron, và các đường đồng mức xác suất là các siêu phẳng **song song** với nó.
- Trọng số có ý nghĩa **định lượng**: e^{w_j} là hệ số nhân của odds khi x_j tăng 1 đơn vị. Odds ratio là hằng số, còn mức thay đổi của xác suất là $p(1-p)w_j$, lớn nhất ở quanh $p = 0{,}5$.

Câu hỏi kiểm tra nhanh

1. Với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ và $\mathbf{x} = [1 \quad 7 \quad 3]$, hãy tính s, p và $\hat{y}$. Tính thêm odds của mẫu này.
2. Tính $\sigma'(0)$ và $\sigma'(2)$ bằng công thức $\sigma'(s) = \sigma(s)\big(1-\sigma(s)\big)$. Giá trị nào lớn hơn, và điều đó nói lên gì về ảnh hưởng của điểm số lên xác suất ở hai vị trí?
3. Nhà trường dùng ngưỡng $\tau = 0{,}7$ để quyết định sinh viên "an toàn". Với $\mathbf{w}$ như ở câu 1 và sinh viên có $\mathbf{x} = [1 \quad 6 \quad 3]$, mô hình dự đoán nhãn nào với ngưỡng 0,5 và với ngưỡng 0,7?
4. Với cùng $\mathbf{w}$, sinh viên có $s = 0{,}5$ (odds $\approx 1{,}649$) vắng thêm một buổi. Tính odds và xác suất mới, và kiểm tra bằng odds ratio e^{w_2}.
5. Một mô hình có $w_1 = 0{,}7$ cho đặc trưng "số giờ học". Tính odds ratio khi học thêm 1 giờ và khi học thêm 2 giờ.
6. Hai sinh viên đều bị dự đoán $\hat{y} = 1$, một người có $p = 0{,}55$ và một người có $p = 0{,}99$. Perceptron phân biệt được hai trường hợp này không? Logistic Regression thì sao?
7. Với sinh viên 2 trong ví dụ ($y_2 = 0$, $p_2 \approx 0{,}076$), xác suất mô hình gán cho **nhãn thật** của sinh viên này là bao nhiêu?

Gợi ý đáp án

1. $s = -2{,}5 + 7 - 3 = 1{,}5$, $p = \sigma(1{,}5) \approx 0{,}818$ và $\hat{y} = 1$. Odds $= e^{1{,}5} \approx 4{,}482$ (kiểm tra: $\frac{0{,}818}{0{,}182} \approx 4{,}49$).
2. $\sigma'(0) = 0{,}5 \cdot 0{,}5 = 0{,}25$. $\sigma'(2) = \sigma(2)\big(1-\sigma(2)\big) \approx 0{,}881 \cdot 0{,}119 \approx 0{,}105$. Giá trị tại 0 lớn hơn: xác suất nhạy với điểm số nhất khi mẫu nằm sát đường biên, và kém nhạy hơn khi mẫu đã nằm xa.
3. $s = -2{,}5 + 6 - 3 = 0{,}5$ nên $p \approx 0{,}622$. Với ngưỡng 0,5: $\hat{y} = 1$. Với ngưỡng 0,7: $p < 0{,}7$ nên $\hat{y} = 0$ (cần $s \geq 0{,}847$ mới đạt, mà $s = 0{,}5$).
4. Sau khi vắng thêm một buổi: $s = 0{,}5 - 1 = -0{,}5$, odds $= e^{-0{,}5} \approx 0{,}607$, $p = \sigma(-0{,}5) \approx 0{,}378$. Theo odds ratio: $1{,}649 \cdot e^{-1} \approx 1{,}649 \cdot 0{,}368 \approx 0{,}607$, khớp.
5. Học thêm 1 giờ: $e^{0{,}7} \approx 2{,}014$ (odds gấp khoảng 2 lần). Học thêm 2 giờ: $e^{1{,}4} \approx 4{,}055$ (bằng $2{,}014^2$, vì các tác động nhân lên với nhau).
6. Perceptron **không** phân biệt: cả hai đều chỉ nhận nhãn $+1$. Logistic Regression cho xác suất 0,55 (gần như ngang ngửa, rất không chắc chắn) và 0,99 (gần như chắc chắn), nên phân biệt rõ.
7. $q_2 = 1 - p_2 \approx 1 - 0{,}076 = 0{,}924$, vì nhãn thật là 0 nên xác suất của nhãn thật là $1 - p$.

3. Huấn luyện mô hình Logistic Regression

Ở mục 2, ta đã có mô hình $p = \sigma(\mathbf{xw})$, nhưng vector tham số $\mathbf{w}$ vẫn chưa biết. Ta cần một quá trình **huấn luyện** để máy tính tự tìm $\mathbf{w}$ từ dữ liệu. Ta tiếp tục bám theo khung đã học ở hồi quy tuyến tính và Perceptron:

$$\text{Mô hình} + \text{Hàm mất mát} + \text{Thuật toán tối ưu} = \text{Huấn luyện}$$

Mô hình đã có (mục 2). Mục này làm hai thành phần còn lại và trả lời ba câu hỏi:

- **Hàm mất mát nào** đo tốt sự phù hợp giữa xác suất dự đoán và nhãn thật?
- **Gradient** của hàm mất mát đó là gì, và quy tắc cập nhật $\mathbf{w}$ trông như thế nào?
- **Thuật toán** chạy ra sao, và các lựa chọn như tốc độ học hay cách chia dữ liệu ảnh hưởng thế nào?

Ta dùng quy ước ký hiệu cũ: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn $y_i \in \{0,1\}$, điểm số $s_i = \mathbf{x}_i\mathbf{w}$ và xác suất $p_i = \sigma(s_i)$.

3.1. Vì sao cần hàm mất mát mới?

Hai hàm mất mát đã gặp đều không phù hợp với một mô hình cho ra xác suất.

Không dùng mất mát Perceptron

Mất mát Perceptron $\max(0, -m)$ được thiết kế cho **nhãn cứng**: nó chỉ phạt mẫu bị sai, và bằng 0 ngay khi mẫu đúng. Với mô hình xác suất, ta muốn nhiều hơn: nếu mẫu thực sự đậu mà mô hình gán $p = 0{,}51$ thì mô hình "đúng nhưng chưa chắc", và ta vẫn muốn đẩy p lên cao hơn. Mất mát Perceptron không làm được điều này, và nó cũng không có ý nghĩa xác suất.

Không nên dùng MSE

Ta có thể thử đo sai số giữa xác suất dự đoán và nhãn bằng bình phương sai số, giống hồi quy tuyến tính:

$$J_{\text{MSE}}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - \sigma(s_i)\big)^2$$

Cách này **về hình thức** hợp lý (nhãn và xác suất đều nằm trong $[0,1]$) nhưng có một vấn đề nghiêm trọng khi kết hợp với sigmoid. Xét một mẫu có nhãn thật $y = 1$, và gọi ℓ là mất mát của mẫu đó. Theo quy tắc đạo hàm hàm hợp, đạo hàm theo điểm số s là

$$\frac{\partial \ell_{\text{MSE}}}{\partial s} = -2\,(1-p)\cdot\sigma'(s) = -2\,(1-p)\cdot p\,(1-p)$$

Vì xuất hiện thừa số $\sigma'(s) = p(1-p)$, gradient **tiến về** 0 **khi** p **tiến về** 0 **hoặc** 1. Hệ quả là khi mô hình **sai một cách chắc chắn**, gradient lại cực nhỏ. Ví dụ, mẫu có $y = 1$ nhưng mô hình cho $s = -6$, tức $p = \sigma(-6) \approx 0{,}0025$ (mô hình tin chắc là rớt, trong khi thực tế đậu). Khi đó:

$$\left|\frac{\partial \ell_{\text{MSE}}}{\partial s}\right| = 2 \cdot 0{,}9975 \cdot 0{,}0025 \cdot 0{,}9975 \approx 0{,}005$$

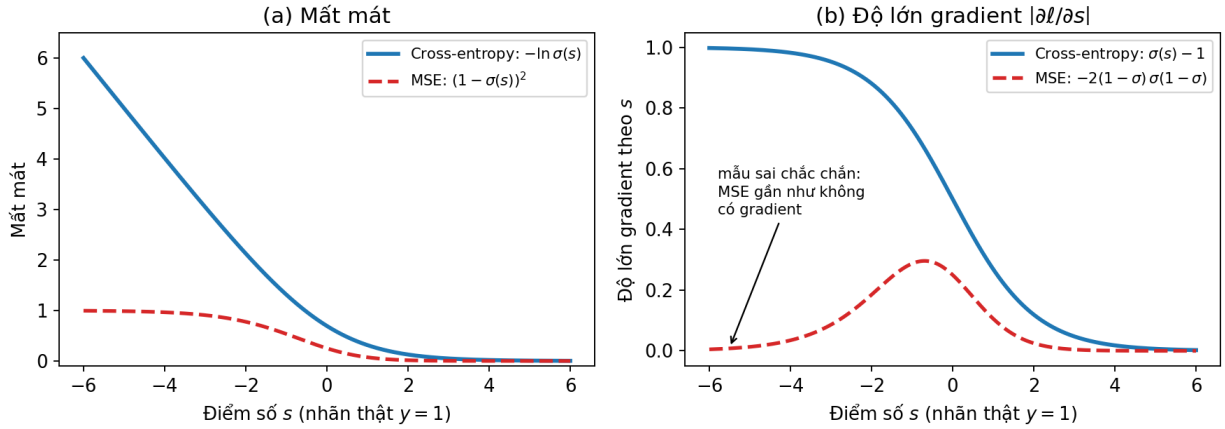
Gần như bằng 0: thuật toán gần như **không học** từ chính những mẫu mà nó sai nặng nhất. Hình 8 cho thấy điều đó. Ngoài ra, MSE kết hợp với sigmoid là hàm **không lồi** theo **w**, có thể có những vùng phẳng khiến Gradient Descent bị kẹt.

Hàm mất mát cần có gì?

Từ hai phân tích trên, ta đặt ra các yêu cầu cho hàm mất mát của một mô hình xác suất:

Yêu cầu	Lý do
Phạt nặng khi sai chắc chắn	Mô hình gán xác suất thấp cho điều đã xảy ra phải bị phạt rất nặng
Gradient lớn khi sai nặng	Để thuật toán học nhanh chính từ những mẫu sai nhất, không "tắt" như MSE
Trơn (khả vi)	Để dùng được Gradient Descent
Lồi	Để chỉ có một nghiệm tối ưu, không bị kẹt ở cực tiểu địa phương
Có cơ sở xác suất	Để việc chọn hàm mất mát không phải là tùy tiện

Hàm mất mát thỏa mãn tất cả các yêu cầu này là **cross-entropy**, xuất phát từ **nguyên lý hợp lý cực đại**. Ta xây dựng nó ở mục 3.2.



Hình 8. So sánh cross-entropy và MSE cho một mẫu có nhãn thật $y = 1$. (a) Cross-entropy phạt rất nặng khi s rất âm (mô hình sai chắc chắn), còn MSE bị chặn ở mức 1. (b) Gradient của cross-entropy lớn khi mẫu sai nặng, còn gradient của MSE tiến về 0 ở đúng vùng đó nên mô hình khó thoát ra.

3.2. Hợp lý cực đại và hàm mất mát cross-entropy

Ý tưởng: chọn **w** để dữ liệu quan sát được "có khả năng xảy ra nhất"

Mỗi bộ tham số **w** cho một mô hình xác suất khác nhau. Ta hãy hỏi: **với mô hình này, xác suất để thu được đúng bộ nhãn $y_1, \dots, y_n$ mà ta đã quan sát là bao nhiêu?** Bộ tham số nào làm xác suất này lớn nhất thì ta chọn. Đây là **nguyên lý hợp lý cực đại (maximum likelihood)**.

Ở mục 2.3, ta đã có xác suất mô hình gán cho nhãn thật của một mẫu:

$$q_i = P(y_i \mid \mathbf{x}_i) = p_i^{\,y_i}\,(1-p_i)^{1-y_i}$$

Nếu coi các mẫu **độc lập** với nhau, xác suất của toàn bộ tập nhãn là tích của các q_i, gọi là **hàm hợp lý (likelihood)**:

$$L(\mathbf{w}) = \prod_{i=1}^{n} p_i^{\,y_i}\,(1-p_i)^{1-y_i}, \qquad p_i = \sigma(\mathbf{x}_i\mathbf{w})$$

Ví dụ ở mục 2.4, với $\mathbf{w} = [-2{,}5 \quad 1 \quad -1]^T$ ta tính được $L \approx 0{,}795$. Ta so sánh với một vài bộ tham số khác trên cùng 6 sinh viên:

Bộ tham số **w**	Hàm hợp lý $L(\mathbf{w})$	$\ln L(\mathbf{w})$
$[0 \quad 0 \quad 0]^T$ (mọi $p_i = 0{,}5$)	$0{,}5^6 \approx 0{,}0156$	$-4{,}159$
$[-2{,}5 \quad 1 \quad 0]^T$ (bỏ qua số buổi vắng)	$\approx 0{,}0414$	$-3{,}185$
$[-2{,}5 \quad 0 \quad -1]^T$ (bỏ qua số giờ học)	$\approx 0{,}00002$	$-10{,}626$
$[-2{,}5 \quad 1 \quad -1]^T$	$\approx 0{,}795$	$-0{,}230$

Bộ tham số cuối cùng cho dữ liệu "dễ xảy ra" nhất. Bộ thứ ba gán xác suất rất thấp cho các nhãn thật (nó nghiêng hẳn về "rớt" cho cả ba sinh viên đậu), nên L gần 0.

Từ hợp lý cực đại đến cross-entropy

Ta muốn **làm $L(\mathbf{w})$ lớn nhất**, nhưng tích của nhiều số nhỏ hơn 1 rất khó dùng (rất nhỏ, dễ tràn số, đạo hàm của tích phức tạp). Ta lấy **logarit**, và nhờ ln là hàm tăng nên **w** làm L lớn nhất cũng làm $\ln L$ lớn nhất, đồng thời tích biến thành tổng:

$$\ln L(\mathbf{w}) = \sum_{i=1}^{n} [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

Theo thói quen, ta muốn **cực tiểu hóa** một hàm mất mát, nên đổi dấu và lấy trung bình trên n mẫu. Kết quả là hàm mất mát **cross-entropy nhị phân** (còn gọi là **log-loss**):

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n} [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

Như vậy:

$$\underset{\mathbf{w}}{\text{cực đại hóa}}\ L(\mathbf{w}) \Leftrightarrow \underset{\mathbf{w}}{\text{cực đại hóa}}\ \ln L(\mathbf{w}) \Leftrightarrow \underset{\mathbf{w}}{\text{cực tiểu hóa}}\ J(\mathbf{w})$$

(Hệ số $\frac{1}{n}$ không đổi vị trí của nghiệm, chỉ làm J có cùng thang đo bất kể số mẫu, nhờ đó tốc độ học ít phụ thuộc vào n.)

Đọc hàm mất mát cho từng mẫu

Mất mát của một mẫu chỉ có hai nhánh, tùy theo nhãn thật:

$$\ell_i = \begin{cases} -\ln p_i & \text{nếu } y_i = 1 \\ -\ln(1-p_i) & \text{nếu } y_i = 0 \end{cases} = -\ln q_i$$

Cả hai nhánh đều có dạng $-\ln q_i$, trong đó q_i là **xác suất mô hình gán cho điều đã xảy ra**. Có thể hiểu $-\ln q$ là **mức "ngạc nhiên"** của mô hình: càng ít ngờ tới điều đã xảy ra thì càng ngạc nhiên và bị phạt càng nặng.

Xác suất gán cho nhãn thật q	0,99	0,9	0,5	0,1	0,01
Mất mát $-\ln q$	0,010	0,105	0,693	2,303	4,605

Khi $q \to 0$ (mô hình khẳng định điều trái ngược với sự thật), mất mát tiến tới $+\infty$. Đây là điều ta cần: **phạt rất nặng việc sai một cách chắc chắn**. Ngược lại, khi $q \to 1$ thì mất mát $\to 0$.

Hai trường hợp cụ thể: với $y = 1$, mô hình cho $p = 0{,}9$ bị phạt $-\ln 0{,}9 \approx 0{,}105$, còn $p = 0{,}1$ bị phạt $-\ln 0{,}1 \approx 2{,}303$. Với $y = 0$, mô hình cho $p = 0{,}9$ bị phạt $-\ln(1-0{,}9) = -\ln 0{,}1 \approx 2{,}303$.

Tính J trên dữ liệu sinh viên

Với $J = -\frac{1}{n}\ln L$, các giá trị ở bảng trước cho

Bộ tham số $\mathbf{w}$	$J(\mathbf{w})$
$[0 \quad 0 \quad 0]^T$	0,693
$[-2{,}5 \quad 1 \quad 0]^T$	0,531
$[-2{,}5 \quad 0 \quad -1]^T$	1,771
$[-2{,}5 \quad 1 \quad -1]^T$	0,038

Bộ tham số tốt nhất có J nhỏ nhất. Một sự kiện đáng nhớ: với $\mathbf{w} = \mathbf{0}$, mọi $p_i = 0{,}5$ nên mỗi mẫu bị phạt $-\ln 0{,}5 = \ln 2$ và

$$J(\mathbf{0}) = \ln 2 \approx 0{,}693$$

Đây là giá trị ban đầu của hàm mất mát khi khởi tạo $\mathbf{w} = \mathbf{0}$, với **mọi** bộ dữ liệu, vì mỗi mẫu đều đóng góp đúng $\ln 2$ dù nhãn là 0 hay 1. Ta sẽ dùng nó để kiểm tra cài đặt ở mục 3.7.

Liên hệ với dạng margin ở bài Perceptron

Ở mục 6.3 của bài Perceptron, ta đã gặp mất mát logistic viết theo margin với nhãn $\{-1, +1\}$: $\ln(1 + e^{-m})$. Hai cách viết là **một**. Đặt $y_{\pm} = 2y - 1$ và $m = y_{\pm}\, s$:

- Nếu $y = 1$: $-\ln\sigma(s) = \ln(1 + e^{-s}) = \ln(1 + e^{-m})$, vì $m = s$.
- Nếu $y = 0$: $-\ln\big(1 - \sigma(s)\big) = -\ln\sigma(-s) = \ln(1 + e^{s}) = \ln(1 + e^{-m})$, vì $m = -s$.

Vậy $\ell_i = \ln(1 + e^{-m_i})$ ở cả hai trường hợp. Nhãn $\{0,1\}$ cho công thức tiện cho phần xác suất và gradient, còn nhãn $\{-1, +1\}$ cho thấy mối liên hệ với Perceptron.

3.3. Gradient của hàm mất mát và quy tắc cập nhật

Gradient của mất mát một mẫu

Ta tính gradient của $\ell_i = -[y_i \ln p_i + (1 - y_i)\ln(1 - p_i)]$ theo $\mathbf{w}$ qua ba bước nhờ quy tắc đạo hàm hàm hợp, theo chuỗi $\mathbf{w} \to s_i \to p_i \to \ell_i$.

Bước 1: đạo hàm theo p_i.

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}$$

Bước 2: đạo hàm của sigmoid (đã chứng minh ở mục 2.3):

$$\frac{\partial p_i}{\partial s_i} = \sigma'(s_i) = p_i\,(1 - p_i)$$

Bước 3: đạo hàm của điểm số $s_i = \mathbf{x}_i\mathbf{w}$ theo $\mathbf{w}$ là $\mathbf{x}_i^T$.

Nhân ba kết quả, thừa số $p_i(1 - p_i)$ **triệt tiêu** các mẫu số ở bước 1:

$$\frac{\partial \ell_i}{\partial s_i} = \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}\right) p_i\,(1 - p_i) = -y_i(1 - p_i) + (1 - y_i)\,p_i = p_i - y_i$$

Vậy gradient của mất mát một mẫu là

$$\nabla \ell_i(\mathbf{w}) = (p_i - y_i)\,\mathbf{x}_i^T$$

Đây là một kết quả rất gọn và đẹp. Thừa số $p_i(1 - p_i)$ gây ra vấn đề "gradient tắt" ở MSE được **triệt tiêu hoàn toàn** nhờ cross-entropy: gradient theo điểm số chỉ còn là $p_i - y_i$, tức **sai số giữa xác suất dự đoán và nhãn thật**. Khi mẫu sai chắc chắn ($y_i = 1$ nhưng $p_i \approx 0{,}0025$), ta có $p_i - y_i \approx -0{,}9975$, gần như lớn nhất (so với $\approx 0{,}005$ của MSE).

Gradient của hàm mất mát tổng

Cộng gradient của n mẫu và lấy trung bình:

$$\nabla J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\ \mathbf{x}_i^T = \frac{1}{n}\,\mathbf{X}^T(\mathbf{p} - \mathbf{y})$$

trong đó $\mathbf{p} = \sigma(\mathbf{X}\mathbf{w})$. Kiểm tra kích thước: $\mathbf{X}^T$ có kích thước $(d+1) \times n$ và $(\mathbf{p} - \mathbf{y})$ có kích thước $n \times 1$ nên $\mathbf{X}^T(\mathbf{p} - \mathbf{y})$ có kích thước $(d+1) \times 1$, khớp với $\mathbf{w}$. Viết theo từng thành phần:

$$\frac{\partial J}{\partial w_0} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,, \qquad \frac{\partial J}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\ x_{ij} \quad (j = 1, \dots, d)$$

Quy tắc cập nhật

Áp dụng Gradient Descent $\mathbf{w} \leftarrow \mathbf{w} - \eta\nabla J(\mathbf{w})$, ta được **quy tắc cập nhật của Logistic Regression**:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n}\,\mathbf{X}^T(\mathbf{p} - \mathbf{y}) = \mathbf{w} + \frac{\eta}{n}\,\mathbf{X}^T(\mathbf{y} - \mathbf{p})$$

Trong đó $\eta > 0$ là **tốc độ học (learning rate)**. Cách đọc quy tắc này rất trực quan: với mỗi mẫu, ta nhân sai số $(y_i - p_i)$ với vector đặc trưng $\mathbf{x}_i^T$ rồi cộng vào $\mathbf{w}$.

- Mẫu có $y_i = 1$ nhưng p_i còn nhỏ: $(y_i - p_i) > 0$ lớn, nên **cộng** $\mathbf{x}_i^T$ vào $\mathbf{w}$, kéo $\mathbf{w}$ lại gần mẫu để tăng p_i.
- Mẫu có $y_i = 0$ nhưng p_i còn lớn: $(y_i - p_i) < 0$, nên **trừ** $\mathbf{x}_i^T$, đẩy $\mathbf{w}$ ra xa mẫu để giảm p_i.
- Mẫu đã được dự đoán gần đúng ($p_i \approx y_i$): $(y_i - p_i) \approx 0$, gần như không đóng góp.

So sánh ba quy tắc cập nhật

Ba mô hình ta đã học có quy tắc cập nhật cùng một "khung", chỉ khác phần "sai số":

Mô hình	Quy tắc cập nhật theo toàn bộ dữ liệu
Hồi quy tuyến tính	$\mathbf{w} \leftarrow \mathbf{w} - \frac{2\eta}{n}\,\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y})$
Logistic Regression	$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n}\,\mathbf{X}^T(\sigma(\mathbf{X}\mathbf{w}) - \mathbf{y})$
Perceptron (nhãn ± 1, từng mẫu)	$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \mathbb{1}[m_i \le 0] \cdot y_{\pm,i}\,\mathbf{x}_i^T$

(Hệ số 2 ở hồi quy có thể gộp vào η, và cách viết hệ số phụ thuộc vào quy ước của bài hồi quy.) Hồi quy tuyến tính và Logistic Regression **gần như giống hệt nhau**: cả hai đều có dạng $\mathbf{X}^T\big(\text{dự đoán} - \text{thực tế}\big)$. Khác biệt duy nhất là "dự đoán" là $\mathbf{X}\mathbf{w}$ (số thực) hay $\sigma(\mathbf{X}\mathbf{w})$ (xác suất). Đó là một hệ quả đẹp của việc chọn cross-entropy: nó cho gradient có cùng dạng với MSE của hồi quy tuyến tính, trong khi MSE kết hợp với sigmoid thì không.

Mối liên hệ với Perceptron cũng rõ: theo dạng margin, gradient cross-entropy cho $\mathbf{w} \leftarrow \mathbf{w} + \eta\,\sigma(-m_i)\,y_{\pm,i}\,\mathbf{x}_i^T$ (đã thấy ở mục 6.3 bài Perceptron). Thật vậy, với $y_i = 1$ ta có $y_i - p_i = 1 - \sigma(s_i) = \sigma(-s_i) = \sigma(-m_i)$ và $y_{\pm,i} = 1$. Với $y_i = 0$ ta có $y_i - p_i = -\sigma(s_i) = -\sigma(-m_i)$ (vì

$m_i = -s_i$) và $y_{\pm,i} = -1$, nên $(y_i - p_i) = \sigma(-m_i)\, y_{\pm,i}$. Perceptron là trường hợp "trọng số cứng" $\mathbb{1}[m_i \le 0]$, còn Logistic Regression dùng trọng số mềm $\sigma(-m_i)$.

3.4. Vai trò của learning rate

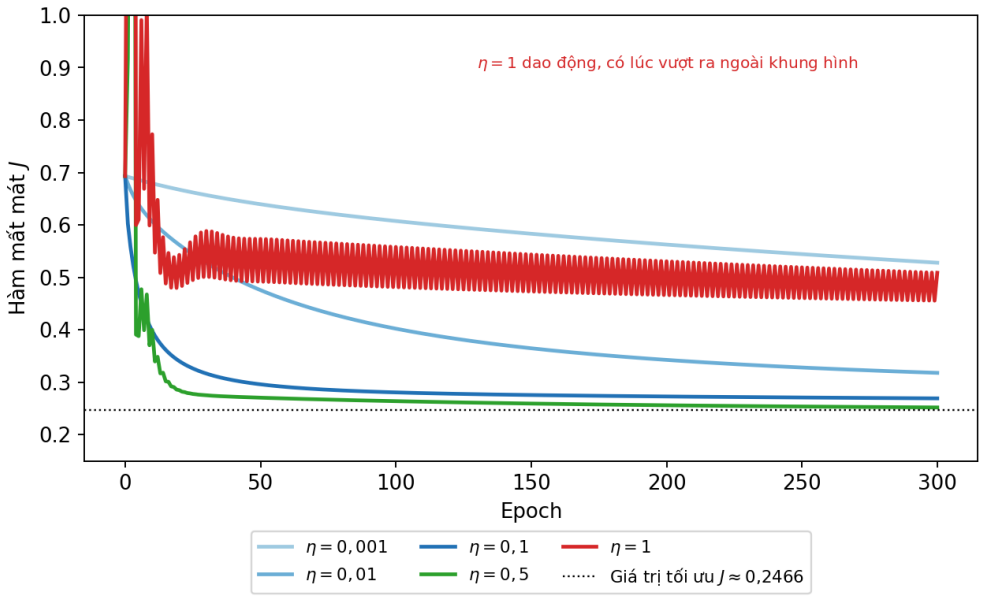
Tốc độ học η quyết định mỗi bước cập nhật đi xa bao nhiêu. Khác với Perceptron (mục 3.6 của bài Perceptron, nơi với $\mathbf{w}^{(0)} = \mathbf{0}$ thì η không đổi bộ phân loại), **Logistic Regression rất nhạy với** η, vì độ lớn của gradient phụ thuộc vào các xác suất p_i hiện tại, và η là hệ số duy nhất điều khiển kích thước bước.

Ta kiểm chứng bằng thí nghiệm trên 280 mẫu huấn luyện (dữ liệu mô phỏng ở mục 5.2 của bài Perceptron, nhãn đổi sang $0/1$, đặc trưng giữ nguyên thang đo). Ta chạy Gradient Descent trên toàn bộ dữ liệu với $\mathbf{w}^{(0)} = \mathbf{0}$ và các giá trị η khác nhau. Bảng ghi giá trị J sau số epoch cho trước; giá trị nhỏ nhất có thể đạt được trên dữ liệu này là $J \approx 0{,}2466$:

η	Sau 10 epoch	Sau 100 epoch	Sau 500 epoch	Sau 2000 epoch
0,001	0,679	0,607	0,476	0,343
0,01	0,607	0,402	0,297	0,273
0,05	0,473	0,296	0,271	0,256
0,1	0,398	0,281	0,264	0,250
0,2	0,336	0,273	0,256	0,247
0,5	0,400	0,264	0,248	**0,2466**
1	0,773	0,558	0,479	0,443

Hình 9 vẽ đường cong học tương ứng. Có thể rút ra:

- η **quá nhỏ** (0,001): J giảm rất chậm, sau 2000 epoch vẫn còn rất xa mức tối ưu.
- η **vừa phải** (0,1 đến 0,5): J giảm nhanh và hội tụ về $\approx 0{,}2466$. Trong đó $\eta = 0{,}5$ nhanh nhất nhưng ở các epoch đầu, J có lúc vọt lên tới khoảng 2,7 rồi mới giảm trở lại, cho thấy nó đã ở sát giới hạn an toàn.
- η **quá lớn** (1): bước nhảy quá dài làm **w vọt qua** điểm tối ưu và dao động. J có lúc lên tới khoảng 5,4 và vẫn dao động quanh mức 0,45 đến 0,55 sau hàng trăm epoch, không hội tụ.



Hình 9. Hàm mất mát J theo epoch với các tốc độ học khác nhau (Gradient Descent trên toàn bộ dữ liệu). η nhỏ thì chậm, η quá lớn thì dao động, và có một khoảng giá trị vừa phải cho kết quả tốt.

Cách nhận biết η chưa phù hợp khi huấn luyện: vẽ J theo epoch. Nếu J **tăng** hoặc **dao động mạnh** thì giảm η. Nếu J giảm đều nhưng quá chậm thì tăng η.

Giá trị η phù hợp còn phụ thuộc vào **thang đo của các đặc trưng**: đặc trưng có giá trị càng lớn thì gradient càng lớn và càng cần η nhỏ. Ở mục 4.2, ta sẽ thấy việc chuẩn hóa đặc trưng giúp chọn η dễ hơn nhiều.

3.5. Thuật toán Gradient Descent qua từng bước

Các biến thể

Gradient $\nabla J(\mathbf{w}) = \frac{1}{n}\sum_i (p_i - y_i)\,\mathbf{x}_i^T$ là trung bình của n gradient riêng lẻ. Tùy số mẫu dùng để ước lượng nó ở mỗi bước, ta có ba biến thể:

Biến thể	Số mẫu mỗi lần cập nhật	Số lần cập nhật mỗi epoch	Quy tắc
Batch Gradient Descent	n (toàn bộ)	1	$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y})$
Stochastic Gradient Descent (SGD)	1	n	$\mathbf{w} \leftarrow \mathbf{w} - \eta\,(p_i - y_i)\,\mathbf{x}_i^T$
Mini-batch Gradient Descent	b $(1 < b < n)$	$\lceil n/b \rceil$	$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{b}\mathbf{X}_B^T(\mathbf{p}_B - \mathbf{y}_B)$

Trong dòng cuối, B là tập chỉ số của mini-batch hiện tại, $\mathbf{X}_B$ là ma trận gồm các hàng của $\mathbf{X}$ tương ứng và $\mathbf{p}_B$, $\mathbf{y}_B$ là các vector con tương ứng. Một **epoch** là một lượt duyệt qua toàn bộ n mẫu, bất kể biến thể nào.

Thuật toán

Đầu vào: ma trận đặc trưng $\mathbf{X}$ ($n \times (d+1)$), vector nhãn $\mathbf{y}$ ($n \times 1$, các phần tử 0 hoặc 1), tốc độ học $\eta > 0$, kích thước mini-batch b (với $b = n$ ta được batch, với $b = 1$ ta được SGD), số epoch tối đa $T_{\max}$, ngưỡng dừng ε.

Đầu ra: vector tham số $\mathbf{w}$.

8. **Khởi tạo** $\mathbf{w} = \mathbf{0}$ (vector cột kích thước $(d+1) \times 1$).
9. **Lặp** cho $t = 1,2, \dots, T_{\max}$ (mỗi lần lặp là một epoch):
    1. Xáo trộn thứ tự các mẫu (với $b = n$ thì không cần).
    2. Chia các mẫu thành các mini-batch gồm b mẫu.
    3. **Với mỗi** mini-batch B:
        1. Tính xác suất $\mathbf{p}_B = \sigma(\mathbf{X}_B\mathbf{w})$.
        2. Tính gradient $\mathbf{g} = \frac{1}{b}\mathbf{X}_B^T(\mathbf{p}_B - \mathbf{y}_B)$.
        3. Cập nhật $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\mathbf{g}$.
    4. **Nếu** điều kiện dừng thỏa (xem bên dưới): **dừng**.
10. **Trả về** $\mathbf{w}$.

Điều kiện dừng

Khác với Perceptron (dừng khi một epoch không còn cập nhật), Logistic Regression **thường không đạt gradient chính xác bằng** 0 **trong hữu hạn bước**, nên ta dùng một trong các điều kiện sau:

- Độ lớn gradient đủ nhỏ: $\| \nabla J(\mathbf{w}) \| < \varepsilon$ (khi dùng batch).
- Hàm mất mát gần như không giảm nữa: $|J_t - J_{t-1}| <$ tol.
- Đạt số epoch tối đa $T_{\max}$ (luôn nên có, như một "van an toàn").

Với SGD và mini-batch, J dao động vì mỗi bước chỉ dùng một phần dữ liệu, nên điều kiện dừng thường dựa vào số epoch cố định hoặc theo dõi J trên toàn bộ dữ liệu sau mỗi epoch.

So sánh các biến thể

- **Batch:** hướng đi **chính xác** và đường cong học mượt, nhưng mỗi bước tốn chi phí tính trên **toàn bộ** dữ liệu, và chỉ có 1 bước mỗi epoch nên chậm khi n lớn.
- **SGD:** mỗi bước rất **rẻ** và có n bước mỗi epoch nên tiến bộ nhanh, nhưng hướng đi **nhiễu** nên J dao động quanh điểm tối ưu và không hội tụ hẳn nếu η cố định.
- **Mini-batch:** là **sự dung hòa**: ít nhiễu hơn SGD, rẻ hơn batch, và tận dụng tốt phép nhân ma trận trên phần cứng hiện đại. Đây là lựa chọn phổ biến nhất trong thực tế.

3.6. Ví dụ tính toán thủ công một vòng huấn luyện

Ta chạy **batch Gradient Descent** trên dữ liệu 6 sinh viên của mục 1 với nhãn $\{0,1\}$, khởi tạo $\mathbf{w}^{(0)} = \mathbf{0}$ và $\eta = 0{,}1$. Nhắc lại

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 3 \\ 1 & 4 & 5 \\ 1 & 6 & 1 \\ 1 & 8 & 2 \\ 1 & 9 & 0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

Bước 1 (từ $\mathbf{w}^{(0)} = \mathbf{0}$)

Xác suất. Vì $\mathbf{w}^{(0)} = \mathbf{0}$ nên $\mathbf{s} = \mathbf{0}$ và $p_i = \sigma(0) = 0{,}5$ cho mọi mẫu:

$$\mathbf{p} - \mathbf{y} = \begin{bmatrix} 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ 0{,}5 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0{,}5 \\ 0{,}5 \\ 0{,}5 \\ -0{,}5 \\ -0{,}5 \\ -0{,}5 \end{bmatrix}$$

Hàm mất mát. $J\left(\mathbf{w}^{(0)}\right) = \ln 2 \approx 0{,}6931$, như đã nói ở mục 3.2.

Gradient. Ta tính $\mathbf{X}^T(\mathbf{p} - \mathbf{y})$ theo từng cột của $\mathbf{X}$:

$$\mathbf{X}^T(\mathbf{p} - \mathbf{y}) = \begin{bmatrix} 0{,}5 \cdot 3 - 0{,}5 \cdot 3 \\ 0{,}5\,(2+3+4) - 0{,}5\,(6+8+9) \\ 0{,}5\,(4+3+5) - 0{,}5\,(1+2+0) \end{bmatrix} = \begin{bmatrix} 0 \\ 4{,}5 - 11{,}5 \\ 6 - 1{,}5 \end{bmatrix} = \begin{bmatrix} 0 \\ -7 \\ 4{,}5 \end{bmatrix}$$

$$\nabla J\left(\mathbf{w}^{(0)}\right) = \frac{1}{6}\begin{bmatrix} 0 \\ -7 \\ 4{,}5 \end{bmatrix} \approx \begin{bmatrix} 0 \\ -1{,}1667 \\ 0{,}75 \end{bmatrix}$$

Cập nhật.

$$\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - 0{,}1 \cdot \nabla J\left(\mathbf{w}^{(0)}\right) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - 0{,}1\begin{bmatrix} 0 \\ -1{,}1667 \\ 0{,}75 \end{bmatrix} = \begin{bmatrix} 0 \\ 0{,}1167 \\ -0{,}075 \end{bmatrix}$$

Dấu của gradient hợp lý: $\partial J / \partial w_1 < 0$ nên w_1 **tăng** (đặc trưng giờ học nên đóng góp dương), còn $\partial J / \partial w_2 > 0$ nên w_2 **giảm** (số buổi vắng nên đóng góp âm).

Bước 2 (từ $\mathbf{w}^{(1)}$)

Điểm số và xác suất. Với $\mathbf{w}^{(1)} = [0 \quad 0{,}1167 \quad -0{,}075]^T$:

$$\mathbf{s} = \mathbf{X}\mathbf{w}^{(1)} = \begin{bmatrix} -0{,}0667 \\ 0{,}125 \\ 0{,}0917 \\ 0{,}625 \\ 0{,}7833 \\ 1{,}05 \end{bmatrix} \Rightarrow \mathbf{p} = \sigma(\mathbf{s}) \approx \begin{bmatrix} 0{,}4833 \\ 0{,}5312 \\ 0{,}5229 \\ 0{,}6514 \\ 0{,}6864 \\ 0{,}7408 \end{bmatrix} \Rightarrow \mathbf{p} - \mathbf{y} \approx \begin{bmatrix} 0{,}4833 \\ 0{,}5312 \\ 0{,}5229 \\ -0{,}3486 \\ -0{,}3136 \\ -0{,}2592 \end{bmatrix}$$

Hàm mất mát. $J\left(\mathbf{w}^{(1)}\right) \approx 0{,}5438$, giảm so với $0{,}6931$.

Gradient.

$$\nabla J\left(\mathbf{w}^{(1)}\right) = \frac{1}{6}\mathbf{X}^T(\mathbf{p} - \mathbf{y}) \approx \frac{1}{6}\begin{bmatrix} 0{,}6160 \\ -2{,}2818 \\ 5{,}1656 \end{bmatrix} \approx \begin{bmatrix} 0{,}1027 \\ -0{,}3803 \\ 0{,}8609 \end{bmatrix}$$

Cập nhật.

$$\mathbf{w}^{(2)} = \begin{bmatrix} 0 \\ 0{,}1167 \\ -0{,}075 \end{bmatrix} - 0{,}1\begin{bmatrix} 0{,}1027 \\ -0{,}3803 \\ 0{,}8609 \end{bmatrix} \approx \begin{bmatrix} -0{,}0103 \\ 0{,}1547 \\ -0{,}1611 \end{bmatrix}, \qquad J\left(\mathbf{w}^{(2)}\right) \approx 0{,}4603$$

Các bước tiếp theo

Tiếp tục theo cách tương tự, ta thu được bảng:

Bước t	$\mathbf{w}^{(t)}$	$J\left(\mathbf{w}^{(t)}\right)$
0	$[0 \quad 0 \quad 0]^T$	0,6931
1	$[0 \quad 0{,}1167 \quad -0{,}075]^T$	0,5438
2	$[-0{,}0103 \quad 0{,}1547 \quad -0{,}1611]^T$	0,4603
3	$[-0{,}0194 \quad 0{,}1866 \quad -0{,}2360]^T$	0,3974
4	$[-0{,}0274 \quad 0{,}2153 \quad -0{,}3012]^T$	0,3491
5	$[-0{,}0344 \quad 0{,}2414 \quad -0{,}3585]^T$	0,3111

Hàm mất mát **giảm đều** ở mọi bước, đúng như ta mong đợi. Quá trình học theo thứ tự trực giác: trọng số w_1 của giờ học tăng dần (dương), trọng số w_2 của buổi vắng giảm dần (âm), tức là mô hình đang học dần cùng một hướng với bộ $\mathbf{w}$ đã dùng ở các mục trước.

Nhận xét

- **Mỗi bước giảm J một chút** thay vì "sửa hết lỗi một lần" như Perceptron. Thuật toán không dừng khi mọi mẫu đã được phân loại đúng: mỗi mẫu đúng vẫn còn có thể được gán xác suất chắc chắn hơn.
- **Cả 6 mẫu đều đóng góp** vào gradient ở mọi bước (khác với Perceptron chỉ dùng mẫu sai), với trọng số $|p_i - y_i|$.
- Dữ liệu 6 sinh viên này **phân tách được hoàn toàn**. Với loại dữ liệu này J sẽ giảm **mãi mãi** về 0 nếu cứ tiếp tục (vì $\mathbf{w}$ càng lớn thì xác suất càng gần 0 hoặc 1), một hiện tượng đặc biệt mà ta sẽ phân tích ở mục 4.4.

3.7. Mã hóa thuật toán Logistic Regression từ đầu bằng Python

Ta cài đặt thuật toán bằng NumPy. Mỗi công thức có một dòng mã tương ứng, giống bài Perceptron.

Ký hiệu toán	Trong mã Python
Ma trận đặc trưng $\mathbf{X}$ (có cột 1)	*Xb* (kết quả của *them_bias*)
Vector tham số $\mathbf{w}$	*w*
Vector điểm số $\mathbf{s} = \mathbf{X}\mathbf{w}$	*Xb @ w*
Vector xác suất $\mathbf{p} = \sigma(\mathbf{s})$	*sigmoid(Xb @ w)*
Hàm mất mát $J(\mathbf{w})$	*mat_mat(Xb, y, w)*
Gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y})$	*Xb.T @ (p - y) / n*
Cập nhật $\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla J$	*w = w - eta * gradient(...)*

Các hàm cơ bản

```python
import numpy as np

def them_bias(X):
    """Thêm cột 1 vào đầu ma trận đặc trưng: (n, d) -> (n, d+1)."""
    return np.hstack([np.ones((X.shape[0], 1)), X])

def sigmoid(s):
    """sigma(s) = 1 / (1 + e^{-s}), viết qua tanh để không bị tràn số:
    sigma(s) = 0.5 * (1 + tanh(s / 2))."""
    return 0.5 * (1.0 + np.tanh(0.5 * s))

def mat_mat(Xb, y, w):
    """Hàm mất mát cross-entropy trung bình J(w)."""
    p = np.clip(sigmoid(Xb @ w), 1e-12, 1 - 1e-12)      # tránh log(0)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

def gradient(Xb, y, w):
    """Gradient của J: (1/n) * X^T (p - y)."""
    p = sigmoid(Xb @ w)
    return Xb.T @ (p - y) / len(y)

def du_doan_xac_suat(Xb, w):
    """Xác suất p = sigma(Xw) của lớp dương."""
    return sigmoid(Xb @ w)

def du_doan(Xb, w, nguong=0.5):
    """Nhãn dự đoán: 1 nếu p >= nguong, ngược lại 0."""
    return (du_doan_xac_suat(Xb, w) >= nguong).astype(int)
```

Có hai chi tiết về **độ ổn định số học**. Công thức $\sigma(s) = 1/(1 + e^{-s})$ viết thẳng sẽ tràn số khi s rất âm, nên ta dùng đẳng thức $\sigma(s) = \frac{1}{2}\big(1 + \tanh(s/2)\big)$ cho cùng giá trị mà không bị tràn. Còn *np.clip* giữ p cách xa 0 và 1 một chút để $\ln p$ không thành $-\infty$.

Hàm huấn luyện

Một hàm duy nhất xử lý cả ba biến thể: *kich_thuoc_batch=None* cho batch, bằng 1 cho SGD, và một số b khác cho mini-batch.

```
def huan_luyen(Xb, y, eta=0.1, so_epoch=1000, kich_thuoc_batch=None, seed=0):
    """Gradient Descent: toàn bộ (batch), từng mẫu (SGD) hoặc mini-batch."""
    n, d1 = Xb.shape
    b = n if kich_thuoc_batch is None else kich_thuoc_batch
    rng = np.random.default_rng(seed)
    w = np.zeros(d1)                                    # khởi tạo w = 0
    lich_su = [mat_mat(Xb, y, w)]                       # J sau mỗi epoch
    for epoch in range(so_epoch):
        idx = rng.permutation(n) if b < n else np.arange(n)
        for k in range(0, n, b):
            lo = idx[k:k + b]                           # chỉ số của mini-batc
h hiện tại
            w = w - eta * gradient(Xb[lo], y[lo], w)
        lich_su.append(mat_mat(Xb, y, w))
    return w, lich_su
```

Kiểm tra trên dữ liệu 6 sinh viên

Ta kiểm tra cài đặt bằng cách so với ví dụ tính tay ở mục 3.6 (batch, $\eta = 0{,}1$):

```
np.set_printoptions(precision=4, suppress=True)
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0]], dtype=float
)
y_sv = np.array([0, 0, 0, 1, 1, 1])
Xb_sv = them_bias(X_sv)

for k in (1, 2, 5):
    w_k, ls = huan_luyen(Xb_sv, y_sv, eta=0.1, so_epoch=k)
    print(f"sau {k} buoc: w = {w_k} | J = {ls[-1]:.4f}")
```

```
sau 1 buoc: w = [ 0.      0.1167 -0.075 ] | J = 0.5438
sau 2 buoc: w = [-0.0103  0.1547 -0.1611] | J = 0.4603
sau 5 buoc: w = [-0.0344  0.2414 -0.3585] | J = 0.3111
```

Các giá trị **khớp hoàn toàn** với ví dụ tính tay ($\mathbf{w}^{(1)}$, $\mathbf{w}^{(2)}$ và $\mathbf{w}^{(5)}$ cùng các giá trị J ở bảng mục 3.6).

Kiểm tra gradient bằng đạo hàm số

Một kỹ thuật rất hữu ích để tìm lỗi trong công thức gradient là so với **đạo hàm số** (finite difference): $\frac{\partial J}{\partial w_j} \approx \frac{J(\mathbf{w}+\epsilon\,\mathbf{e}_j)-J(\mathbf{w}-\epsilon\,\mathbf{e}_j)}{2\epsilon}$, trong đó $\mathbf{e}_j$ là vector đơn vị thứ j. Ta kiểm tra trên bộ dữ liệu lớn hơn (dữ liệu mô phỏng ở mục 5.2 bài Perceptron, nhãn đổi sang $0/1$), với 280 mẫu huấn luyện và 120 mẫu kiểm tra:

```
def chia_train_test(X, y, ti_le_test=0.3, seed=0):
    """Xáo trộn rồi chia dữ liệu thành tập huấn luyện và tập kiểm tra."""
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(y))
    n_test = int(len(y) * ti_le_test)
    return X[idx[n_test:]], X[idx[:n_test]], y[idx[n_test:]], y[idx[:n_test]]

rng = np.random.default_rng(42)
n_lop = 200
dau = np.c_[rng.normal(8, 2.2, n_lop), rng.normal(1.5, 1.2, n_lop)]   # lớp 1 (đậu)
rot = np.c_[rng.normal(4, 2.2, n_lop), rng.normal(3.5, 1.5, n_lop)]   # lớp 0 (rớt)
X = np.clip(np.vstack([dau, rot]), 0, None)
y = np.r_[np.ones(n_lop), np.zeros(n_lop)].astype(int)
X_tr, X_te, y_tr, y_te = chia_train_test(X, y, ti_le_test=0.3, seed=0)
Xb_tr, Xb_te = them_bias(X_tr), them_bias(X_te)
print(Xb_tr.shape, Xb_te.shape)

w = np.array([-1.0, 0.3, -0.2]); eps = 1e-6
g_so = np.array([(mat_mat(Xb_tr, y_tr, w + eps * e) - mat_mat(Xb_tr, y_tr, w - eps * e)) / (2 * eps)
                 for e in np.eye(3)])
print("gradient giai tich:", gradient(Xb_tr, y_tr, w))
print("gradient so       :", g_so)
```

```
(280, 3) (120, 3)
gradient giai tich: [ 0.0351 -0.1825  0.3466]
gradient so       : [ 0.0351 -0.1825  0.3466]
```

Hai kết quả trùng đến 4 chữ số thập phân, xác nhận công thức $\nabla J = \frac{1}{n}\mathbf{X}^T(\mathbf{p}-\mathbf{y})$ là đúng.

Huấn luyện bằng ba cách

Ta chạy ba biến thể trong 200 epoch, mỗi biến thể dùng một η hợp lý (SGD cần η nhỏ hơn vì mỗi epoch có tới 280 bước):

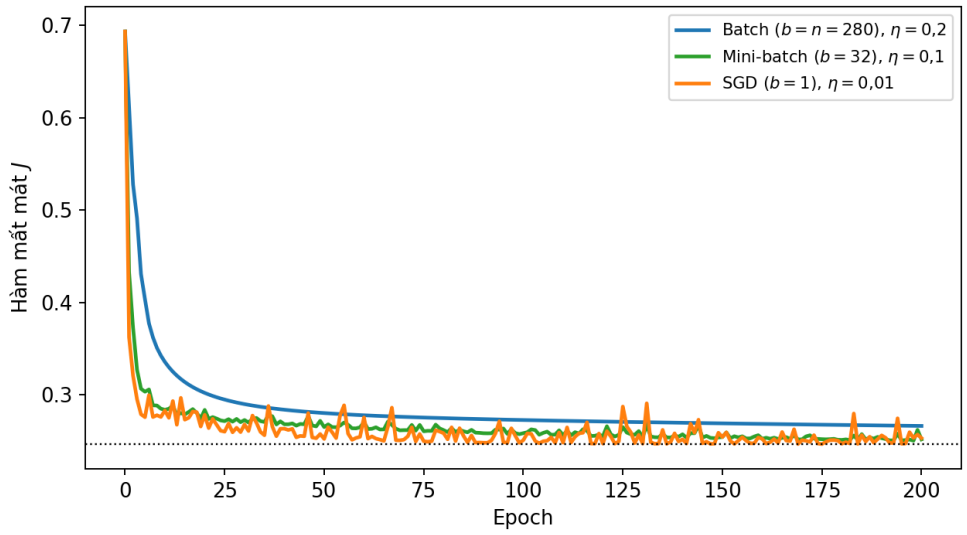
```
cau_hinh = {"batch (b = n = 280)":  dict(eta=0.2,  kich_thuoc_batch=None),
            "mini-batch (b = 32)":  dict(eta=0.1,  kich_thuoc_batch=32),
            "SGD (b = 1)":          dict(eta=0.01, kich_thuoc_batch=1)}
ket_qua = {}
for ten, cfg in cau_hinh.items():
```

```
    w_k, ls = huan_luyen(Xb_tr, y_tr, so_epoch=200, **cfg)
    ket_qua[ten] = (w_k, ls)
    acc_tr = np.mean(du_doan(Xb_tr, w_k) == y_tr)
    acc_te = np.mean(du_doan(Xb_te, w_k) == y_te)
    print(f"{ten:22s} J = {ls[-1]:.4f} | w = {w_k} | train {acc_tr:.4f} | tes
t {acc_te:.4f}")
print("J ban dau =", round(ket_qua["batch (b = n = 280)"][1][0], 4), "= ln 2
=", round(np.log(2), 4))
```

```
batch (b = n = 280)    J = 0.2664 | w = [-0.8499  0.7769 -1.398 ] | train 0.8
786 | test 0.8500
mini-batch (b = 32)    J = 0.2518 | w = [-2.2983  0.9475 -1.3368] | train 0.8
929 | test 0.8917
SGD (b = 1)            J = 0.2533 | w = [-3.4087  1.0946 -1.3571] | train 0.8
821 | test 0.8917
J ban dau = 0.6931 = ln 2 = 0.6931
```

Dòng cuối xác nhận $J(\mathbf{0}) = \ln 2$. Hình 10 vẽ J theo epoch cho ba cách. Có một số nhận xét:

- **Theo số epoch**, mini-batch và SGD **giảm J nhanh hơn** batch, vì chúng thực hiện nhiều lần cập nhật hơn trong mỗi epoch (tương ứng 9 lần và 280 lần so với 1 lần). Sau 200 epoch, batch với $\eta = 0{,}2$ mới đạt $J = 0{,}2664$ và **chưa hội tụ** (đã thấy ở mục 3.4 rằng cần khoảng 2000 epoch với $\eta = 0{,}5$ để đạt mức tối ưu 0,2466). Vì vậy độ chính xác kiểm tra của batch (85,0%) thấp hơn hai cách còn lại (89,2%), không phải do batch "kém hơn" mà do nó **chưa chạy đủ lâu**.
- **SGD và mini-batch dao động** quanh điểm tối ưu (các gai nhọn trên Hình 10) vì mỗi bước chỉ dựa trên một phần dữ liệu. Batch thì mượt.
- Cả ba cách đều tiến về cùng một bộ tham số tối ưu, chỉ khác tốc độ và độ nhiễu.



Hình 10. Hàm mất mát J theo epoch của ba biến thể Gradient Descent trên cùng dữ liệu huấn luyện. Mini-batch và SGD thực hiện nhiều bước hơn mỗi epoch nên giảm nhanh hơn nhưng dao động; batch mượt nhưng chậm hơn. Đường chấm là giá trị tối ưu $J \approx 0{,}2466$.

Cần lưu ý rằng dữ liệu này **chồng lấn** (không phân tách tuyến tính được), nên khác với Perceptron, Logistic Regression hội tụ **êm** về một nghiệm duy nhất mà không cần thuật toán túi hay giới hạn số epoch ép buộc. Mục 4 sẽ giải thích vì sao.

Tóm tắt mục 3

- Mất mát Perceptron không phù hợp với xác suất, còn **MSE kết hợp với sigmoid** gây ra "gradient tắt" ở đúng những mẫu sai chắc chắn và không lồi. Cần một hàm mất mát **phạt nặng khi sai chắc chắn, có gradient lớn khi sai nặng, trơn, lồi**.
- Theo **nguyên lý hợp lý cực đại**, ta chọn $\mathbf{w}$ làm xác suất của các nhãn quan sát được lớn nhất: $L(\mathbf{w}) = \prod_i p_i^{y_i}\,(1-p_i)^{1-y_i}$. Lấy $-\frac{1}{n}\ln$ ta được hàm mất mát **cross-entropy**:

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

  Mất mát của một mẫu là $-\ln q_i$ với q_i là xác suất gán cho nhãn thật. Khi $\mathbf{w} = \mathbf{0}$ thì $J = \ln 2$. Dạng margin là $\ln(1+e^{-m})$.
- **Gradient** có dạng rất gọn: $\nabla J(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{p}-\mathbf{y})$, nhờ thừa số $\sigma'(s) = p(1-p)$ triệt tiêu. **Quy tắc cập nhật** là $\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{n}\mathbf{X}^T(\mathbf{p}-\mathbf{y})$, cùng dạng với hồi quy tuyến tính, chỉ khác "dự đoán" là $\sigma(\mathbf{Xw})$.
- **Tốc độ học** η rất quan trọng (khác với Perceptron): quá nhỏ thì chậm, quá lớn thì dao động, cần theo dõi J theo epoch.
- **Batch, SGD và mini-batch** khác nhau ở số mẫu dùng cho mỗi bước cập nhật: batch chính xác nhưng chậm, SGD nhanh nhưng nhiễu, mini-batch dung hòa. Một epoch là một lượt duyệt hết dữ liệu.
- Cài đặt bằng NumPy khớp với ví dụ tính tay, và có thể kiểm tra gradient bằng **đạo hàm số**.

Câu hỏi kiểm tra nhanh

1. Tính mất mát cross-entropy của một mẫu trong ba trường hợp: ($y = 1$, $p = 0{,}9$), ($y = 1$, $p = 0{,}1$) và ($y = 0$, $p = 0{,}9$). Nhận xét.
2. Cho một mẫu $\mathbf{x} = [1 \quad 2 \quad 3]$ có nhãn $y = 1$ và $\mathbf{w} = \mathbf{0}$. Tính p, gradient của mất mát một mẫu $(p-y)\mathbf{x}^T$, rồi cập nhật $\mathbf{w}$ với $\eta = 0{,}1$. Xác suất của mẫu này sau cập nhật là bao nhiêu?
3. Giải thích bằng số liệu vì sao MSE kết hợp sigmoid gây khó khăn khi mẫu có $y = 1$ nhưng $s = -6$. So sánh gradient theo s của cross-entropy và MSE.
4. Hai mẫu độc lập có xác suất mô hình gán cho nhãn thật lần lượt là $q_1 = 0{,}8$ và $q_2 = 0{,}5$. Tính hàm hợp lý L và hàm mất mát J.
5. Khi huấn luyện, ta thấy J tăng dần qua các epoch. Có thể có nguyên nhân gì và cần làm gì?
6. Với $n = 1000$ mẫu và mini-batch $b = 50$, mỗi epoch có bao nhiêu lần cập nhật khi dùng batch, SGD và mini-batch?
7. Kiểm tra rằng với $y = 0$ và $s = 1$, hai cách tính mất mát $-\ln\big(1-\sigma(s)\big)$ và $\ln(1+e^{-m})$ (với $m = y_{\pm}s$, $y_{\pm} = 2y-1$) cho cùng một giá trị.

Gợi ý đáp án

1. $(y = 1, p = 0{,}9)$: $-\ln 0{,}9 \approx 0{,}105$. $(y = 1, p = 0{,}1)$: $-\ln 0{,}1 \approx 2{,}303$. $(y = 0, p = 0{,}9)$: $-\ln(1 - 0{,}9) = -\ln 0{,}1 \approx 2{,}303$. Dự đoán đúng thì bị phạt nhẹ, dự đoán sai chắc chắn thì bị phạt nặng (gấp hơn 20 lần).
2. $\mathbf{w} = \mathbf{0}$ nên $s = 0$ và $p = 0{,}5$. Gradient: $(0{,}5 - 1)\,\mathbf{x}^T = -0{,}5[1 \quad 2 \quad 3]^T = [-0{,}5 \quad -1 \quad -1{,}5]^T$. Cập nhật: $\mathbf{w} = \mathbf{0} - 0{,}1[-0{,}5 \quad -1 \quad -1{,}5]^T = [0{,}05 \quad 0{,}1 \quad 0{,}15]^T$. Điểm số mới $s = 0{,}05 + 0{,}2 + 0{,}45 = 0{,}7$, nên $p = \sigma(0{,}7) \approx 0{,}668$, tăng từ 0,5 (đúng hướng).
3. Với $s = -6$, $y = 1$: $p = \sigma(-6) \approx 0{,}0025$. Gradient theo s của cross-entropy là $p - y \approx -0{,}9975$ (độ lớn gần 1, là mức lớn nhất). Của MSE là $-2(1 - p)\,p(1 - p) \approx -0{,}005$ (gần bằng 0). Do đó với MSE, mô hình gần như không sửa được sai lầm chắc chắn này.
4. $L = q_1 q_2 = 0{,}8 \cdot 0{,}5 = 0{,}4$. $J = -\frac{1}{2}\ln L = -\frac{1}{2}\ln 0{,}4 \approx 0{,}458$ (hoặc $\frac{-\ln 0{,}8 - \ln 0{,}5}{2} = \frac{0{,}223 + 0{,}693}{2} \approx 0{,}458$).
5. Nguyên nhân thường gặp nhất là tốc độ học η **quá lớn** nên bước nhảy vọt qua điểm tối ưu (như $\eta = 1$ ở Hình 9). Cách xử lý: giảm η (ví dụ chia 2 hoặc chia 10), đồng thời kiểm tra công thức gradient (bằng đạo hàm số) và thang đo của đặc trưng (nên chuẩn hóa).
6. Batch: 1 lần cập nhật mỗi epoch. SGD: 1000 lần. Mini-batch: $\lceil 1000/50 \rceil = 20$ lần.
7. Với $s = 1$: $\sigma(1) \approx 0{,}731$ nên $-\ln(1 - 0{,}731) = -\ln 0{,}269 \approx 1{,}313$. Với $y = 0$ thì $y_{\pm} = -1$ nên $m = -s = -1$ và $\ln(1 + e^{-m}) = \ln(1 + e^1) = \ln 3{,}718 \approx 1{,}313$. Hai giá trị bằng nhau.

4. Phân tích cơ chế học của Logistic Regression

Ở mục 3, ta đã xây dựng thuật toán Gradient Descent cho Logistic Regression và thấy nó giảm đều hàm mất mát cross-entropy. Để hiểu thật sự mô hình **học như thế nào**, ta cần trả lời bốn câu hỏi:

- Khi một mẫu bị phân loại sai (hoặc chưa chắc chắn), nó **kéo** tham số mạnh đến mức nào?
- Khởi tạo, tốc độ học và thang đo của đặc trưng **ảnh hưởng** ra sao đến kết quả?
- Có **bảo đảm** thuật toán hội tụ, và hội tụ về **nghiệm nào**?
- Điều gì xảy ra ở các trường hợp "bất thường": dữ liệu **phân tách hoàn hảo** và dữ liệu **không tuyến tính**?

So với Perceptron, bức tranh ở đây khác hẳn: Perceptron hội tụ khi dữ liệu phân tách được và quay vòng khi không, còn Logistic Regression có tính chất toán học đẹp hơn nhưng lại gặp một "bất thường" ở chiều ngược lại. Ta dùng quy ước ký hiệu cũ: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn $y_i \in \{0,1\}$, điểm số $s_i = \mathbf{x}_i\mathbf{w}$ và xác suất $p_i = \sigma(s_i)$.

4.1. Điều gì xảy ra khi một mẫu bị phân loại sai?

Mức đóng góp của một mẫu vào gradient

Ở mục 3.3, ta đã tính gradient của mất mát một mẫu: $\nabla \ell_i(\mathbf{w}) = (p_i - y_i)\,\mathbf{x}_i^T$. Độ lớn của nó là tích của hai thừa số:

$$\| \nabla \ell_i(\mathbf{w}) \| = |p_i - y_i| \cdot \| \mathbf{x}_i \|$$

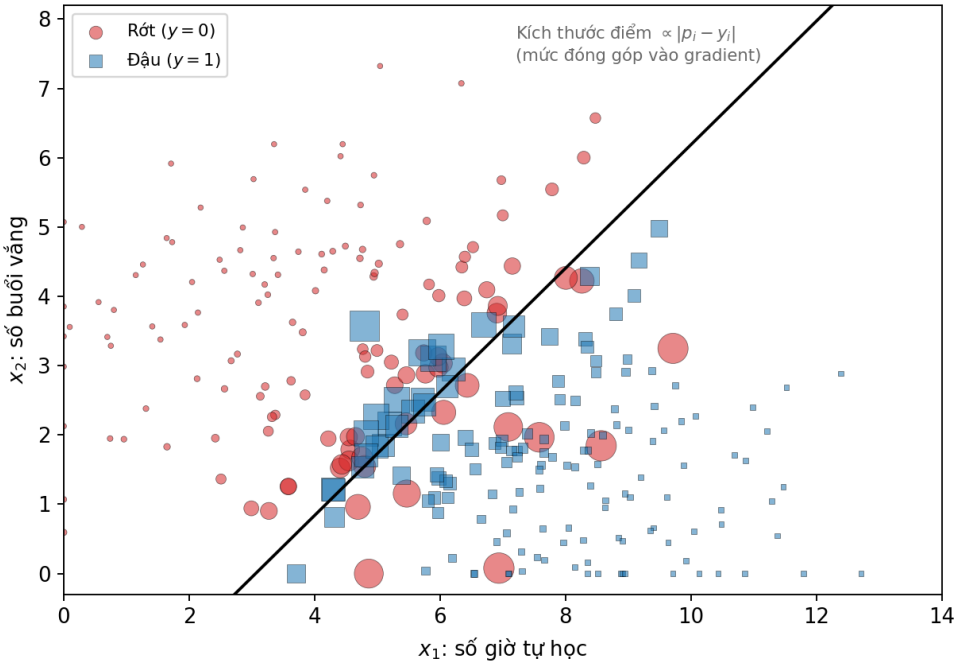
- $|p_i - y_i| \in [0,1)$ là **mức sai của xác suất**: bằng 0 khi mô hình gán xác suất 1 cho nhãn thật, và tiến tới 1 khi mô hình gán xác suất tiến tới 0 cho nhãn thật.
- $\| \mathbf{x}_i \|$ là **độ lớn của vector đặc trưng**: mẫu có đặc trưng lớn kéo mạnh hơn.

Điều quan trọng là thừa số đầu **liên tục** theo mức sai. Ta xét một mô hình đã huấn luyện xong, $\mathbf{w}^* \approx [-3{,}520 \quad 1{,}151 \quad -1{,}291]^T$ (nghiệm tối ưu trên 280 mẫu huấn luyện của mục 3.7), và tính đóng góp của một vài mẫu điển hình:

Mẫu $(x_1; x_2)$	y	s	p	$\|p - y\|$	$\| \mathbf{x} \|$	$\| \nabla \ell \|$	Tình trạng
$(2; 5)$	0	$-7{,}673$	0,0005	0,0005	5,48	0,003	Đúng, rất chắc chắn
$(9; 0{,}5)$	1	6,191	0,998	0,002	9,07	0,019	Đúng, rất chắc chắn
$(6; 3)$	0	$-0{,}488$	0,380	0,380	6,78	2,58	Đúng nhưng sát biên
$(6; 3)$	1	$-0{,}488$	0,380	0,620	6,78	4,20	Sai nhẹ, sát biên
$(4; 4)$	1	$-4{,}080$	0,017	0,983	5,75	5,65	Sai nặng
$(12; 0)$	0	10,289	≈ 1	$\approx 1{,}000$	12,04	12,04	Sai nặng và ở xa

Từ bảng có thể rút ra quy luật: **mẫu được dự đoán đúng và chắc chắn gần như không đóng góp** (gradient ≈ 0), mẫu **sát đường biên** đóng góp vừa phải, và mẫu **sai nặng** đóng góp lớn nhất. Hình 11 minh họa điều này trên toàn bộ dữ liệu: kích thước mỗi điểm tỉ lệ với $|p_i - y_i|$, và

ta thấy các điểm "to" tập trung quanh đường biên và ở phía sai, trong khi các điểm xa đường biên về phía đúng nhỏ gần như không thấy.



Hình 11. Mức đóng góp của từng mẫu huấn luyện vào gradient tại nghiệm tối ưu. Kích thước điểm tỉ lệ với $|p_i - y_i|$. Các mẫu quanh đường biên và các mẫu bị sai có ảnh hưởng lớn nhất; các mẫu nằm xa đường biên về phía đúng gần như không ảnh hưởng.

So sánh với Perceptron

Ở Perceptron (mục 3 của bài trước), một mẫu chỉ có hai trạng thái: **đúng** thì không đóng góp gì, **sai** thì kéo bằng một bước $\pm\eta\,\mathbf{x}_i^T$ có độ lớn không phụ thuộc mức sai. Logistic Regression **mềm** hơn: mức kéo thay đổi liên tục theo mức độ sai, và ngay cả mẫu đúng nhưng chưa chắc chắn ($p = 0{,}38$ cho nhãn 0 ở bảng trên) vẫn còn được "đẩy" thêm một chút. Nhờ vậy mô hình không dừng ở một đường biên sát mẫu mà tiếp tục đẩy các mẫu ra xa đường biên.

Một bước cập nhật làm điểm số của mẫu thay đổi thế nào?

Giả sử ta chỉ cập nhật theo một mẫu i: $\mathbf{w}' = \mathbf{w} - \eta\,(p_i - y_i)\,\mathbf{x}_i^T$. Điểm số mới của chính mẫu đó là

$$s_i{}' = \mathbf{x}_i\mathbf{w}' = s_i - \eta\,(p_i - y_i)\,\mathbf{x}_i\mathbf{x}_i^T = s_i + \eta\,(y_i - p_i)\;\|\,\mathbf{x}_i\,\|^2$$

Đây là phiên bản "mềm" của công thức $m' = m + \eta\,\|\,\mathbf{x}\,\|^2$ ở Perceptron: điểm số luôn dịch về **phía đúng** (tăng nếu $y_i = 1$, giảm nếu $y_i = 0$), với độ dịch tỉ lệ với mức sai $|y_i - p_i|$.

Ví dụ. Mẫu $(4; 4)$ ở bảng trên có $y = 1$, $s = -4{,}080$, $p = 0{,}0166$ và $\|\,\mathbf{x}\,\|^2 = 1 + 16 + 16 = 33$. Một bước SGD với $\eta = 0{,}1$ cho

$$s' = -4{,}080 + 0{,}1 \cdot (1 - 0{,}0166) \cdot 33 \approx -0{,}835, \qquad p' = \sigma(-0{,}835) \approx 0{,}303$$

Xác suất đúng của mẫu tăng từ 0,017 lên 0,303 chỉ sau một bước. Cần lưu ý rằng nếu $\eta \parallel \mathbf{x}_i \parallel^2$ quá lớn, bước cập nhật có thể **vọt qua** phía đúng và làm điểm số đổi dấu thái quá, đó là lý do η phải được chọn cẩn thận (mục 4.3).

Mẫu sai nặng ở xa

Mẫu cuối của bảng, $(12; 0)$ với nhãn 0, có đặc điểm nguy hiểm: cả hai thừa số cùng lớn ($|p - y| \approx 1$ và $\parallel \mathbf{x} \parallel = 12{,}04$), nên gradient của nó **gấp khoảng 3 đến 5 lần** các mẫu sát biên. Một sinh viên học 12 giờ mà rớt thường là nhãn nhiễu hoặc trường hợp đặc biệt, nhưng nó kéo đường biên mạnh hơn hẳn các mẫu thông thường. Đây là điểm yếu chung của các mô hình dựa trên gradient này, và là lý do ta cần quan tâm đến **nhãn nhiễu**, **mẫu ngoại lai** và điều chuẩn hóa (mục 4.4).

4.2. Ảnh hưởng của khởi tạo, tốc độ học và chuẩn hóa đặc trưng

Chuẩn hóa đặc trưng

Trong dữ liệu sinh viên, số giờ học x_1 nằm trong khoảng 0 đến 14 còn số buổi vắng x_2 chỉ từ 0 đến 8, cộng với cột toàn số 1 của hệ số chệch. Các đặc trưng có thang đo khác nhau làm hàm mất mát có dạng **"thung lũng dài hẹp"**: rất dốc theo một hướng và rất thoải theo hướng kia. Hình 12a vẽ các đường đồng mức của J theo (w_1, w_2) (giữ w_0 ở giá trị tối ưu) và đường đi của Gradient Descent từ $\mathbf{w} = \mathbf{0}$: đường đi **dích dắc** qua lại hai bên thung lũng và tiến rất chậm về điểm tối ưu (ngôi sao).

Cách khắc phục là **chuẩn hóa** (standardization) từng đặc trưng về trung bình 0 và độ lệch chuẩn 1:

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}, \qquad \mu_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}\,, \qquad s_j = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(x_{ij} - \mu_j\right)^2}$$

Một quy tắc rất quan trọng: μ_j và s_j chỉ được tính **từ tập huấn luyện**, rồi dùng chính các giá trị đó để chuẩn hóa tập kiểm tra. Nếu tính trên cả tập kiểm tra, thông tin của tập kiểm tra đã "rò rỉ" vào quá trình huấn luyện. Hàm sau cài đặt điều đó, kèm cách chuyển ngược tham số về thang đo gốc:

```
Z_tr, Z_te, mu, s = chuan_hoa(X_tr, X_te)        # mu, s tính từ tập huấn luyệ
n
print("mu =", mu, "| s =", s)
Zb_tr = them_bias(Z_tr)
w_z, ls = huan_luyen(Zb_tr, y_tr, eta=5.0, so_epoch=300)
print("w' (chuan hoa) =", w_z, "| J =", round(ls[-1], 6))
w_goc = np.r_[w_z[0] - np.sum(w_z[1:] * mu / s), w_z[1:] / s]
print("w (thang goc)  =", w_goc)
```

```
mu = [6.0934 2.5169] | s = [2.8113 1.6205]
w' (chuan hoa) = [ 0.243   3.2351 -2.0918] | J = 0.246559
w (thang goc)  = [-3.52    1.1508 -1.2909]
```

Công thức chuyển ngược xuất phát từ việc điểm số không đổi, $\mathbf{zw}' = \mathbf{xw}$:

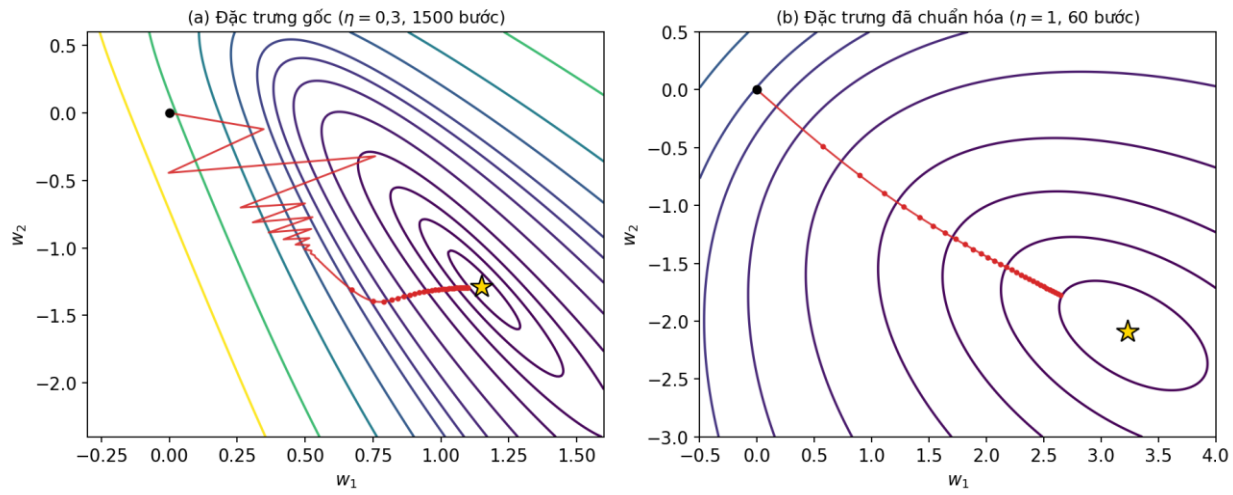
$$w_0{}' + \sum_{j=1}^{d} w_j{}' \frac{x_j - \mu_j}{s_j} = \left(w_0{}' - \sum_{j=1}^{d} \frac{w_j{}' \mu_j}{s_j} \right) + \sum_{j=1}^{d} \frac{w_j{}'}{s_j} x_j \quad \Rightarrow \quad w_j = \frac{w_j{}'}{s_j}, \quad w_0 = w_0{}' - \sum_{j=1}^{d} \frac{w_j{}' \mu_j}{s_j}$$

Kết quả $\mathbf{w} \approx [-3{,}520 \quad 1{,}151 \quad -1{,}291]^T$ **trùng** với nghiệm tối ưu tính trực tiếp trên đặc trưng gốc. Chuẩn hóa chỉ thay đổi **cách đi** đến nghiệm, không thay đổi nghiệm (theo thang gốc).

Hình 12b cho thấy sau chuẩn hóa các đường đồng mức gần tròn hơn nhiều và đường đi của Gradient Descent đi **thẳng** về điểm tối ưu. Bảng sau so sánh số epoch cần để J chỉ còn cách mức tối ưu $J^* = 0{,}24656$ không quá $0{,}001$:

Tốc độ học η	Đặc trưng gốc	Đặc trưng đã chuẩn hóa
0,05	> 5000	1976
0,1	3119	987
0,2	1559	493
0,5	622	196
0,8	không hội tụ (dao động)	122
1	không hội tụ (dao động)	97
2	không hội tụ (dao động)	48
5	không hội tụ (dao động)	16

Sau chuẩn hóa, mô hình không chỉ nhanh hơn nhiều lần mà còn **chịu được tốc độ học lớn hơn rất nhiều** (thử đến $\eta = 10$ vẫn hội tụ, trong khi với đặc trưng gốc $\eta = 0{,}8$ đã dao động). Mục 4.3 giải thích vì sao.



Hình 12. Các đường đồng mức của hàm mất mát J theo (w_1, w_2) (giữ w_0 ở giá trị tối ưu) và đường đi của Gradient Descent từ $\mathbf{w} = \mathbf{0}$ (chấm đen) đến nghiệm tối ưu (ngôi sao). (a) Đặc trưng gốc: thung lũng dài hẹp, đường đi dích dắc và chậm. (b) Đặc trưng đã chuẩn hóa: các đường đồng mức gần tròn, đường đi thẳng.

Chuẩn hóa còn làm trọng số dễ so sánh hơn. Trên đặc trưng chuẩn hóa, $w_1' = 3{,}235$ và $w_2' = -2{,}092$ có cùng đơn vị ("mỗi một độ lệch chuẩn"), nên odds ratio (mục 2.5) cho biết: tăng một độ lệch chuẩn số giờ học (khoảng 2,81 giờ) nhân odds đậu với $e^{3{,}235} \approx 25{,}4$, còn tăng một độ lệch chuẩn số buổi vắng nhân odds với $e^{-2{,}092} \approx 0{,}123$. Từ đó thấy số giờ học là yếu tố có ảnh hưởng mạnh hơn.

Khởi tạo tham số

Thông thường ta khởi tạo $\mathbf{w} = \mathbf{0}$, khi đó mọi $p_i = 0{,}5$ và $J = \ln 2$. Câu hỏi là: khởi tạo khác có làm kết quả cuối khác không? Khác với Perceptron (mục 4.1 của bài trước: thứ tự và khởi tạo cho ra những nghiệm khác nhau), **Logistic Regression cho cùng một nghiệm với mọi khởi tạo**. Ta kiểm chứng trên đặc trưng chuẩn hóa với $\eta = 5$, từ bốn khởi tạo khác hẳn nhau:

Khởi tạo $\mathbf{w}^{(0)}$	$\mathbf{w}$ sau 300 epoch	J
$[0 \quad 0 \quad 0]^T$	$[0{,}2430 \quad 3{,}2351 \quad -2{,}0918]^T$	0,246559
$[5 \quad -3 \quad 2]^T$	$[0{,}2430 \quad 3{,}2351 \quad -2{,}0918]^T$	0,246559
$[-8 \quad 4 \quad 4]^T$	$[0{,}2430 \quad 3{,}2351 \quad -2{,}0918]^T$	0,246559
$[2 \quad -6 \quad -6]^T$	$[0{,}2430 \quad 3{,}2351 \quad -2{,}0918]^T$	0,246559

Cả bốn đường cong học (Hình 13b) bắt đầu từ các mức J rất khác nhau (có khởi tạo cho J ban đầu lên đến 4,2) nhưng đều **hội tụ về cùng một điểm**. Khởi tạo chỉ ảnh hưởng đến **số bước** cần thiết, không ảnh hưởng đến đích. Lý do là hàm mất mát **lồi**, như ta chứng minh ở mục 4.3.

4.3. Hàm mất mát lồi và hội tụ về nghiệm duy nhất

Ma trận Hessian

Để biết hình dạng của $J(\mathbf{w})$, ta tính đạo hàm bậc hai. Từ gradient $\nabla J = \frac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y})$ và $\frac{\partial p_i}{\partial \mathbf{w}} = p_i(1 - p_i)\,\mathbf{x}_i$ (theo đạo hàm của sigmoid, mục 2.3), ta được **ma trận Hessian**:

$$\mathbf{H}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n} p_i\,(1 - p_i)\,\mathbf{x}_i^T\mathbf{x}_i = \frac{1}{n}\,\mathbf{X}^T\mathbf{S}\,\mathbf{X}, \qquad \mathbf{S} = \operatorname{diag}\big(p_1(1 - p_1), \dots, p_n(1 - p_n)\big)$$

Ở đây $\mathbf{x}_i^T\mathbf{x}_i$ là ma trận $(d + 1) \times (d + 1)$ (tích ngoài) và $\mathbf{S}$ là ma trận đường chéo $n \times n$ chứa các số dương $p_i(1 - p_i)$. Kích thước của $\mathbf{H}$ là $(d + 1) \times (d + 1)$.

Hàm mất mát là hàm lồi

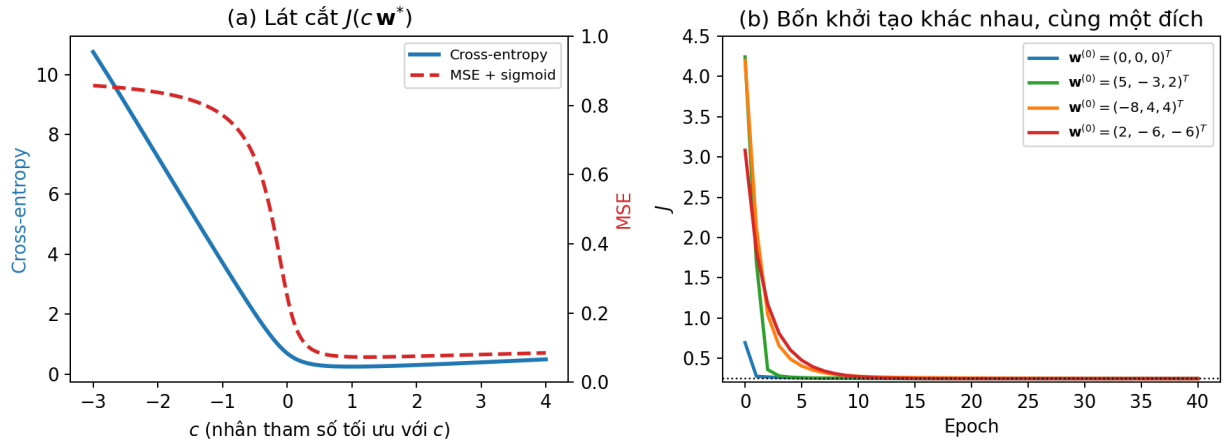
Với **mọi** vector $\mathbf{z} \in \mathbb{R}^{d+1}$:

$$\mathbf{z}^T\mathbf{H}\,\mathbf{z} = \frac{1}{n}\sum_{i=1}^{n} p_i\,(1 - p_i)\,(\mathbf{x}_i\mathbf{z})^2 \geq 0$$

vì mỗi số hạng là tích của một số dương $p_i(1 - p_i)$ và một bình phương. Vậy $\mathbf{H}$ là ma trận **nửa xác định dương** tại mọi $\mathbf{w}$, nghĩa là J là hàm **lồi**. Hệ quả:

- **Mọi cực tiểu địa phương đều là cực tiểu toàn cục.** Không có chuyện bị "kẹt" ở một điểm tệ.
- Gradient Descent với tốc độ học phù hợp **hội tụ về cực tiểu toàn cục từ mọi khởi tạo** (đúng như thí nghiệm ở mục 4.2).
- Nếu các cột của $\mathbf{X}$ độc lập tuyến tính thì $\mathbf{H}$ **xác định dương** (vì mọi $p_i(1 - p_i) > 0$), và cực tiểu là **duy nhất**. Trên dữ liệu sinh viên, các trị riêng của $\mathbf{H}$ tại nghiệm tối ưu là $0{,}0046$; $0{,}0688$ và $3{,}502$, đều dương, nên nghiệm là duy nhất.

Hình 13a minh họa sự khác biệt với MSE: trên cùng một lát cắt (nhân nghiệm tối ưu với một hệ số c), cross-entropy là một đường **lồi** một cực tiểu, còn MSE kết hợp sigmoid có dạng chữ **S** (có đoạn cong lõm), tức là không lồi. Đây là lý do thứ hai (cùng với "gradient tắt" ở mục 3.1) để chọn cross-entropy.



Hình 13. (a) Lát cắt của hàm mất mát theo hướng $\mathbf{w}^*$: cross-entropy là hàm lồi, còn MSE kết hợp sigmoid có dạng chữ S (không lồi). (b) Gradient Descent từ bốn khởi tạo rất khác nhau (đặc trưng đã chuẩn hóa, $\eta = 5$) đều hội tụ về cùng một giá trị J.

Điều kiện tối ưu có ý nghĩa gì?

Tại nghiệm tối ưu, gradient bằng $\mathbf{0}$:

$$\mathbf{X}^T(\mathbf{p} - \mathbf{y}) = \mathbf{0} \quad \Leftrightarrow \quad \sum_{i=1}^{n}(p_i - y_i) = 0 \quad \text{và} \quad \sum_{i=1}^{n}(p_i - y_i)\,x_{ij} = 0 \quad (j = 1, \ldots, d)$$

Dòng đầu (ứng với cột 1 của hệ số chệch) cho

$$\frac{1}{n}\sum_{i=1}^{n} p_i = \frac{1}{n}\sum_{i=1}^{n} y_i$$

tức là **trung bình xác suất dự đoán bằng đúng tỉ lệ lớp dương thực tế** trong tập huấn luyện. Ta kiểm chứng: trên 280 mẫu huấn luyện có 149 sinh viên đậu, tỉ lệ thực tế là $149/280 = 0{,}5321$ và trung bình xác suất dự đoán cũng là $0{,}5321$. Các dòng còn lại nói rằng sai số xác suất $p_i - y_i$ **không tương quan** với bất kỳ đặc trưng nào: mô hình đã "khai thác hết" thông tin tuyến tính trong đặc trưng.

Độ cong quyết định tốc độ và độ ổn định

Gần điểm tối ưu, hàm J xấp xỉ một hàm bậc hai với ma trận $\mathbf{H}$. Trên hàm bậc hai, Gradient Descent hội tụ khi và chỉ khi $|1 - \eta\lambda| < 1$ với **mọi** trị riêng λ của $\mathbf{H}$, tức là

$$\eta < \frac{2}{\lambda_{\max}(\mathbf{H})}$$

Trị riêng lớn $\lambda_{\max}$ quyết định η tối đa cho phép, còn trị riêng nhỏ $\lambda_{\min}$ quyết định tốc độ tiến dọc theo hướng thoải nhất (cỡ $1/(\eta\lambda_{\min})$ bước). Tỉ số $\kappa = \lambda_{\max}/\lambda_{\min}$ (số điều kiện) đo độ "dài hẹp" của thung lũng:

	Đặc trưng gốc	Đặc trưng chuẩn hóa
$\lambda_{\max}$	3,502	0,0775
$\lambda_{\min}$	0,0046	0,0135
Số điều kiện κ	≈ 765	$\approx 5{,}75$
Ngưỡng ổn định $2/\lambda_{\max}$	$\approx 0{,}57$	$\approx 25{,}8$

Bảng này giải thích trọn vẹn các quan sát trước đó:

- Với đặc trưng gốc, ngưỡng ổn định là $\eta \approx 0{,}57$: $\eta = 0{,}5$ hội tụ còn $\eta = 0{,}8$ và $\eta = 1$ dao động, đúng như ở mục 3.4 và bảng ở mục 4.2.
- Chuẩn hóa làm số điều kiện giảm từ 765 xuống 5,75 (thung lũng gần tròn) và nâng ngưỡng ổn định lên $\approx 25{,}8$, nên có thể dùng η lớn hơn nhiều và hội tụ nhanh hơn nhiều.

So sánh với Perceptron

	Perceptron	Logistic Regression
Hàm mất mát	Không trơn, chỉ quan tâm mẫu sai	Trơn và **lồi**
Khi dữ liệu chồng lấn	Không hội tụ, $\mathbf{w}$ dao động (cần thuật toán túi)	**Hội tụ êm** về nghiệm duy nhất
Kết quả phụ thuộc khởi tạo, thứ tự	Có (nhiều nghiệm)	**Không** (một nghiệm duy nhất)
Điều kiện dừng	Một epoch không còn mẫu sai	Gradient nhỏ, J không giảm nữa, hoặc $T_{\max}$

Một ghi chú: các thư viện như scikit-learn không dùng Gradient Descent đơn giản mà dùng các phương pháp có dùng thông tin của $\mathbf{H}$ (như L-BFGS hay phương pháp Newton). Nhờ tính lồi, chúng hội tụ rất nhanh về cùng nghiệm với Gradient Descent.

4.4. Trường hợp dữ liệu phân tách hoàn hảo: $\mathbf{w}$ tăng không giới hạn

Hiện tượng

Dữ liệu 6 sinh viên của mục 1 **phân tách tuyến tính được** hoàn toàn. Với Perceptron đó là tình huống thuận lợi (thuật toán dừng). Với Logistic Regression, đó lại là tình huống "bất thường". Lý do: nếu có $\mathbf{w}^*$ phân loại đúng mọi mẫu, tức mọi margin $m_i = (2y_i - 1)\,\mathbf{x}_i\mathbf{w}^* > 0$, thì nhân $\mathbf{w}^*$ với một hệ số $c > 0$ lớn cho

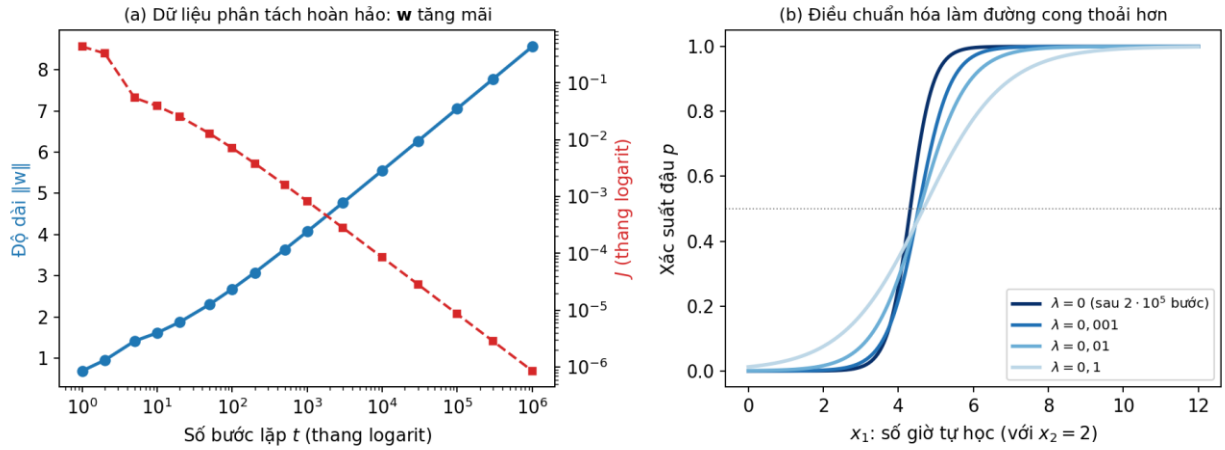
$$J(c\,\mathbf{w}^*) = \frac{1}{n}\sum_{i=1}^{n} \ln\left(1 + e^{-c\,m_i}\right) \underset{c\to\infty}{\to} 0$$

Mất mát có thể nhỏ **tùy ý** nhưng **không bao giờ bằng** 0 (vì $\ln(1 + e^{-cm}) > 0$ với mọi c), nên **không tồn tại cực tiểu hữu hạn**: muốn J nhỏ hơn nữa, $\mathbf{w}$ phải lớn hơn nữa. Tính lồi vẫn đúng, nhưng "đáy thung lũng" nằm ở vô cực.

Ta kiểm chứng bằng cách chạy Gradient Descent ($\eta = 0{,}5$) trên 6 sinh viên rất lâu:

Số bước t	$\| \mathbf{w} \|$	J
10	1,62	$3{,}9 \cdot 10^{-2}$
100	2,68	$7{,}1 \cdot 10^{-3}$
1000	4,07	$8{,}3 \cdot 10^{-4}$
10^4	5,56	$8{,}5 \cdot 10^{-5}$
10^5	7,06	$8{,}5 \cdot 10^{-6}$
10^6	8,56	$8{,}6 \cdot 10^{-7}$

Hình 14a vẽ kết quả này. Có thể thấy $\| \mathbf{w} \|$ **tăng mãi** (khoảng 1,5 mỗi khi số bước tăng 10 lần, tức tăng chậm cỡ logarit), còn J giảm cỡ $1/t$ nhưng không bao giờ đạt 0. Ngoài ra, hướng của $\mathbf{w}$ có một tính chất đẹp: độ tương đồng (cosin) giữa hướng của $\mathbf{w}$ và hướng $[-1 \quad 2 \quad -4]$ (hướng có margin lớn nhất, đã tính ở mục 4.2 của bài Perceptron) tăng từ 0,9925 ở bước 10 lên 0,9980 ở bước 10^6. Đây là một kết quả lý thuyết đã được chứng minh: trên dữ liệu phân tách được, Gradient Descent cho Logistic Regression **dần tiến tới hướng của đường biên có margin lớn nhất** (nghiệm của Support Vector Machine), dù độ dài của $\mathbf{w}$ tăng không ngừng.



Hình 14. (a) Trên dữ liệu phân tách hoàn hảo, độ dài $\| \mathbf{w} \|$ tăng không ngừng (cỡ logarit theo số bước) trong khi J giảm dần về 0 mà không bao giờ đạt. (b) Điều chuẩn hóa L2 giữ $\mathbf{w}$ hữu hạn: λ càng lớn thì đường cong xác suất càng thoải.

Hệ quả

- **Xác suất cực đoan và quá tự tin.** Khi $\mathbf{w}$ lớn, sigmoid rất dốc: mọi mẫu, kể cả mẫu sát đường biên, đều được gán xác suất gần 0 hoặc gần 1. Ví dụ sinh viên B (5 giờ, vắng 2 buổi) có $p = 0{,}907$ sau $2 \cdot 10^5$ bước, dù cách đường biên không xa.
- **Không có điểm dừng tự nhiên.** Thuật toán chỉ dừng vì $T_{\max}$ hoặc ngưỡng ε do ta đặt. Việc dừng sớm này vô hình trung đóng vai trò điều chuẩn hóa.
- **Kết quả phụ thuộc vào điều kiện dừng và bộ giải**, vì không có nghiệm duy nhất để các phương pháp cùng hướng tới.

Tình huống này không hiếm trong thực tế, nhất là khi số đặc trưng lớn so với số mẫu (dữ liệu càng nhiều chiều càng dễ phân tách).

Điều chuẩn hóa L2

Giải pháp phổ biến là thêm vào hàm mất mát một **số hạng phạt** độ lớn của các trọng số, gọi là **điều chuẩn hóa L2** (regularization):

$$J_\lambda(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2}\sum_{j=1}^{d} w_j^2 = J(\mathbf{w}) + \frac{\lambda}{2}\,\mathbf{w}^T\mathbf{D}_0\,\mathbf{w}, \qquad \mathbf{D}_0 = \mathrm{diag}(0,1,1,\dots,1)$$

Trong đó $\lambda \geq 0$ là **hệ số điều chuẩn hóa**, và ma trận đường chéo $\mathbf{D}_0$ có phần tử đầu bằng 0 để **không phạt hệ số chệch** w_0 (vì w_0 chỉ điều chỉnh xác suất nền, không phải độ nhạy với đặc trưng). Gradient và quy tắc cập nhật trở thành

$$\nabla J_\lambda(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{p}-\mathbf{y}) + \lambda\,\mathbf{D}_0\,\mathbf{w}, \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla J_\lambda(\mathbf{w})$$

Số hạng mới $\lambda\mathbf{D}_0\mathbf{w}$ luôn **kéo $\mathbf{w}$ về $\mathbf{0}$**, chống lại xu hướng tăng vô hạn. Hàm J_λ vẫn lồi (cộng một hàm lồi với một hàm bậc hai lồi), và giờ **có cực tiểu hữu hạn duy nhất** ngay cả khi dữ liệu phân tách hoàn hảo. Mã nguồn chỉ cần thêm tham số *lam* vào hàm huấn luyện:

```
X_sv = np.array([[2, 4], [3, 3], [4, 5], [6, 1], [8, 2], [9, 0]], dtype=float
)
y_sv = np.array([0, 0, 0, 1, 1, 1])
Xb_sv = them_bias(X_sv)
for lam in (0.0, 0.001, 0.01, 0.1):
    w_l, ls = huan_luyen(Xb_sv, y_sv, eta=0.5, so_epoch=200000, lam=lam)
    p_B = sigmoid(np.array([1, 5, 2]) @ w_l)
    print(f"lam = {lam:<6} w = {w_l} | |w~| = {np.linalg.norm(w_l[1:]):.3f} |
p(sv B) = {p_B:.3f}")
```

```
lam = 0.0    w = [-1.1577  3.3382 -6.6285] | |w~| = 7.422 | p(sv B) = 0.907
lam = 0.001  w = [-6.9733  2.3751 -1.9   ] | |w~| = 3.042 | p(sv B) = 0.751
lam = 0.01   w = [-4.742   1.5997 -1.2854] | |w~| = 2.052 | p(sv B) = 0.665
lam = 0.1    w = [-2.9394  0.9385 -0.7219] | |w~| = 1.184 | p(sv B) = 0.577
```

Bảng cho thấy hiệu ứng rất rõ: $\lambda = 0$ cho $\|\,\tilde{\mathbf{w}}\,\|$ lớn và vẫn đang tăng (sau $2 \cdot 10^5$ bước), còn $\lambda > 0$ cho một **$\mathbf{w}$ hữu hạn, ổn định**, và λ càng lớn thì $\mathbf{w}$ càng nhỏ. Xác suất của sinh viên B cũng **dịu đi** từ 0,907 xuống 0,751; 0,665 và 0,577, gần 0,5 hơn, vì mô hình dè dặt hơn (Hình 14b).

Có một **đánh đổi**: λ lớn quá thì mô hình bị "bó" và không khớp dữ liệu tốt (độ chính xác giảm, xác suất bị kéo về 0,5), còn λ nhỏ quá thì không chống được hiện tượng trên. Giá trị λ phù hợp thường được chọn bằng cách thử trên một tập kiểm định riêng (mục 5).

> **Liên hệ với scikit-learn.** *LogisticRegression* mặc định đã dùng điều chuẩn hóa L2 và tham số *C* điều khiển độ mạnh của nó. Hàm mục tiêu của thư viện là $C\sum_i \ell_i + \frac{1}{2}\|$

$\tilde{\mathbf{w}}\|^2$, tương đương $\lambda = \frac{1}{C\,n}$ trong cách viết ở trên. C càng nhỏ thì điều chuẩn hóa càng mạnh. Ta sẽ gặp lại ở mục 5.5.

Điều này cũng đối lập với Perceptron: Perceptron **dừng** ngay khi tìm được một đường biên phân tách, còn Logistic Regression thì "không bao giờ hài lòng" và luôn muốn đẩy các mẫu ra xa hơn nữa, nên trong thực tế nó luôn đi kèm điều chuẩn hóa.

4.5. Giới hạn của đường biên tuyến tính: bài toán XOR và đặc trưng đa thức

Logistic Regression không giải được XOR

Vì $p = 0{,}5$ khi và chỉ khi $\mathbf{xw} = 0$, đường biên của Logistic Regression vẫn là siêu phẳng, nên mô hình **không phân loại được XOR** (hay bất kỳ dữ liệu không phân tách tuyến tính được). Nhưng cách "thất bại" khác hẳn Perceptron. Với XOR ($y = [0 \quad 1 \quad 1 \quad 0]^T$) và ma trận $\mathbf{X}$ ở mục 4.3 của bài Perceptron, gradient tại $\mathbf{w} = \mathbf{0}$ (khi mọi $p_i = 0{,}5$) là

$$\nabla J(\mathbf{0}) = \frac{1}{4}\mathbf{X}^T(\mathbf{0},5 \cdot \mathbf{1} - \mathbf{y}) = \frac{1}{4}\begin{bmatrix}1 & 1 & 1 & 1\\ 0 & 0 & 1 & 1\\ 0 & 1 & 0 & 1\end{bmatrix}\begin{bmatrix}0{,}5\\ -0{,}5\\ -0{,}5\\ 0{,}5\end{bmatrix} = \begin{bmatrix}0\\0\\0\end{bmatrix}$$

Gradient bằng $\mathbf{0}$ ngay tại $\mathbf{w} = \mathbf{0}$, và vì J lồi nên **$\mathbf{w} = \mathbf{0}$ chính là cực tiểu toàn cục duy nhất**. Ta chạy Gradient Descent từ khởi tạo xa, $\mathbf{w}^{(0)} = [2 \quad -3 \quad 1]^T$, và thu được $\mathbf{w} \approx \mathbf{0}$ với $p = 0{,}5$ cho cả bốn mẫu và $J = \ln 2 \approx 0{,}693$.

Cách thất bại này **khác hẳn Perceptron**: Perceptron quay vòng mãi không dừng (mục 4.3 của bài trước), còn Logistic Regression **hội tụ êm** về một mô hình nói rằng "xác suất đậu của mọi trường hợp đều là 50%", tức là mô hình trung thực thừa nhận rằng đặc trưng tuyến tính **không cho thông tin gì** về nhãn. Điều này có ích vì xác suất 0,5 cảnh báo ta rằng mô hình không biết.

Thêm đặc trưng đa thức

Như ở mục 4.3 của bài Perceptron, ta có thể thêm các đặc trưng phi tuyến để dữ liệu trở nên phân tách được trong không gian mới. Với XOR, thêm đặc trưng x_1x_2, vector đặc trưng trở thành $\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad x_1x_2]$. Dữ liệu nay phân tách tuyến tính được, nhưng khi đó ta gặp ngay hiện tượng ở mục 4.4: nếu $\lambda = 0$ thì $\mathbf{w}$ tăng không giới hạn (sau 10^5 bước, $\|\tilde{\mathbf{w}}\|$ đã khoảng 37 và vẫn tăng). Cần điều chuẩn hóa L2:

λ	$\mathbf{w}$	Xác suất p cho (0,0), (0,1), (1,0), (1,1)
0,001	$[-2{,}389 \quad 5{,}076 \quad 5{,}076 \quad -10{,}856]^T$	0,084; 0,936; 0,936; 0,043
0,01	$[-0{,}735 \quad 1{,}827 \quad 1{,}827 \quad -4{,}450]^T$	0,324; 0,749; 0,749; 0,178

Với cả hai giá trị λ, ngưỡng 0,5 cho nhãn $\hat{\mathbf{y}} = [0 \quad 1 \quad 1 \quad 0]^T = \mathbf{y}$, đúng với XOR. λ nhỏ cho xác suất quyết liệt hơn, λ lớn cho xác suất dè dặt hơn (đúng như quan sát ở mục 4.4).

Ví dụ: dữ liệu hình vòng tròn

Một ví dụ trực quan hơn: hai lớp có dạng **vòng tròn đồng tâm**, lớp 1 ở bên trong và lớp 0 ở vòng ngoài (300 điểm: 210 huấn luyện và 90 kiểm tra). Không có đường thẳng nào tách được hai lớp này. Ta so sánh hai mô hình, cùng được huấn luyện bằng Gradient Descent trên đặc trưng đã chuẩn hóa:

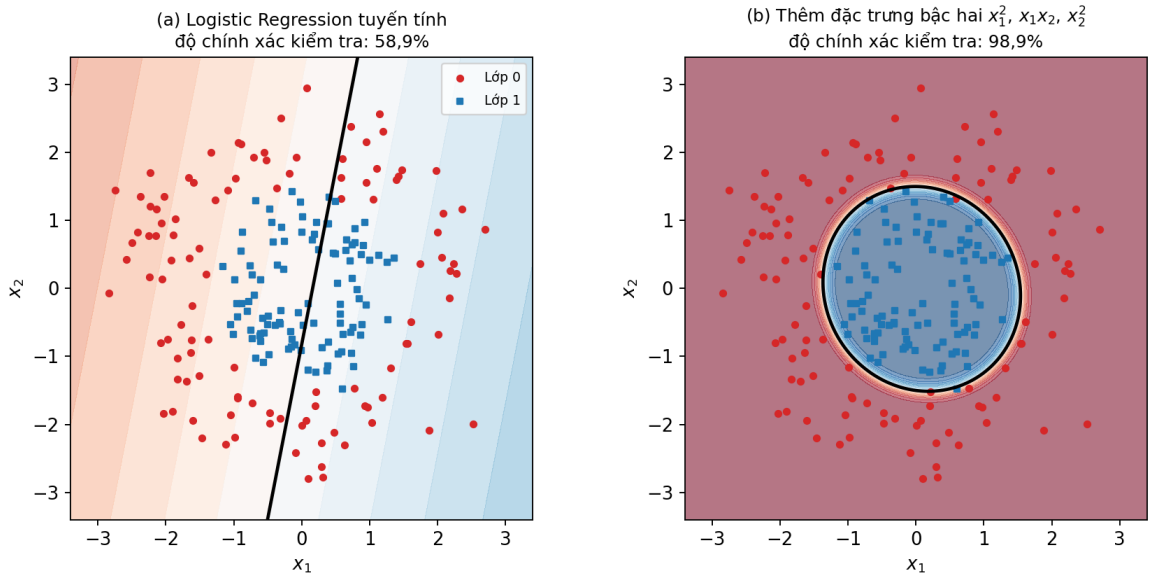
- **Logistic Regression tuyến tính** với $\mathbf{x} = [1 \quad x_1 \quad x_2]$.
- **Logistic Regression với đặc trưng bậc hai** $\mathbf{x}' = [1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1x_2 \quad x_2^2]$.

Mô hình	Số tham số	Accuracy huấn luyện	Accuracy kiểm tra
Tuyến tính	3	51,0%	58,9%
Đặc trưng bậc hai	6	99,0%	98,9%

Mô hình tuyến tính gần như **đoán mò** (chỉ hơn 50% một chút), trong khi chỉ cần thêm ba đặc trưng bậc hai, độ chính xác tăng vọt lên gần 99%. Hình 15 cho thấy lý do: đường biên $p = 0{,}5$ của mô hình thứ hai là phương trình

$$w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2 = 0$$

tức là một **đường conic** (ở đây là một đường tròn) trong không gian đặc trưng gốc, dù mô hình vẫn **tuyến tính theo các đặc trưng mới**.



Hình 15. Dữ liệu hình vòng tròn. (a) Logistic Regression tuyến tính chỉ có thể vẽ một đường thẳng nên gần như đoán mò. (b) Thêm các đặc trưng bậc hai cho phép đường biên là một đường tròn, phân loại gần như hoàn hảo. Nền màu thể hiện xác suất p; đường đen là $p = 0{,}5$.

Cái giá của đặc trưng đa thức

Số đặc trưng đa thức tăng rất nhanh. Với d đặc trưng gốc và bậc tối đa k, số đặc trưng (không kể hệ số chệch) là

$$\binom{d+k}{k} - 1$$

Ví dụ, $d = 2$ và $k = 2$ cho $\binom{4}{2} - 1 = 5$ đặc trưng (như trên), còn $d = 10$ và $k = 3$ cho $\binom{13}{3} - 1 = 285$ đặc trưng. Nhiều đặc trưng làm mô hình dễ **học thuộc** dữ liệu huấn luyện và dữ liệu dễ trở nên phân tách hoàn hảo (mục 4.4), nên điều chuẩn hóa L2 và chuẩn hóa đặc trưng gần như luôn đi kèm. Ngoài ra, cách này đòi hỏi ta phải **tự đoán** dạng đặc trưng phù hợp. Một hướng tổng quát hơn là để mô hình **tự học đặc trưng** bằng cách xếp nhiều lớp, dẫn đến **mạng nơ-ron**, nội dung của mục 6.

Tóm tắt mục 4

- **Đóng góp của mỗi mẫu** vào gradient có độ lớn $|p_i - y_i| \cdot \| \mathbf{x}_i \|$: mẫu đúng và chắc chắn gần như không đóng góp, mẫu sát biên đóng góp vừa phải, mẫu sai nặng đóng góp lớn nhất. Một bước cập nhật dịch điểm số của mẫu về phía đúng: $s_i' = s_i + \eta\,(y_i - p_i)\,\| \mathbf{x}_i \|^2$.
- **Chuẩn hóa đặc trưng** $z_{ij} = (x_{ij} - \mu_j)/s_j$ (tính từ tập huấn luyện) làm hàm mất mát gần "tròn", giảm số điều kiện từ ≈ 765 xuống $\approx 5{,}75$ và cho phép dùng η lớn hơn nhiều. **Khởi tạo** không ảnh hưởng nghiệm cuối.
- **Hàm mất mát lồi**: Hessian $\mathbf{H} = \frac{1}{n}\mathbf{X}^T\mathbf{S}\mathbf{X}$ nửa xác định dương, nên mọi cực tiểu địa phương là toàn cục và (khi $\mathbf{H}$ xác định dương) nghiệm là **duy nhất**. Tại nghiệm, trung bình xác suất dự đoán bằng tỉ lệ lớp dương. Gradient Descent ổn định khi $\eta < 2/\lambda_{\max}(\mathbf{H})$.
- **Dữ liệu phân tách hoàn hảo**: $J \to 0$ nhưng không đạt, $\| \mathbf{w} \|$ tăng mãi (xác suất cực đoan, quá tự tin). **Điều chuẩn hóa L2** $J_\lambda = J + \frac{\lambda}{2}\mathbf{w}^T\mathbf{D}_0\mathbf{w}$ đảm bảo nghiệm hữu hạn duy nhất, với đánh đổi giữa khớp dữ liệu và độ lớn của $\mathbf{w}$.
- **Đường biên vẫn là siêu phẳng**. Với XOR, Logistic Regression hội tụ êm về $\mathbf{w} = \mathbf{0}$ ($p = 0{,}5$ ở mọi nơi) thay vì quay vòng như Perceptron. **Đặc trưng đa thức** (kèm điều chuẩn hóa) cho phép đường biên cong, nhưng số đặc trưng tăng rất nhanh.

Câu hỏi kiểm tra nhanh

8. Một mẫu có $y = 1$, $p = 0{,}2$ và $\| \mathbf{x} \|^2 = 10$. Tính $|p - y|$ và điểm số mới s' sau một bước SGD chỉ theo mẫu này với $\eta = 0{,}05$. Xác suất mới là bao nhiêu?
9. So sánh độ lớn gradient (theo điểm số) của hai mẫu cùng nhãn $y = 1$ với $p = 0{,}05$ và $p = 0{,}45$. Mẫu nào kéo mạnh hơn và gấp mấy lần?
10. Số giờ học có $\mu = 6{,}093$ và $s = 2{,}811$ (tính từ tập huấn luyện). Tính giá trị chuẩn hóa của sinh viên học 9 giờ và sinh viên học 3 giờ. Ý nghĩa của dấu của z là gì?
11. Vì sao khởi tạo không ảnh hưởng đến nghiệm cuối của Logistic Regression, nhưng lại ảnh hưởng đến nghiệm của Perceptron?
12. Chứng minh ngắn gọn rằng gradient của XOR tại $\mathbf{w} = \mathbf{0}$ bằng $\mathbf{0}$ (có thể dùng tính đối xứng). Từ đó nêu kết luận về mô hình thu được.
13. Hai mẫu có margin $m = 1$ và $m = 2$. Tính $J(c\,\mathbf{w}^*) = \frac{1}{2}[\ln(1 + e^{-c}) + \ln(1 + e^{-2c})]$ với $c = 1$, $c = 5$ và $c = 10$. Nhận xét.

14. Viết gradient của J_λ và giải thích vì sao ta không phạt hệ số chệch w_0.
15. Với $d = 4$ đặc trưng gốc, bậc tối đa $k = 3$, có bao nhiêu đặc trưng đa thức (không kể hệ số chệch)?

Gợi ý đáp án

16. $|p - y| = 0{,}8$. Điểm số hiện tại $s = \ln\frac{0{,}2}{0{,}8} \approx -1{,}386$. Sau một bước: $s' = s + \eta(y - p)\,\|\,\mathbf{x}\,\|^2 = -1{,}386 + 0{,}05 \cdot 0{,}8 \cdot 10 = -0{,}986$, nên $p' = \sigma(-0{,}986) \approx 0{,}272$ (tăng từ 0,2).
17. Độ lớn gradient theo điểm số là $|p - y|$: 0,95 và 0,55. Mẫu có $p = 0{,}05$ (sai chắc chắn hơn) kéo mạnh hơn, gấp $0{,}95/0{,}55 \approx 1{,}73$ lần.
18. $z(9) = \frac{9-6{,}093}{2{,}811} \approx 1{,}034$ và $z(3) = \frac{3-6{,}093}{2{,}811} \approx -1{,}100$. Dấu dương nghĩa là học nhiều hơn mức trung bình, dấu âm nghĩa là ít hơn mức trung bình; độ lớn cho biết lệch bao nhiêu độ lệch chuẩn.
19. Logistic Regression có hàm mất mát lồi với nghiệm duy nhất nên mọi con đường đều dẫn đến cùng đích, khởi tạo chỉ ảnh hưởng số bước. Perceptron cho dữ liệu phân tách được có **vô số** nghiệm đúng và thuật toán dừng ngay ở nghiệm đầu tiên nó gặp, nên khởi tạo (và thứ tự mẫu) quyết định nghiệm nào.
20. $\mathbf{X}^T(0{,}5 \cdot \mathbf{1} - \mathbf{y}) = \mathbf{X}^T[0{,}5, -0{,}5, -0{,}5, 0{,}5]^T$. Cột đầu: $0{,}5 - 0{,}5 - 0{,}5 + 0{,}5 = 0$. Cột x_1 (chỉ khác 0 ở mẫu 3 và 4): $-0{,}5 + 0{,}5 = 0$. Cột x_2 (khác 0 ở mẫu 2 và 4): $-0{,}5 + 0{,}5 = 0$. Vậy gradient $= \mathbf{0}$ và vì J lồi nên $\mathbf{w} = \mathbf{0}$ là cực tiểu toàn cục: mô hình gán $p = 0{,}5$ cho mọi mẫu.
21. $c = 1$: $1/2\,(0{,}3133 + 0{,}1269) \approx 0{,}2201$. $c = 5$: $1/2\,(0{,}00672 + 0{,}0000454) \approx 0{,}00338$. $c = 10$: $1/2\,(0{,}0000454 + 0{,}000000002) \approx 0{,}0000227$. J giảm rất nhanh về 0 khi c tăng nhưng luôn dương, nên không có cực tiểu hữu hạn.
22. $\nabla J_\lambda(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y}) + \lambda\,\mathbf{D}_0\,\mathbf{w}$ với $\mathbf{D}_0 = \mathrm{diag}(0, 1, \ldots, 1)$. Không phạt w_0 vì nó chỉ quyết định xác suất nền (tỉ lệ lớp), nên phạt nó sẽ ép xác suất trung bình lệch khỏi tỉ lệ thực tế mà không giảm được độ phức tạp của mô hình.
23. $\binom{4+3}{3} - 1 = 35 - 1 = 34$ đặc trưng.

5. Đánh giá và trực quan hóa mô hình Logistic Regression

Huấn luyện xong một mô hình mới chỉ là một nửa công việc. Câu hỏi quan trọng không kém là: **mô hình tốt đến đâu, và ta có thể tin vào xác suất mà nó đưa ra đến mức nào?** Vì Logistic Regression cho ra **xác suất**, ta có nhiều công cụ đánh giá hơn Perceptron: ngoài độ chính xác của nhãn, ta còn đánh giá được chính các xác suất, và còn chọn được ngưỡng quyết định phù hợp với bài toán.

Mục này gồm năm phần:

- Quan sát quá trình học và **bản đồ xác suất** của mô hình trên mặt phẳng đặc trưng.
- Đánh giá trên **tập huấn luyện và tập kiểm tra**, và nhận diện hiện tượng **học thuộc (overfitting)**.
- Các chỉ số quen thuộc (accuracy, ma trận nhầm lẫn, precision, recall, F1) và cách **chọn ngưỡng quyết định**.
- Các công cụ đánh giá **chất lượng xác suất**: log-loss, đường ROC, AUC và biểu đồ hiệu chuẩn.
- So sánh với `LogisticRegression` của **scikit-learn** và cách chọn siêu tham số bằng kiểm định chéo.

Ta tiếp tục dùng quy ước ký hiệu cũ: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn $y_i \in \{0,1\}$. Lớp dương là **"đậu"** ($y = 1$). Toàn bộ mục này dùng bộ dữ liệu mô phỏng 400 sinh viên ở mục 3.7 (cùng cách chia, 280 mẫu huấn luyện và 120 mẫu kiểm tra) và mô hình đã huấn luyện ở mục 4.2:

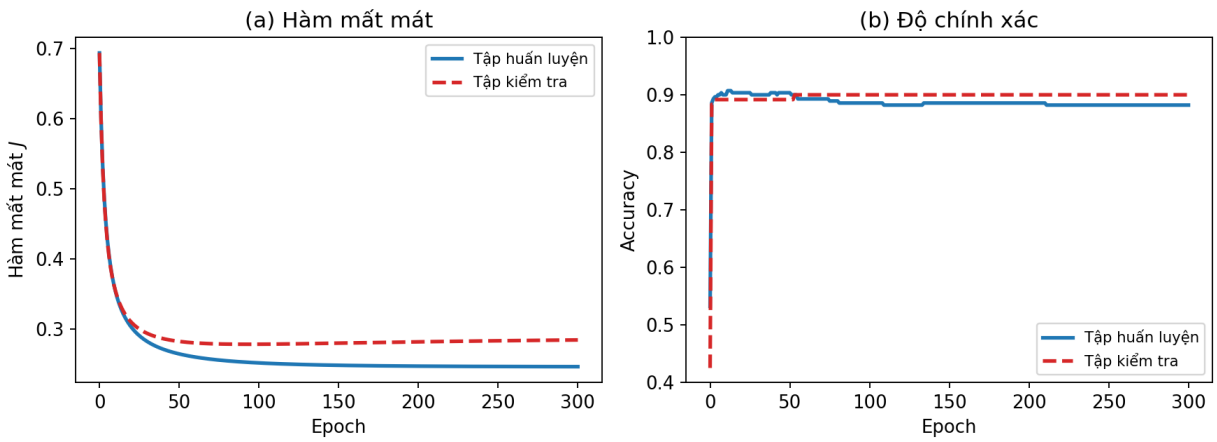
$$\mathbf{w} = \begin{bmatrix} -3{,}520 \\ 1{,}151 \\ -1{,}291 \end{bmatrix}$$

trên đặc trưng gốc (số giờ tự học x_1 và số buổi vắng x_2), hay tương đương $\mathbf{w}' = [0{,}243 \quad 3{,}235 \quad -2{,}092]^T$ trên đặc trưng đã chuẩn hóa.

5.1. Minh họa hàm mất mát giảm dần và các đường đồng mức xác suất

Đường cong học

Cách đơn giản nhất để "nhìn thấy" quá trình học là vẽ hàm mất mát và độ chính xác theo từng epoch, trên **cả hai tập**. Hình 16 vẽ kết quả với Gradient Descent trên đặc trưng chuẩn hóa, $\eta = 0{,}5$ và 300 epoch.



Hình 16. Đường cong học. (a) Hàm mất mát J trên tập huấn luyện và tập kiểm tra giảm dần rồi ổn định. (b) Accuracy tăng vọt trong khoảng 10 *epoch đầu rồi gần như không đổi dù J vẫn còn giảm.*

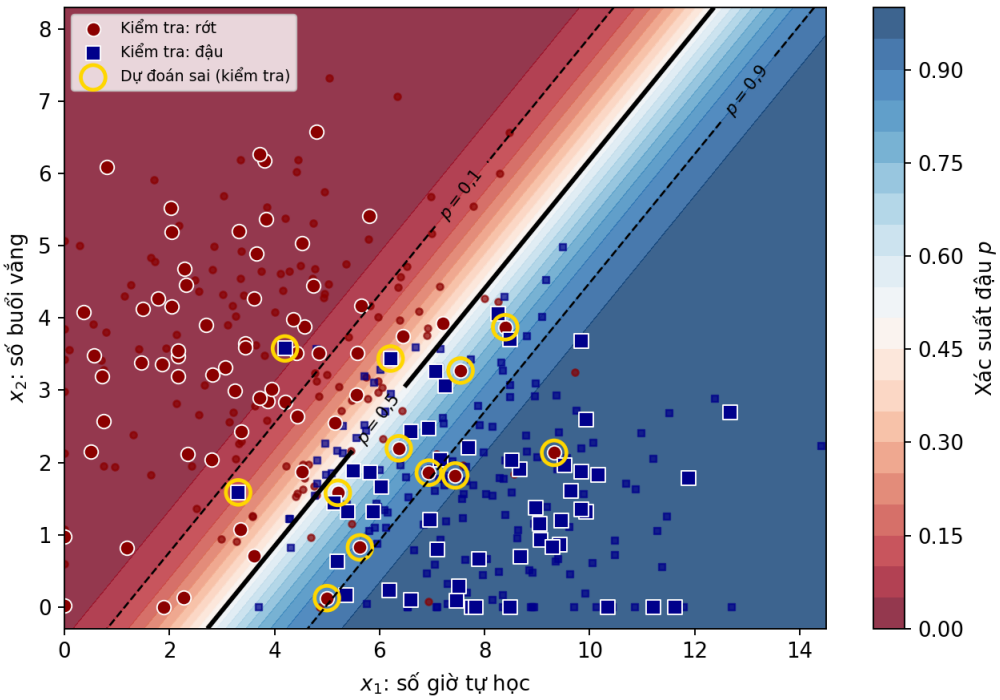
Epoch	J huấn luyện	J kiểm tra	Accuracy huấn luyện	Accuracy kiểm tra
0	0,6931	0,6931	53,2%	42,5%
10	0,3540	0,3548	90,0%	89,2%
50	0,2649	0,2827	90,4%	89,2%
300	0,2468	0,2848	88,2%	90,0%

Có thể rút ra ba nhận xét:

- Ở epoch 0, $\mathbf{w} = \mathbf{0}$ nên $J = \ln 2 \approx 0{,}6931$ trên cả hai tập (đúng như mục 3.2). Accuracy lúc này bằng **tỉ lệ lớp dương**, vì mọi mẫu đều có $p = 0{,}5$ và theo quy ước $\hat{y} = 1$: 53,2% ở tập huấn luyện và 42,5% ở tập kiểm tra.
- J trên tập kiểm tra **cao hơn một chút** so với tập huấn luyện sau khi hội tụ (0,285 so với 0,247). Khoảng chênh nhỏ này là bình thường, vì mô hình luôn "quen" dữ liệu đã học hơn.
- **Accuracy bão hòa rất sớm** (khoảng epoch 10), trong khi J còn tiếp tục giảm thêm nhiều epoch. Điều này cho thấy **hàm mất mát phản ánh chi tiết hơn accuracy**: sau khi nhãn đã đúng, mô hình vẫn đang cải thiện **chất lượng của xác suất** (đẩy p về gần 0 hoặc 1 hơn với những mẫu đúng, và sửa các xác suất chưa hợp lý). Accuracy không "thấy" được những cải thiện này.

Bản đồ xác suất

Hình 17 vẽ xác suất đậu p tại mọi điểm của mặt phẳng đặc trưng (nền màu), cùng đường biên $p = 0{,}5$ (nét liền) và các đường $p = 0{,}1$, $p = 0{,}9$ (nét đứt), như đã mô tả ở mục 2.5. Trên bản đồ có 280 điểm huấn luyện (nhỏ) và 120 điểm kiểm tra (to, có viền trắng), trong đó 12 mẫu kiểm tra bị dự đoán sai được khoanh vàng.



Hình 17. Bản đồ xác suất đậu của mô hình trên mặt phẳng đặc trưng. Nền màu là xác suất p, đường liền là $p = 0{,}5$, hai đường đứt là $p = 0{,}1$ và $p = 0{,}9$. Các điểm to là tập kiểm tra, vòng vàng là các mẫu kiểm tra bị dự đoán sai.

Với $\| \tilde{\mathbf{w}} \| = \sqrt{1{,}151^2 + (-1{,}291)^2} \approx 1{,}729$, khoảng cách từ đường biên đến mỗi đường $p = 0{,}1$ và $p = 0{,}9$ là

$$\frac{\ln 9}{\| \tilde{\mathbf{w}} \|} = \frac{2{,}197}{1{,}729} \approx 1{,}27$$

Như vậy dải "chưa chắc chắn" ($0{,}1 < p < 0{,}9$) rộng khoảng 2,5 đơn vị. Bản đồ cho thấy hai điều đáng chú ý:

- **Phần lớn lỗi nằm trong dải này**: trong 12 mẫu kiểm tra bị sai, có 9 mẫu có xác suất nằm giữa 0,1 và 0,9 (từ 0,147 đến 0,888). Đây là những sai lầm "có cảnh báo": mô hình đã tỏ ra không chắc chắn.
- **Có 3 lỗi "sai chắc chắn"** nằm ngoài dải này, với xác suất đậu 0,035 (thực tế đậu), 0,937 và 0,989 (thực tế rớt). Những mẫu này nằm xa đường biên về phía sai, nhiều khả năng là các trường hợp ngoại lệ hoặc nhãn nhiễu. Chúng rất đắt giá về mất mát: mất mát của ba mẫu là $-\ln 0{,}035 \approx 3{,}34$; $-\ln(1 - 0{,}937) \approx 2{,}76$ và $-\ln(1 - 0{,}989) \approx 4{,}47$, và **riêng ba mẫu này đóng góp khoảng** 31% **tổng mất mát kiểm tra**, dù chỉ chiếm 2,5% số mẫu. Đây chính là điều ta đã thấy ở mục 3.2: cross-entropy phạt rất nặng việc sai một cách chắc chắn.

5.2. Đánh giá trên tập huấn luyện và tập kiểm tra

Nguyên tắc

Nguyên tắc như ở bài Perceptron: **tập kiểm tra bị giấu đi trong suốt quá trình huấn luyện** và chỉ dùng để đánh giá ở cuối. Hàm mất mát và các chỉ số trên tập huấn luyện cho biết mô hình **nhớ** dữ liệu cũ tốt đến đâu, còn trên tập kiểm tra cho biết nó **dự đoán** dữ liệu mới tốt đến đâu. Hai tập có $149/280 = 53{,}2\%$ và $51/120 = 42{,}5\%$ sinh viên đậu. Sự lệch này là do chia ngẫu nhiên. Khi dùng *scikit-learn*, ta thường **chia phân tầng (stratified)** để hai tập có tỉ lệ lớp gần như bằng nhau.

Các kết quả của mô hình trên hai tập:

```
print("J train =", mat_mat(Zb_tr, y_tr, w_z).round(4), "| J test =", mat_mat(
Zb_te, y_te, w_z).round(4))
print("acc train =", np.mean((p_tr >= 0.5) == y_tr).round(4), "| acc test =",
np.mean((p_te >= 0.5) == y_te).round(4))
```

```
J train = 0.2466 | J test = 0.2877
acc train = 0.8821 | acc test = 0.9
```

Chỉ số	Tập huấn luyện ($n = 280$)	Tập kiểm tra ($n = 120$)
Log-loss J	0,2466	0,2877
Accuracy	88,2%	90,0%
AUC (mục 5.4)	0,962	0,953

Accuracy trên tập kiểm tra **cao hơn** tập huấn luyện 1,8 điểm phần trăm, điều này trông ngược với trực giác. Nguyên nhân là **dao động ngẫu nhiên**: với $n = 120$ mẫu, sai số chuẩn của một tỉ lệ cỡ 90% là

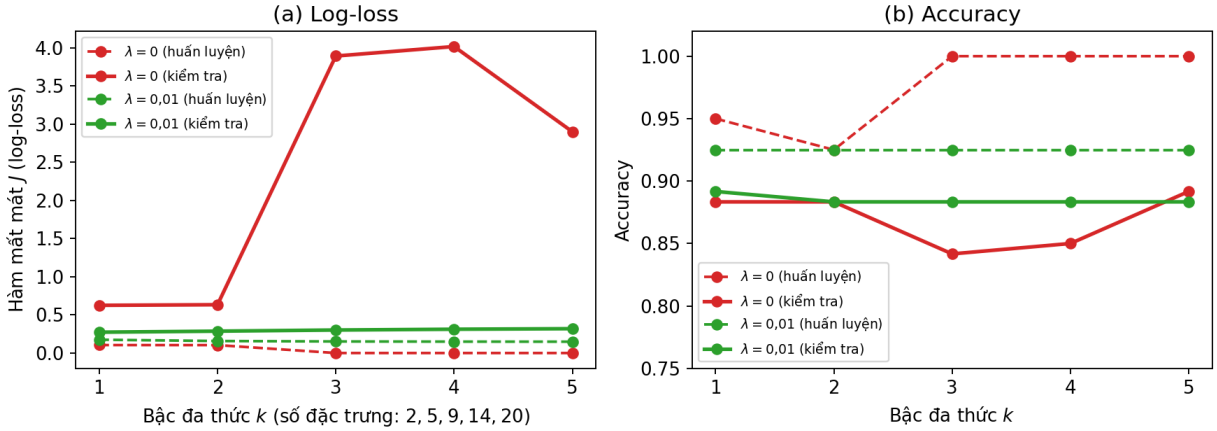
$$\text{SE} \approx \sqrt{\frac{\text{acc}\,(1-\text{acc})}{n}} = \sqrt{\frac{0{,}9 \cdot 0{,}1}{120}} \approx 0{,}027$$

tức khoảng $\pm 2{,}7$ điểm phần trăm. Chênh lệch 1,8 điểm nằm trong khoảng sai số này nên **không có ý nghĩa thống kê**. Hãy luôn nhớ rằng một con số accuracy trên tập kiểm tra nhỏ có "độ nhiễu" đáng kể.

Hiện tượng học thuộc (overfitting)

Ở mục 4.4 và 4.5, ta đã thấy mô hình có nhiều đặc trưng và ít dữ liệu dễ "học thuộc" dữ liệu huấn luyện. Ta kiểm chứng bằng một thí nghiệm. Chỉ lấy 40 **mẫu huấn luyện** (20 đậu và 20 rớt), dựng các đặc trưng đa thức bậc $k = 1, 2, \ldots, 5$ của (x_1, x_2) (tương ứng 2,5,9,14,20 đặc trưng), huấn luyện bằng bộ giải L-BFGS của SciPy với hàm mất mát và gradient đã cài đặt, rồi đánh giá trên 120 mẫu kiểm tra. Ta so hai trường hợp: không điều chuẩn hóa ($\lambda = 0$, giới hạn 300 vòng lặp vì dữ liệu huấn luyện trở nên phân tách được) và có điều chuẩn hóa L2 ($\lambda = 0{,}01$):

Bậc k	λ	J huấn luyện	J kiểm tra	Acc. huấn luyện	Acc. kiểm tra
1	0	0,105	0,625	95,0%	88,3%
1	0,01	0,175	0,273	92,5%	89,2%
3	0	0,000	3,893	100%	84,2%
3	0,01	0,152	0,302	92,5%	88,3%
5	0	0,000	2,901	100%	89,2%
5	0,01	0,149	0,319	92,5%	88,3%



Hình 18. Log-loss (a) và accuracy (b) trên tập huấn luyện (nét đứt) và tập kiểm tra (nét liền) khi tăng bậc đa thức, với 40 mẫu huấn luyện. Không điều chuẩn hóa ($\lambda = 0$, đỏ): log-loss kiểm tra tăng vọt khi mô hình học thuộc. Có điều chuẩn hóa ($\lambda = 0{,}01$, xanh): ổn định ở mọi bậc.

Bảng và Hình 18 cho thấy:

- Khi không điều chuẩn hóa và mô hình đủ phức tạp ($k \geq 3$), **accuracy huấn luyện đạt** 100% và J huấn luyện bằng 0 (mô hình học thuộc), nhưng J kiểm tra **tăng vọt** lên 3,9 và 2,9, gấp khoảng 9 đến 13 lần so với trường hợp có điều chuẩn hóa (0,30 và 0,32). Khoảng cách $J_{\text{kiểm tra}} - J_{\text{huấn luyện}}$ rất lớn là dấu hiệu rõ rệt của overfitting.
- **Log-loss phát hiện overfitting nhạy hơn accuracy.** Với $k = 5$ và $\lambda = 0$, accuracy kiểm tra là 89,2%, **bằng hoặc cao hơn** trường hợp có điều chuẩn hóa (88,3%), nên nhìn riêng accuracy ta sẽ không nhận ra vấn đề. Nhưng log-loss kiểm tra là 2,90 so với 0,32, vạch trần việc mô hình **quá tự tin** ở những mẫu nó dự đoán sai, đúng như hiện tượng **w** lớn và xác suất cực đoan ở mục 4.4.
- Điều chuẩn hóa L2 giữ log-loss kiểm tra ổn định (0,27 đến 0,32) ở **mọi** bậc đa thức, đánh đổi bằng log-loss huấn luyện cao hơn một chút.

Lưu ý. Trong thí nghiệm này, ta dùng tập kiểm tra để so sánh các cấu hình **chỉ để minh họa**. Trong thực tế, các quyết định như chọn bậc đa thức hay chọn λ phải dựa trên một **tập kiểm định (validation set)** riêng hoặc **kiểm định chéo (cross-validation)**, để tập kiểm tra vẫn "chưa bị nhìn thấy" cho đến lần đánh giá cuối cùng. Ta sẽ làm điều đó ở mục 5.5.

5.3. Accuracy, confusion matrix, precision, recall, F1 và ngưỡng quyết định

Ma trận nhầm lẫn

Các khái niệm giống bài Perceptron, nhưng nay với nhãn {0,1}. Lớp dương là "đậu", và ma trận nhầm lẫn **C** có **hàng là nhãn thực**, **cột là nhãn dự đoán**:

$$\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$$

Trong đó FN (bỏ sót) là sinh viên đậu bị dự đoán rớt, còn FP (báo nhầm) là sinh viên rớt bị dự đoán đậu. Hàm cài đặt:

```
def ma_tran_nham_lan(y, y_hat):
    """Ma trận nhầm lẫn C = [[TN, FP], [FN, TP]] (hàng: nhãn thực, cột: nhãn
dự đoán)."""
    tp = np.sum((y == 1) & (y_hat == 1)); tn = np.sum((y == 0) & (y_hat == 0)
)
    fp = np.sum((y == 0) & (y_hat == 1)); fn = np.sum((y == 1) & (y_hat == 0)
)
    return np.array([[tn, fp], [fn, tp]])

C = ma_tran_nham_lan(y_te, (p_te >= 0.5).astype(int))
print(C)
tn, fp, fn, tp = C.ravel()
print("precision =", round(tp / (tp + fp), 3), "| recall =", round(tp / (tp +
fn), 3),
      "| do dac hieu =", round(tn / (tn + fp), 3), "| F1 =", round(2 * tp / (
2 * tp + fp + fn), 3))
```

```
[[60  9]
 [ 3 48]]
precision = 0.842 | recall = 0.941 | do dac hieu = 0.87 | F1 = 0.889
```

Trên tập kiểm tra, với ngưỡng 0,5:

$$\mathbf{C}_{\text{kiểm tra}} = \begin{bmatrix} 60 & 9 \\ 3 & 48 \end{bmatrix}, \qquad \mathbf{C}_{\text{huấn luyện}} = \begin{bmatrix} 118 & 13 \\ 20 & 129 \end{bmatrix}$$

Các chỉ số tính từ $\mathbf{C}_{\text{kiểm tra}}$:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{n} = \frac{48 + 60}{120} = 0{,}900, \qquad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{48}{57} \approx 0{,}842$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{48}{51} \approx 0{,}941, \qquad \text{F1} = \frac{2\,\text{TP}}{2\,\text{TP} + \text{FP} + \text{FN}} = \frac{96}{108} \approx 0{,}889$$

Ngoài ra ta dùng thêm **độ đặc hiệu (specificity)**, là recall của **lớp âm**: trong số sinh viên thật sự rớt, mô hình nhận ra đúng bao nhiêu:

$$\text{Độ đặc hiệu} = \frac{\text{TN}}{\text{TN} + \text{FP}} = \frac{60}{69} \approx 0{,}870$$

So với Perceptron túi trên **cùng tập kiểm tra** (mục 5.2 bài Perceptron, $\mathbf{C} = \begin{bmatrix} 59 & 10 \\ 3 & 48 \end{bmatrix}$), Logistic Regression sai ít hơn một mẫu (một báo nhầm). Sự khác biệt rất nhỏ, vì cả hai đều là bộ phân loại tuyến tính trên cùng dữ liệu. Khác biệt thực sự nằm ở **thứ mà Logistic Regression cho thêm**: xác suất, và vì vậy cả khả năng đổi ngưỡng.

Đổi ngưỡng quyết định

Perceptron có ngưỡng cố định ở điểm số 0. Với Logistic Regression, ta dự đoán "đậu" khi $p \geq \tau$ với τ tùy chọn (mục 2.3). Bảng sau cho các chỉ số trên tập kiểm tra khi τ thay đổi:

τ	TP	FP	FN	TN	Accuracy	Precision	Recall	Độ đặc hiệu	F1
0,1	50	20	1	49	0,825	0,714	0,980	0,710	0,826
0,3	49	12	2	57	0,883	0,803	0,961	0,826	0,875
0,5	48	9	3	60	0,900	0,842	0,941	0,870	0,889
0,7	43	8	8	61	0,867	0,843	0,843	0,884	0,843
0,75	40	6	11	63	0,858	0,870	0,784	0,913	0,825
0,8	38	5	13	64	0,850	0,884	0,745	0,928	0,809
0,9	34	2	17	67	0,842	0,944	0,667	0,971	0,782

Có một **sự đánh đổi** rất rõ (Hình 19a): khi τ **tăng**, mô hình dự đoán "đậu" khó hơn, nên **precision và độ đặc hiệu tăng** (ít báo nhầm hơn) nhưng **recall giảm** (bỏ sót nhiều hơn). Khi τ **giảm** thì ngược lại. Accuracy đạt cao nhất gần $\tau = 0{,}5$, nhưng điều đó chỉ đúng khi hai loại sai có tầm quan trọng như nhau.

Chọn ngưỡng theo hậu quả của từng loại sai

Giả sử hai loại sai có **chi phí** khác nhau: c_{FP} cho một báo nhầm và c_{FN} cho một bỏ sót. Với một mẫu có xác suất đậu thực sự là p (giả sử mô hình ước lượng đúng), chi phí kỳ vọng của hai quyết định là:

$$\mathbb{E}[\text{chi phí} \mid \hat{y} = 1] = (1 - p)\, c_{\text{FP}}, \qquad \mathbb{E}[\text{chi phí} \mid \hat{y} = 0] = p\, c_{\text{FN}}$$

Ta nên dự đoán $\hat{y} = 1$ khi chi phí kỳ vọng nhỏ hơn, tức khi $(1 - p)\, c_{\text{FP}} \leq p\, c_{\text{FN}}$. Giải ra:

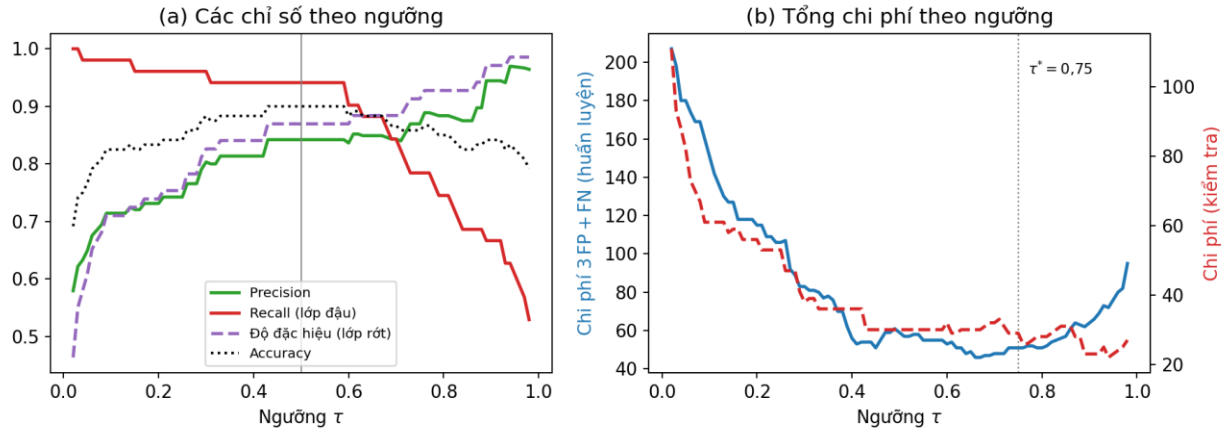
$$\hat{y} = 1 \Leftrightarrow p \geq \tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}}$$

Nếu hai chi phí bằng nhau thì $\tau^* = 0{,}5$. Nếu báo nhầm đắt hơn thì ngưỡng cao hơn, và ngược lại.

Ví dụ. Nhà trường muốn phát hiện sinh viên có nguy cơ rớt. Nếu **không phát hiện được** một sinh viên sắp rớt (dự đoán đậu mà thực tế rớt, tức FP) tốn kém gấp 3 lần một cảnh báo thừa (dự đoán rớt mà thực tế đậu, tức FN), ta có $c_{\text{FP}} = 3$ và $c_{\text{FN}} = 1$, nên

$$\tau^* = \frac{3}{3+1} = 0{,}75$$

Chỉ nên kết luận "đậu" khi xác suất đậu từ 75% trở lên. Hình 19b vẽ tổng chi phí 3 FP + FN theo τ.



Hình 19. (a) Precision, recall, độ đặc hiệu và accuracy trên tập kiểm tra theo ngưỡng τ (đường dọc là $\tau = 0{,}5$). (b) Tổng chi phí $3\,FP + FN$ trên tập huấn luyện (xanh) và tập kiểm tra (đỏ) theo τ; đường dọc là ngưỡng lý thuyết $\tau^* = 0{,}75$.

Cột cuối của bảng thực nghiệm cho chi phí 3 FP + FN trên tập kiểm tra: 61; 38; 30; 32; 29; 28; 23 tương ứng với $\tau = 0{,}1; 0{,}3; 0{,}5; 0{,}7; 0{,}75; 0{,}8; 0{,}9$. Cần đọc kết quả này một cách thận trọng:

- Tăng τ từ 0,1 lên 0,5 **giảm mạnh** chi phí ($61 \to 30$), xác nhận ý tưởng rằng khi báo nhầm đắt hơn thì nên nâng ngưỡng.
- Từ $\tau = 0{,}5$ đến $\tau = 0{,}8$, chi phí chỉ dao động từ 28 đến 32, **khác biệt không đáng kể** trên 120 mẫu. Trên tập huấn luyện, chi phí nhỏ nhất đạt ở $\tau \approx 0{,}66$, gần giá trị lý thuyết 0,75.
- Trên tập kiểm tra, chi phí còn giảm tiếp xuống 23 ở $\tau = 0{,}9$, tức thấp hơn lý thuyết dự đoán. Một nguyên nhân khả dĩ là xác suất của mô hình hơi **quá tự tin ở vùng cao** (xem biểu đồ hiệu chuẩn ở mục 5.4), và công thức τ^* chỉ đúng khi xác suất được hiệu chuẩn tốt.
- Việc quan sát chi phí trên tập kiểm tra rồi chọn τ theo đó là **vi phạm quy tắc vàng**. Trong thực tế ta chọn τ trên tập kiểm định, hoặc dựa vào τ^* lý thuyết khi có ước lượng chi phí đáng tin cậy.

5.4. Đánh giá chất lượng xác suất: log-loss, đường ROC và AUC

Các chỉ số ở mục 5.3 đều phụ thuộc vào **một ngưỡng cụ thể**. Nhưng Logistic Regression cho ra xác suất, và ta muốn đánh giá cả **bản thân xác suất** lẫn **khả năng xếp hạng** của chúng, độc lập với việc chọn ngưỡng.

Log-loss

Chỉ số đầu tiên là **chính hàm mất mát** trên tập kiểm tra, còn gọi là **log-loss**:

$$\text{Log-loss} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$

Trên tập kiểm tra, log-loss của mô hình là $0{,}2877$. Để biết con số này tốt hay không, ta so với hai mốc tham chiếu:

- Mô hình "không biết gì" luôn trả $p = 0{,}5$ có log-loss $\ln 2 \approx 0{,}693$.
- **Mô hình hằng** luôn trả tỉ lệ lớp dương của tập huấn luyện ($p = 0{,}532$) cho mọi sinh viên có log-loss kiểm tra là $0{,}705$.

Mô hình Logistic Regression giảm log-loss từ khoảng $0{,}7$ xuống $0{,}288$. Có một cách diễn giải trực quan: vì $e^{-\text{log-loss}} = e^{-0{,}2877} \approx 0{,}750$, **trung bình nhân của xác suất mô hình gán cho nhãn thật là khoảng** $0{,}75$, so với $0{,}5$ nếu chỉ đoán mò.

Một thước đo xác suất khác, đơn giản hơn, là **điểm Brier**: trung bình bình phương sai số giữa xác suất và nhãn:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2$$

Trên tập kiểm tra, điểm Brier của mô hình là $0{,}0868$ so với $0{,}2559$ của mô hình hằng (càng nhỏ càng tốt).

Đường ROC và AUC

Khi đổi ngưỡng τ, mỗi giá trị cho một cặp (**tỉ lệ dương giả FPR**, **tỉ lệ dương thật TPR**):

$$\text{TPR} = \frac{\text{TP}}{\text{TP}+\text{FN}}\ (=\text{recall}), \qquad \text{FPR} = \frac{\text{FP}}{\text{FP}+\text{TN}}\ (=1-\text{độ đặc hiệu})$$

Đường ROC là đường nối các điểm (FPR, TPR) khi cho τ chạy từ 1 xuống 0. Ví dụ ba điểm trên đường ROC của mô hình:

Ngưỡng τ	FPR	TPR
$0{,}3$	$12/69 \approx 0{,}174$	$49/51 \approx 0{,}961$
$0{,}5$	$9/69 \approx 0{,}130$	$48/51 \approx 0{,}941$
$0{,}7$	$8/69 \approx 0{,}116$	$43/51 \approx 0{,}843$

Một mô hình lý tưởng đi qua góc trên bên trái $(0{,}1)$ (bắt hết lớp dương mà không báo nhầm nào). Mô hình đoán ngẫu nhiên nằm trên đường chéo. **AUC** (area under the curve) là diện tích dưới đường ROC, nhận giá trị từ $0{,}5$ (đoán ngẫu nhiên) đến 1 (hoàn hảo). Trên tập kiểm tra, AUC của mô hình là $0{,}953$ (Hình 20b).

AUC có một diễn giải xác suất rất dễ hiểu: **đó là xác suất để một sinh viên đậu được chọn ngẫu nhiên có p cao hơn một sinh viên rớt được chọn ngẫu nhiên**. Công thức đếm cặp:

$$\text{AUC} = \frac{1}{n_+ \, n_-} \sum_{i:\, y_i = 1} \; \sum_{j:\, y_j = 0} \left(\mathbb{1}[p_i > p_j] + 1/2 \; \mathbb{1}[p_i = p_j] \right)$$

trong đó n_+ và n_- là số mẫu thuộc lớp dương và lớp âm. Ta kiểm chứng bằng cách tự đếm trên $51 \times 69 = 3519$ cặp của tập kiểm tra:

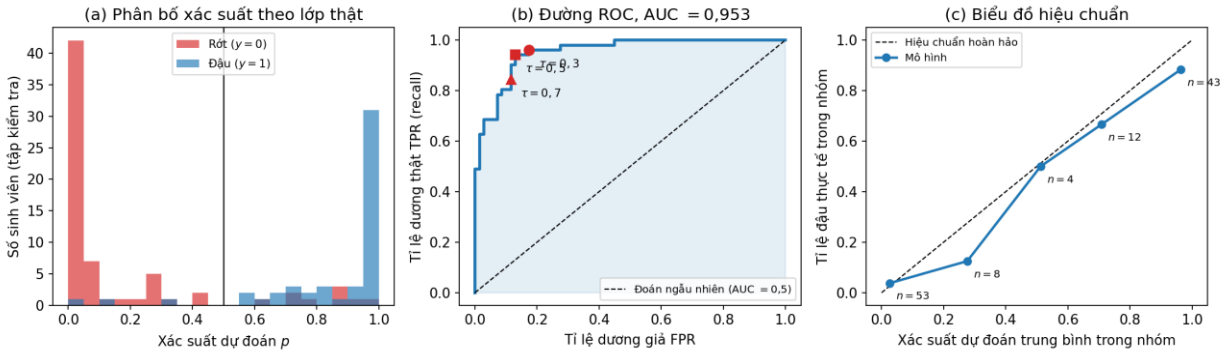
```
pos, neg = p_te[y_te == 1], p_te[y_te == 0]
auc_cap = np.mean([(a > b) + 0.5 * (a == b) for a in pos for b in neg])
print("so cap =", len(pos) * len(neg), "| AUC (dem cap) =", round(auc_cap, 4)
,
      "| AUC (sklearn) =", round(roc_auc_score(y_te, p_te), 4))
```

```
so cap = 3519 | AUC (dem cap) = 0.9534 | AUC (sklearn) = 0.9534
```

Vậy khi chọn ngẫu nhiên một sinh viên đậu và một sinh viên rớt, mô hình gán xác suất đậu cao hơn cho đúng người đậu trong khoảng 95,3% số trường hợp. **AUC chỉ phụ thuộc vào thứ tự (xếp hạng) của các xác suất**, không phụ thuộc vào ngưỡng, nên có thể dùng để so sánh các mô hình mà không cần chọn ngưỡng trước. Nó cũng có thể áp dụng cho điểm số của Perceptron, nhưng Perceptron không có xác suất để đánh giá log-loss hay độ hiệu chuẩn.

Phân bố xác suất và biểu đồ hiệu chuẩn

Hình 20a vẽ phân bố xác suất dự đoán theo lớp thật: một mô hình tốt cho xác suất của sinh viên rớt dồn về gần 0 và của sinh viên đậu dồn về gần 1, ít mẫu ở giữa. Ta thấy hai "đỉnh" khá tách biệt, đúng như vậy. Các mẫu nằm lẫn ở phía sai là những mẫu bị dự đoán sai.



Hình 20. (a) Phân bố xác suất dự đoán theo lớp thật trên tập kiểm tra. (b) Đường ROC với AUC $= 0{,}953$*; ba điểm đánh dấu là các ngưỡng* $0{,}3$*;* $0{,}5$ *và* $0{,}7$*. (c) Biểu đồ hiệu chuẩn: xác suất dự đoán trung bình so với tỉ lệ đậu thực tế trong từng nhóm.*

Một câu hỏi khác là: **xác suất có đúng nghĩa là xác suất không?** Nếu mô hình nói $p = 0{,}8$ cho 100 sinh viên, ta mong khoảng 80 người trong số đó thực sự đậu. Mô hình thỏa điều này được gọi là **hiệu chuẩn tốt (well-calibrated)**. Cách kiểm tra là chia các mẫu theo xác suất dự đoán thành vài nhóm và so **xác suất dự đoán trung bình** với **tỉ lệ đậu thực tế** của từng nhóm (Hình 20c):

Nhóm theo p	Số mẫu	p dự đoán trung bình	Tỉ lệ đậu thực tế
$[0;\ 0{,}2)$	53	0,026	0,038
$[0{,}2;\ 0{,}4)$	8	0,276	0,125
$[0{,}4;\ 0{,}6)$	4	0,512	0,500
$[0{,}6;\ 0{,}8)$	12	0,709	0,667
$[0{,}8;\ 1]$	43	0,964	0,884

Các điểm nằm khá gần đường chéo (hiệu chuẩn hoàn hảo), nên mô hình **hiệu chuẩn tương đối tốt**, với một độ lệch nhỏ ở nhóm cao nhất: mô hình dự đoán trung bình 0,964 nhưng thực tế chỉ 0,884 sinh viên đậu, tức là hơi **quá tự tin**. Cần lưu ý các nhóm giữa chỉ có vài mẫu (4, 8, 12) nên các con số của chúng rất nhiễu. Hiện tượng quá tự tin là điều ta đã dự báo ở mục 4.4 khi **w** lớn, và có thể giảm bằng điều chuẩn hóa L2.

Khi nào dùng chỉ số nào?

Câu hỏi	Chỉ số phù hợp
Tỉ lệ dự đoán nhãn đúng (hai loại sai quan trọng như nhau, lớp cân bằng)	Accuracy
Hai loại sai có hậu quả khác nhau, hoặc lớp lệch	Ma trận nhầm lẫn, precision, recall, F1
Mô hình xếp hạng tốt đến đâu, không phụ thuộc ngưỡng	ROC, AUC
Chất lượng của chính xác suất	Log-loss, điểm Brier, biểu đồ hiệu chuẩn
Cạnh tranh giữa các cấu hình trong lúc huấn luyện	Log-loss (nhạy hơn accuracy, xem mục 5.2)

5.5. Huấn luyện và so sánh với Logistic Regression trong scikit-learn

Trong thực tế ta thường dùng lớp *LogisticRegression* của *scikit-learn*. Việc cài đặt từ đầu ở mục 3.7 giúp ta hiểu rõ thư viện làm gì bên trong. Ta huấn luyện trên **đặc trưng gốc** với C rất lớn (gần như không điều chuẩn hóa) để so sánh công bằng với mô hình của ta:

```
from sklearn.linear_model import LogisticRegression

clf = LogisticRegression(C=1e10, max_iter=10000).fit(X_tr, y_tr)
print("intercept_ =", clf.intercept_, "| coef_ =", clf.coef_, "| classes_ =",
clf.classes_)
print("tu cai dat (thang goc): w =", w_goc)
p_sk = clf.predict_proba(X_te)[:, 1]
p_ours = sigmoid(them_bias(X_te) @ w_goc)
print("max |p_sk - p_ours| =", np.abs(p_sk - p_ours).max().round(4))
```

```
intercept_ = [-3.5251] | coef_ = [[ 1.1519 -1.2917]] | classes_ = [0 1]
tu cai dat (thang goc): w = [-3.52    1.1508 -1.2909]
max |p_sk - p_ours| = 0.0005
```

Các tham số và thuộc tính cần chú ý:

- ***C***: nghịch đảo độ mạnh của điều chuẩn hóa L2. Như đã nói ở mục 4.4, hàm mục tiêu của thư viện là $C\sum_i \ell_i + \frac{1}{2}\|\,\tilde{\mathbf{w}}\,\|^2$, tương đương $\lambda = \frac{1}{C\,n}$. C càng **nhỏ** thì điều chuẩn hóa càng **mạnh**. Giá trị mặc định là $C = 1$.
- ***intercept_* và *coef_***: thư viện lưu hệ số chệch w_0 **riêng** trong *intercept_*, còn $w_1, \ldots, w_d$ trong *coef_* (không cần thêm cột 1 vào **X**).
- ***predict_proba*** trả ma trận $n \times 2$ gồm $P(y=0)$ và $P(y=1)$ (thứ tự theo *classes_*), ***decision_function*** trả điểm số $s = \mathbf{xw}$, và ***predict*** trả nhãn bằng cách lấy lớp có $s > 0$ (khác quy ước "$s \geq 0$ thì $\hat{y} = 1$" của ta ở đúng điểm $s = 0$, rất hiếm gặp).
- *max_iter* và *solver* (mặc định *lbfgs*, một phương pháp dùng thông tin độ cong như đã nhắc ở mục 4.3).

Hệ số của thư viện $(-3{,}5251;\ 1{,}1519;\ -1{,}2917)$ **gần như trùng** với mô hình tự cài đặt $(-3{,}520;\ 1{,}1508;\ -1{,}2909)$, và xác suất dự đoán lệch nhau tối đa chỉ $0{,}0005$. Khác biệt rất nhỏ này là do mô hình của ta dừng sau 300 epoch (rất gần nhưng chưa chính xác tại điểm tối ưu), còn bộ giải của thư viện chạy đến khi đạt dung sai chặt hơn.

So sánh các mô hình

Ta so sánh trên cùng dữ liệu:

```
from sklearn.metrics import accuracy_score, log_loss, roc_auc_score, confusio
n_matrix, classification_report

clf_mac_dinh = LogisticRegression().fit(X_tr, y_tr)            # C = 1 (mặc
định)
for ten, m in [("sklearn C=1e10", clf), ("sklearn mac dinh (C=1)", clf_mac_di
nh)]:
    p = m.predict_proba(X_te)[:, 1]
    print(f"{ten:24s} train {m.score(X_tr, y_tr):.4f} | test {m.score(X_te, y
_te):.4f} | "
          f"log-loss {log_loss(y_te, p):.4f} | AUC {roc_auc_score(y_te, p):.4
f}")
```

```
tu cai dat               train 0.8821 | test 0.9000 | log-loss 0.2877 | AUC 0
.9534
sklearn C=1e10           train 0.8821 | test 0.9000 | log-loss 0.2878 | AUC 0
.9534
sklearn mac dinh (C=1)   train 0.8857 | test 0.9000 | log-loss 0.2842 | AUC 0
.9531
```

Cùng với các mô hình ở bài Perceptron:

Mô hình	Accuracy huấn luyện	Accuracy kiểm tra	Log-loss kiểm tra	AUC kiểm tra
Logistic Regression, tự cài đặt	88,2%	90,0%	0,2877	0,9534

Mô hình	Accuracy huấn luyện	Accuracy kiểm tra	Log-loss kiểm tra	AUC kiểm tra
Logistic Regression, *scikit-learn* ($C = 10^{10}$)	88,2%	90,0%	0,2878	0,9534
Logistic Regression, *scikit-learn* mặc định ($C = 1$)	88,6%	90,0%	0,2842	0,9531
Perceptron, *scikit-learn* mặc định	—	89,2%	không có	0,9523
Hồi quy tuyến tính + ngưỡng 0,5	—	89,2%	không có	0,9531

Với mô hình mặc định ($C = 1$), các hệ số là intercept_ $= -3{,}402$ và coef_ $=$ $[1{,}1016 \quad -1{,}2216]$, nhỏ hơn một chút so với bản không điều chuẩn hóa: đó là tác động của điều chuẩn hóa L2, kéo $\mathbf{w}$ về gần $\mathbf{0}$.

Có hai nhận xét quan trọng:

- Trên bộ dữ liệu này, **mọi mô hình tuyến tính đều cho kết quả gần như bằng nhau** về accuracy và AUC (đều khoảng 89% đến 90% và 0,95). Điều này hợp lý: dữ liệu khá cân bằng, ít mẫu ngoại lai, và hai lớp chồng lấn nên không mô hình tuyến tính nào vượt qua giới hạn đó. Lợi thế của Logistic Regression **không nằm ở accuracy** trên dữ liệu "dễ" như vậy mà nằm ở những gì nó cho thêm: **xác suất** (nên có log-loss, hiệu chuẩn, đổi ngưỡng), tính **ổn định** (hội tụ êm, ít bị ảnh hưởng bởi mẫu ngoại lai như đã thấy ở mục 6 bài Perceptron) và nền tảng **mở rộng** sang mạng nơ-ron.
- Hồi quy tuyến tính và Perceptron **không có xác suất** nên không có log-loss và không đánh giá được độ hiệu chuẩn, dù AUC của điểm số vẫn tính được.

Chọn siêu tham số C bằng kiểm định chéo

Tham số C (độ mạnh điều chuẩn hóa) là một **siêu tham số**: không học được từ gradient mà phải chọn. Ta không được chọn nó theo tập kiểm tra. Cách đúng là **kiểm định chéo K lần** trên **tập huấn luyện**: chia tập huấn luyện thành K phần, lần lượt dùng mỗi phần làm tập kiểm định và $K - 1$ phần còn lại để huấn luyện, rồi lấy trung bình điểm số:

$$\text{CV} = \frac{1}{K}\sum_{k=1}^{K} \text{điểm số}_k$$

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
for C in (0.001, 0.01, 0.1, 1, 10, 100, 1000):
    mo_hinh = make_pipeline(StandardScaler(), LogisticRegression(C=C))   # ch
uẩn hóa nằm trong pipeline
    acc = cross_val_score(mo_hinh, X_tr, y_tr, cv=cv)
    ll = -cross_val_score(mo_hinh, X_tr, y_tr, cv=cv, scoring="neg_log_loss")
```

```
    print(f"C = {C:<6} CV accuracy = {acc.mean():.4f} +- {acc.std():.4f} | CV
log-loss = {ll.mean():.4f}")
```

```
C = 0.001  CV accuracy = 0.6893 +- 0.0582 | CV log-loss = 0.6499
C = 0.01   CV accuracy = 0.8893 +- 0.0175 | CV log-loss = 0.4802
C = 0.1    CV accuracy = 0.9000 +- 0.0182 | CV log-loss = 0.3059
C = 1      CV accuracy = 0.8857 +- 0.0087 | CV log-loss = 0.2556
C = 10     CV accuracy = 0.8821 +- 0.0087 | CV log-loss = 0.2528
C = 100    CV accuracy = 0.8821 +- 0.0087 | CV log-loss = 0.2530
C = 1000   CV accuracy = 0.8821 +- 0.0087 | CV log-loss = 0.2531
```

Việc đặt *StandardScaler* **bên trong** pipeline là quan trọng: nó đảm bảo trong mỗi lần kiểm định chéo, trung bình và độ lệch chuẩn chỉ được tính từ phần huấn luyện của lần đó, tránh rò rỉ thông tin (mục 4.2). Kết quả cho thấy:

- C **quá nhỏ** (0,001) làm điều chuẩn hóa quá mạnh, mô hình bị "bó" (underfit): accuracy chỉ 68,9% và log-loss 0,650 gần mức đoán mò.
- Từ $C \approx 1$ trở lên, kết quả gần như không đổi, vì với chỉ 2 đặc trưng và 280 mẫu, mô hình không có nguy cơ overfit.
- **Log-loss kiểm định chéo nhỏ nhất ở** $C = 10$ **(0,2528)**, nên đây là lựa chọn của ta. Log-loss phân biệt các giá trị C rõ hơn accuracy: accuracy dao động $\pm 1{,}8$ điểm quanh 0,88 đến 0,90 nên không giúp chọn được.

Sau khi chọn $C = 10$ bằng kiểm định chéo, **chỉ lúc này** ta mới huấn luyện lại trên toàn bộ tập huấn luyện và đánh giá **một lần** trên tập kiểm tra:

```
C = 10 tren tap kiem tra: accuracy = 0.9 | log-loss = 0.286
```

Báo cáo đầy đủ bằng các hàm có sẵn

Các chỉ số ở mục 5.3 và 5.4 đều có hàm dựng sẵn. Ta kiểm tra rằng kết quả trùng với phép tính tay:

```
print(confusion_matrix(y_te, clf.predict(X_te), labels=[0, 1]))
print(classification_report(y_te, (p_ours >= 0.5).astype(int), labels=[0, 1],
                            target_names=["rot", "dau"], digits=3))
```

```
[[60  9]
 [ 3 48]]
              precision    recall  f1-score   support

         rot      0.952     0.870     0.909        69
         dau      0.842     0.941     0.889        51

    accuracy                          0.900       120
   macro avg      0.897     0.905     0.899       120
weighted avg      0.906     0.900     0.901       120
```

Dòng "dau" cho precision 0,842, recall 0,941 và F1 0,889, **đúng bằng** các giá trị tính tay. Dòng "rot" là các chỉ số khi coi lớp rớt là lớp dương: precision $\frac{60}{60+3} \approx 0{,}952$ và recall $\frac{60}{60+9} \approx 0{,}870$

(chính là độ đặc hiệu của lớp đậu). Hai dòng cuối là **trung bình**: *macro avg* là trung bình đơn giản của hai lớp, còn *weighted avg* có trọng số theo số mẫu (*support*) của từng lớp.

> **Lời khuyên thực hành.** Khi dùng Logistic Regression trên dữ liệu thật: (1) **chuẩn hóa đặc trưng** bên trong pipeline; (2) chọn siêu tham số như C bằng **kiểm định chéo** trên tập huấn luyện; (3) báo cáo kết quả **một lần** trên tập kiểm tra, gồm cả ma trận nhầm lẫn, log-loss và AUC chứ không chỉ accuracy; (4) nếu hậu quả của hai loại sai khác nhau, **chọn ngưỡng** theo $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}}$ trên tập kiểm định.

Tóm tắt mục 5

- **Đường cong học** cho thấy J giảm dần trên cả hai tập. Accuracy bão hòa sớm hơn rất nhiều so với J, nên log-loss phản ánh sự cải thiện về chất lượng xác suất mà accuracy không thấy. **Bản đồ xác suất** cho thấy phần lớn lỗi nằm trong dải $0{,}1 < p < 0{,}9$, còn vài lỗi "sai chắc chắn" đóng góp một phần rất lớn vào mất mát.
- Đánh giá trên **tập kiểm tra** (giấu trong lúc huấn luyện). Hiện tượng **overfitting** thể hiện ở khoảng cách lớn giữa J kiểm tra và J huấn luyện, và **log-loss nhạy với overfitting hơn accuracy**. **Điều chuẩn hóa L2** giữ log-loss kiểm tra ổn định.
- Từ ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$, ta tính accuracy, precision, recall, độ đặc hiệu và F1. Vì có xác suất, ta có thể **đổi ngưỡng** τ: τ tăng thì precision tăng và recall giảm. Với chi phí c_{FP}, c_{FN}, ngưỡng tối ưu là $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}}$ (khi xác suất hiệu chuẩn tốt).
- **Chất lượng xác suất** được đánh giá bằng **log-loss** (và điểm Brier), **đường ROC và AUC** (khả năng xếp hạng, không phụ thuộc ngưỡng, bằng xác suất một mẫu dương được xếp cao hơn một mẫu âm) và **biểu đồ hiệu chuẩn** (xác suất dự đoán có đúng nghĩa là xác suất không).
- *LogisticRegression* của *scikit-learn* thực hiện cùng mô hình: với C rất lớn, kết quả **trùng khớp** với mã tự cài đặt. Siêu tham số C ($\lambda = 1/(Cn)$) được chọn bằng **kiểm định chéo** trên tập huấn luyện, và tập kiểm tra chỉ dùng **một lần** ở cuối. Trên dữ liệu dễ, mọi mô hình tuyến tính cho accuracy gần nhau, lợi thế của Logistic Regression là xác suất, ổn định và khả năng mở rộng.

Câu hỏi kiểm tra nhanh

24. Vì sao ở epoch 0 (khi $\mathbf{w} = \mathbf{0}$), accuracy trên tập huấn luyện bằng đúng tỉ lệ lớp dương $0{,}532$?
25. Cho ma trận nhầm lẫn $\mathbf{C} = \begin{bmatrix} 70 & 10 \\ 5 & 15 \end{bmatrix}$ (hàng: nhãn thực 0,1; cột: nhãn dự đoán 0,1). Tính accuracy, precision, recall, độ đặc hiệu và F1 cho lớp dương.
26. Một mô hình trên tập kiểm tra có accuracy 91% nhưng log-loss 2,5. Điều này gợi ý hiện tượng gì? Vì sao accuracy không phát hiện được?
27. Nếu bỏ sót một bệnh nhân (FN) tốn kém gấp 9 lần một cảnh báo thừa (FP), ngưỡng tối ưu τ^* để dự đoán "bệnh" là bao nhiêu? Ngưỡng cao hay thấp hơn $0{,}5$, và vì sao?
28. Trong tập kiểm tra có 10 mẫu dương và 20 mẫu âm. Một mô hình xếp hạng đúng 190 trong 200 cặp (dương, âm), và hòa 0 cặp. Tính AUC. Diễn giải kết quả.

29. Vì sao AUC không phụ thuộc vào ngưỡng quyết định? AUC có phản ánh được độ hiệu chuẩn của xác suất không?
30. Trong kiểm định chéo, vì sao phải đặt bước chuẩn hóa (`StandardScaler`) bên trong pipeline, thay vì chuẩn hóa toàn bộ dữ liệu trước rồi mới chia?
31. Vì sao khi so sánh các giá trị C, ta dựa vào kiểm định chéo trên tập huấn luyện chứ không dựa vào tập kiểm tra?

Gợi ý đáp án

32. Khi $\mathbf{w} = \mathbf{0}$, mọi $p_i = 0{,}5$. Theo quy ước $p \geq 0{,}5 \Rightarrow \hat{y} = 1$ nên mô hình dự đoán $\hat{y} = 1$ cho **mọi** mẫu. Khi đó chỉ các mẫu thật sự thuộc lớp dương được đoán đúng, nên accuracy bằng tỉ lệ lớp dương ($149/280 = 0{,}532$).
33. TN $= 70$, FP $= 10$, FN $= 5$, TP $= 15$, $n = 100$. Accuracy $= \frac{15+70}{100} = 0{,}85$. Precision $= \frac{15}{15+10} = 0{,}60$. Recall $= \frac{15}{15+5} = 0{,}75$. Độ đặc hiệu $= \frac{70}{70+10} = 0{,}875$. F1 $= \frac{2 \cdot 15}{2 \cdot 15+10+5} = \frac{30}{45} \approx 0{,}667$.
34. Gợi ý **mô hình quá tự tin** (overfitting): nó đoán nhãn đúng ở hầu hết mẫu (accuracy cao) nhưng ở những mẫu sai thì gán xác suất cực đoan cho nhãn sai, và cross-entropy phạt rất nặng những trường hợp đó (ví dụ $-\ln 0{,}01 \approx 4{,}6$ cho một mẫu). Accuracy chỉ đếm đúng hay sai nên không phân biệt "sai nhẹ" và "sai chắc chắn".
35. Lấy lớp dương là "bệnh". Khi đó $c_{\text{FN}} = 9$ (bỏ sót một bệnh nhân) và $c_{\text{FP}} = 1$ (báo nhầm một người khỏe mạnh). Ngưỡng $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}} = \frac{1}{1+9} = 0{,}1$. Ngưỡng **thấp hơn nhiều** so với $0{,}5$: vì bỏ sót nguy hiểm hơn nhiều, ta sẵn sàng dự đoán "bệnh" ngay cả khi xác suất bệnh chỉ từ 10%.
36. $n_+ n_- = 10 \cdot 20 = 200$ cặp, xếp đúng 190 cặp nên AUC $= \frac{190}{200} = 0{,}95$. Xác suất để một mẫu dương ngẫu nhiên được xếp cao hơn một mẫu âm ngẫu nhiên là 95%.
37. AUC được tính từ **thứ tự** của các xác suất (so sánh từng cặp dương, âm) nên không cần chọn ngưỡng, và nó tổng hợp hiệu năng trên **mọi** ngưỡng. AUC **không** phản ánh độ hiệu chuẩn: nếu ta thay mọi xác suất p bằng p^2 (giữ nguyên thứ tự) thì AUC không đổi, dù xác suất đã bị méo.
38. Nếu chuẩn hóa toàn bộ dữ liệu trước, trung bình và độ lệch chuẩn sẽ được tính cả từ các mẫu sẽ làm tập kiểm định trong từng lần chia, nên thông tin của tập kiểm định "rò rỉ" vào huấn luyện và điểm số kiểm định chéo bị lạc quan. Đặt trong pipeline thì mỗi lần chia tự tính lại từ phần huấn luyện của lần đó.
39. Nếu chọn C theo tập kiểm tra, tập này đã tham gia vào quyết định huấn luyện nên không còn "chưa nhìn thấy", và điểm số cuối cùng trên nó sẽ quá lạc quan. Kiểm định chéo trên tập huấn luyện cho phép chọn C mà vẫn giữ tập kiểm tra nguyên vẹn để đánh giá một lần ở cuối.

6. Tổng kết và mở rộng

Đây là mục cuối của bài giảng. Ta sẽ nhìn lại toàn bộ những gì đã học, đánh giá ưu và nhược điểm của Logistic Regression, đặt nó cạnh Perceptron để thấy rõ mối quan hệ giữa hai mô hình, rồi **mở đường** sang các mô hình mạnh hơn mà Logistic Regression là nền tảng: **Softmax Regression** (nhiều lớp) và **mạng nơ-ron**. Cuối mục là hệ thống bài tập.

Ta dùng quy ước ký hiệu cũ: **chữ thường in đậm** là vector, **chữ hoa in đậm** là ma trận, vector đặc trưng $\mathbf{x}_i$ là **vector hàng**, vector tham số $\mathbf{w}$ là **vector cột**, nhãn nhị phân $y_i \in \{0,1\}$, điểm số $s_i = \mathbf{x}_i\mathbf{w}$ và xác suất $p_i = \sigma(s_i)$.

Nhìn lại toàn bộ bài giảng

Bài giảng đi qua năm bước, mỗi bước trả lời một câu hỏi:

Mục	Câu hỏi	Kết quả chính
1	Perceptron còn thiếu gì? Làm sao có xác suất?	Giả thiết log-odds tuyến tính $\ln\frac{p}{1-p} = \mathbf{xw}$, suy ra $p = \sigma(\mathbf{xw})$. Nhãn $y \in \{0,1\}$.
2	Mô hình Logistic Regression là gì?	$\mathbf{p} = \sigma(\mathbf{Xw})$, đường biên $\mathbf{xw} = 0$, các đường đồng mức xác suất song song, e^{w_j} là odds ratio.
3	Làm sao huấn luyện?	Hợp lý cực đại cho hàm mất mát cross-entropy $J = -\frac{1}{n}\sum_i [y_i \ln p_i + (1-y_i)\ln(1-p_i)]$, gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{p}-\mathbf{y})$.
4	Hội tụ ra sao? Khi nào bất thường?	J lồi nên nghiệm duy nhất, không phụ thuộc khởi tạo. Dữ liệu phân tách hoàn hảo làm $\mathbf{w}$ phát tán, cần điều chuẩn hóa L2. Đường biên vẫn phẳng.
5	Mô hình tốt đến đâu?	Chia train/test, ma trận nhầm lẫn, ngưỡng $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}}+c_{\text{FN}}}$, log-loss, ROC, AUC, hiệu chuẩn, kiểm định chéo.

Có ba ý tưởng cốt lõi nên mang theo:

40. **Mô hình xác suất và hợp lý cực đại cho ta hàm mất mát một cách có nguyên tắc.** Cross-entropy không phải là lựa chọn tùy tiện: nó là $-\frac{1}{n}\ln$ của xác suất mà mô hình gán cho dữ liệu quan sát được.
41. **Gradient luôn có dạng $\mathbf{X}^T(\text{dự đoán} - \text{thực tế})$.** Dạng này xuất hiện ở hồi quy tuyến tính, Logistic Regression, Softmax Regression và cả lớp cuối của mạng nơ-ron (mục 6.3).
42. **Mô hình tuyến tính có giới hạn rõ ràng** (đường biên phẳng), và hầu hết sự phát triển tiếp theo của học máy là cách vượt qua giới hạn đó mà vẫn giữ được ba thành phần: mô hình, hàm mất mát, thuật toán tối ưu.

6.1. Ưu điểm và hạn chế của Logistic Regression

Ưu điểm

- **Cho ra xác suất.** Khác với Perceptron chỉ cho nhãn cứng, Logistic Regression cho biết **độ tin cậy**, nhờ đó ta chọn được ngưỡng theo hậu quả của từng loại sai (mục 5.3), đánh giá được bằng log-loss, AUC và biểu đồ hiệu chuẩn (mục 5.4).
- **Dễ diễn giải.** Mỗi trọng số cho biết cả hướng lẫn cường độ ảnh hưởng của đặc trưng, qua odds ratio e^{w_j} (mục 2.5).
- **Bài toán tối ưu "dễ chịu".** Hàm mất mát **lồi**, nghiệm duy nhất, mọi khởi tạo cho cùng kết quả, và có thể tối ưu bằng các phương pháp đã được kiểm chứng (mục 4.3).
- **Học êm khi dữ liệu có nhiễu hoặc chồng lấn**, không dao động mãi như Perceptron, và ít bị ảnh hưởng bởi những mẫu "rất đúng" ở xa (mục 6.2 bài Perceptron).
- **Nhanh và mở rộng được.** Chi phí mỗi epoch tỉ lệ với $n \cdot d$, có thể dùng SGD hay mini-batch cho dữ liệu rất lớn (mục 3.5), và học trực tuyến được.
- **Dễ kết hợp với điều chuẩn hóa** để kiểm soát độ phức tạp và xử lý dữ liệu phân tách hoàn hảo (mục 4.4).
- **Là nền tảng và đường cơ sở (baseline) tốt.** Nó là viên gạch của Softmax Regression và mạng nơ-ron, và trong thực tế thường là mô hình đầu tiên được thử trên một bài toán phân loại mới.

Hạn chế

Mỗi hạn chế dưới đây đã xuất hiện ở một mục trước, và đều dẫn đến một hướng phát triển:

Hạn chế	Nguyên nhân	Hướng khắc phục
Chỉ có **đường biên phẳng**: không giải được XOR, dữ liệu hình vòng tròn (mục 4.5)	Log-odds là hàm **tuyến tính** của đặc trưng	Thêm đặc trưng phi tuyến; mạng nơ-ron (mục 6.3)
Khi dữ liệu **phân tách hoàn hảo**, $\mathbf{w}$ tăng không giới hạn và xác suất cực đoan (mục 4.4)	Không có cực tiểu hữu hạn của J	Điều chuẩn hóa L2 hoặc L1
Nhạy với **thang đo** đặc trưng, hội tụ chậm nếu không chuẩn hóa (mục 4.2)	Số điều kiện của Hessian lớn	Chuẩn hóa đặc trưng
Một mẫu **sai nặng ở xa** (nhãn nhiễu, ngoại lai) kéo mạnh đường biên (mục 4.1)	Gradient $\lvert p_i - y_i \rvert \cdot \lVert \mathbf{x}_i \rVert$	Làm sạch dữ liệu, điều chuẩn hóa, các hàm mất mát bền vững hơn
Đa cộng tuyến: các đặc trưng tương quan mạnh làm trọng số bất ổn, khó diễn giải	Hessian gần suy biến (có trị riêng rất nhỏ)	Điều chuẩn hóa, loại bớt đặc trưng
Lớp **mất cân bằng** làm accuracy gây hiểu lầm	Mô hình thiên về lớp đông	Dùng precision, recall, AUC; đổi ngưỡng; gán trọng số lớp

Hạn chế	Nguyên nhân	Hướng khắc phục
Chỉ phân loại **hai lớp**	Một đầu ra xác suất	Softmax Regression (mục 6.3)

Đa cộng tuyến: khi trọng số không còn diễn giải được

Hạn chế về đa cộng tuyến đáng được xem xét kỹ, vì nó ảnh hưởng trực tiếp đến cách ta diễn giải odds ratio. Ta thử một thí nghiệm. Với dữ liệu sinh viên ở mục 3.7, ta thêm một đặc trưng thứ ba là **bản sao gần giống** của số giờ học: $x_1' = x_1 + \varepsilon$ với nhiễu ε rất nhỏ (độ lệch chuẩn 0,05), nên x_1 và x_1' có hệ số tương quan 0,9999. Ta huấn luyện Logistic Regression trên ba đặc trưng (x_1, x_1', x_2):

Cấu hình	Hệ số chệch	Trọng số (w_1, w_1', w_2)	Tổng $w_1 + w_1'$	Accuracy kiểm tra
Chỉ có (x_1, x_2), không điều chuẩn hóa	$-3{,}520$	$(1{,}151;\ -1{,}291)$		90,0%
Thêm bản sao, **không** điều chuẩn hóa	$-3{,}540$	$(-3{,}112;\ 4{,}272;\ -1{,}310)$	1,161	90,0%
Thêm bản sao, điều chuẩn hóa ($C = 1$)	$-3{,}471$	$(0{,}478;\ 0{,}640;\ -1{,}233)$	1,118	90,0%

Trường hợp thứ hai cho thấy hiện tượng: hai trọng số $-3{,}112$ và $4{,}272$ có **dấu ngược nhau và độ lớn lớn**, trong khi chúng gần như triệt tiêu nhau (tổng 1,161, xấp xỉ 1,151 ban đầu). Mô hình **dự đoán không đổi** (cùng accuracy), nhưng nếu đọc odds ratio riêng lẻ ta sẽ kết luận vô lý: $e^{-3{,}112} \approx 0{,}045$ ("thêm giờ học làm giảm odds") và $e^{4{,}272} \approx 71$ ("thêm giờ học làm odds tăng 71 lần") cho **cùng một đại lượng**. Ta chỉ có thể diễn giải **tổng** của chúng. Điều chuẩn hóa L2 ($C = 1$) khắc phục điều này bằng cách chia đều trọng số cho hai bản sao (0,478 và 0,640).

Điều chuẩn hóa L1

Ngoài L2 (mục 4.4), một lựa chọn phổ biến khác là **điều chuẩn hóa L1**, phạt tổng giá trị tuyệt đối của trọng số:

$$J_\lambda^{\mathrm{L1}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_{j=1}^{d} |w_j|$$

Khác với L2 chỉ làm trọng số **nhỏ đi**, L1 có xu hướng đưa nhiều trọng số về **đúng bằng** 0, tức là tự động **chọn đặc trưng** (mô hình thưa). Đổi lại, số hạng $|w_j|$ không khả vi tại 0 nên không dùng Gradient Descent thông thường được (cần các bộ giải chuyên dụng). Bài tập C1 so sánh L1 và L2 trên dữ liệu thật.

6.2. So sánh Perceptron và Logistic Regression

Hai mô hình có chung điểm xuất phát (điểm số tuyến tính $s = \mathbf{xw}$ và đường biên $\mathbf{xw} = 0$) nhưng khác nhau ở **đầu ra, hàm mất mát và cách cập nhật**. Với nhãn Perceptron $y_\pm = 2y - 1 \in \{-1, +1\}$ và margin $m = y_\pm\, s$:

	Perceptron	Logistic Regression
Điểm số	$s = \mathbf{xw}$	$s = \mathbf{xw}$ (giống hệt, là log-odds)
Hàm kích hoạt	Hàm dấu, hàm bước	Sigmoid $\sigma(s)$
Đầu ra	Nhãn cứng	Xác suất $p \in (0,1)$, từ đó suy ra nhãn
Mất mát theo margin	$\max(0, -m)$	$\ln(1 + e^{-m})$
Trọng số của mẫu khi cập nhật	Cứng: $\mathbb{1}[m \leq 0] \in \{0,1\}$	Mềm: $\sigma(-m) \in (0,1)$
Mẫu nào ảnh hưởng	Chỉ mẫu sai	Mọi mẫu, mạnh nhất là mẫu sai nặng
Dữ liệu phân tách được	Dừng sau hữu hạn bước ($k \leq R^2/\gamma^2$)	$\mathbf{w}$ tăng mãi, cần điều chuẩn hóa
Dữ liệu chồng lấn	Không hội tụ, $\mathbf{w}$ dao động (cần thuật toán túi)	Hội tụ êm về nghiệm duy nhất
Phụ thuộc khởi tạo, thứ tự	Có (nhiều nghiệm đúng)	Không (hàm mất mát lồi)
Ý nghĩa độ lớn của w	Không ảnh hưởng đến bộ phân loại	Điều khiển độ tự tin của xác suất
Ý nghĩa của w_j	Hướng ảnh hưởng	Hướng và cường độ: odds ratio e^{w_j}
Đổi ngưỡng, ROC, log-loss	Không (không có xác suất)	Có
Đạo hàm của hàm kích hoạt	0 hầu như mọi nơi	$\sigma(1-\sigma) > 0$ ở mọi nơi

Perceptron là trường hợp "cứng" của Logistic Regression

Hai mô hình có mối liên hệ chặt chẽ hơn so với một bảng so sánh gợi ý. Ta nhân điểm số với một hệ số $c > 0$ (tức dùng $c\,\mathbf{w}$ thay cho $\mathbf{w}$) và cho $c \to \infty$. Với mất mát logistic chia cho c:

$$\lim_{c\to\infty} \frac{1}{c}\ln(1 + e^{-c\,m}) = \max(0, -m)$$

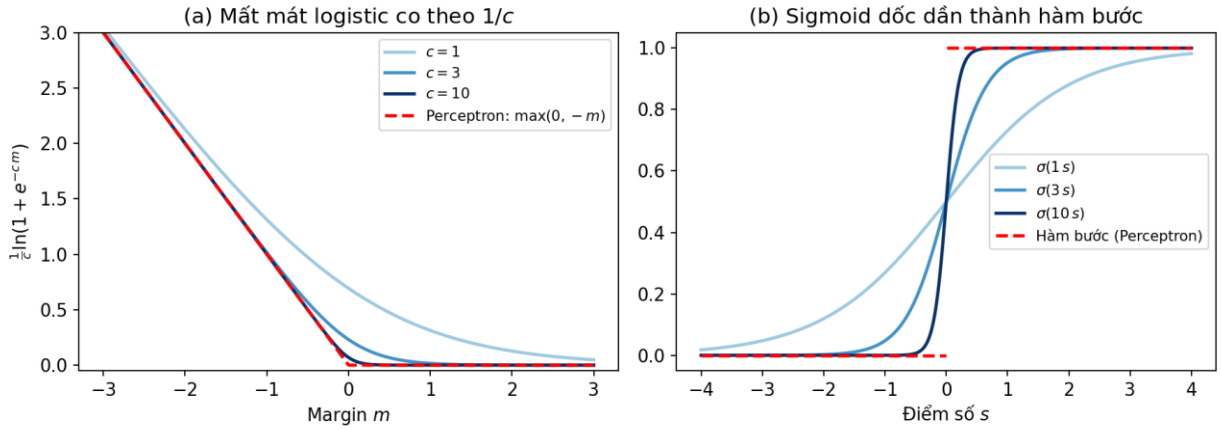
Chứng minh ngắn: nếu $m > 0$ thì $0 < \frac{1}{c}\ln(1 + e^{-cm}) \leq \frac{e^{-cm}}{c} \to 0$. Nếu $m < 0$ thì $\frac{1}{c}\ln(1 + e^{-cm}) = -m + \frac{1}{c}\ln(1 + e^{cm}) \to -m$. Ta kiểm chứng bằng số:

Margin m	$c = 1$	$c = 5$	$c = 20$	$c = 100$	$\max(0, -m)$
-2	2,1269	2,000009	2,0000	2,0000	2
0,5	0,4741	0,0158	0,000002	0,0000	0

Hình 21a vẽ sự co lại này: khi c tăng, đường cong logistic ngày càng sát mất mát Perceptron. Tương tự ở phía hàm kích hoạt (Hình 21b), $\sigma(c\,s)$ trở thành **hàm bước** khi $c \to \infty$, và trọng số cập nhật $\sigma(-c\,m)$ tiến tới trọng số cứng $\mathbb{1}[m < 0]$ của Perceptron. Nói cách khác:

$$\text{Perceptron} = \lim_{c\to\infty} \text{Logistic Regression với } c\,\mathbf{w}$$

Hiểu theo chiều ngược lại: **Logistic Regression là phiên bản "mềm", khả vi của Perceptron**. Việc làm mềm này chính là điều giúp nó có xác suất, có hàm mất mát lồi và có thể ghép thành mạng nhiều lớp.



Hình 21. (a) Mất mát logistic chia cho c tiến tới mất mát Perceptron khi c tăng. (b) Sigmoid $\sigma(c\,s)$ tiến tới hàm bước khi c tăng. Perceptron là giới hạn "cứng" của Logistic Regression.

So sánh bằng thực nghiệm

Ở bài Perceptron (mục 6.2) ta đã chạy thí nghiệm trên dữ liệu sinh viên mô phỏng, lấy trung bình của 20 lần sinh dữ liệu và chia ngẫu nhiên khác nhau:

Mô hình	Accuracy kiểm tra, dữ liệu gốc	Accuracy kiểm tra, thêm 15 mẫu ngoại lai đúng
Perceptron (thuật toán túi)	86,5% ± 2,8	86,5% ± 2,7
Logistic Regression	87,3% ± 2,8	87,3% ± 2,8

Và trong mục 5.5 của bài này (một lần chia cố định), cả hai cho accuracy kiểm tra 89,2% và 90,0%, AUC 0,952 và 0,953. Kết quả cho thấy cần thành thật: **về accuracy trên dữ liệu "dễ" như thế này, hai mô hình gần như ngang nhau** (chênh lệch nhỏ hơn độ lệch chuẩn giữa các lần chạy). Lợi thế của Logistic Regression không nằm ở một con số accuracy cao hơn mà ở:

- **xác suất** (đổi ngưỡng, log-loss, hiệu chuẩn, ROC);
- **quá trình học ổn định**: hội tụ êm, không phụ thuộc thứ tự mẫu và khởi tạo, không cần thuật toán túi;
- **khả năng mở rộng**: sang nhiều lớp (Softmax) và nhiều tầng (mạng nơ-ron), điều mà hàm dấu không làm được.

Perceptron vẫn có chỗ đứng khi cần một mô hình **đơn giản nhất có thể**, học trực tuyến với chi phí cập nhật cực thấp (chỉ cập nhật khi sai), hoặc khi mục tiêu thuần túy là dạy khái niệm.

6.3. Từ Logistic Regression đến Softmax Regression và mạng nơ-ron

Softmax Regression: nhiều hơn hai lớp

Khi có K lớp, ta dùng **một vector tham số cho mỗi lớp**, xếp thành các cột của ma trận $\mathbf{W}$ kích thước $(d+1)\times K$. Khi đó mỗi hàng của $\mathbf{S}=\mathbf{XW}$ chứa K điểm số của một mẫu. Để biến chúng thành xác suất, ta thay hàm sigmoid bằng **hàm softmax**, áp dụng cho từng hàng:

$$p_{ik}=P(y_i=k\mid \mathbf{x}_i)=\frac{e^{s_{ik}}}{\sum_{j=1}^{K} e^{s_{ij}}},\qquad \mathbf{P}=\operatorname{softmax}(\mathbf{XW})$$

Mỗi hàng của $\mathbf{P}$ có các phần tử dương cộng lại bằng 1. Lớp dự đoán là lớp có xác suất lớn nhất: $\hat{y}_i=\operatorname{argmax}_k p_{ik}$. **Ví dụ**: một mẫu có $K=3$ điểm số $[0{,}5 \quad 1{,}5 \quad 2{,}5]$ cho

$$e^{0,5}=1{,}6487,\quad e^{1,5}=4{,}4817,\quad e^{2,5}=12{,}1825\ \Rightarrow\ \text{tổng}=18{,}3129\ \Rightarrow\ \mathbf{p}\approx[0{,}090 \quad 0{,}245 \quad 0{,}665]$$

(Hình 22a). Softmax là **mở rộng tự nhiên của sigmoid**: khi $K=2$ với hai cột $\mathbf{w}_0$ và $\mathbf{w}_1$, đặt $s_0=\mathbf{x}\mathbf{w}_0$ và $s_1=\mathbf{x}\mathbf{w}_1$ thì

$$p_1=\frac{e^{s_1}}{e^{s_0}+e^{s_1}}=\frac{1}{1+e^{-(s_1-s_0)}}=\sigma\big(\mathbf{x}(\mathbf{w}_1-\mathbf{w}_0)\big)$$

đúng là Logistic Regression với vector tham số $\mathbf{w}_1-\mathbf{w}_0$. Một lưu ý tính toán: softmax không đổi khi ta cộng cùng một hằng số vào mọi điểm số, nên trong lập trình ta trừ đi giá trị lớn nhất của từng hàng trước khi lấy mũ để tránh tràn số.

Hàm mất mát. Dùng nhãn **one-hot** (ma trận $\mathbf{Y}$ kích thước $n\times K$, $y_{ik}=1$ nếu mẫu i thuộc lớp k, ngược lại bằng 0), hợp lý cực đại cho **cross-entropy nhiều lớp**:

$$J(\mathbf{W})=-\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\ln p_{ik}$$

Với $K=2$, nó trở về đúng cross-entropy nhị phân. Mất mát của một mẫu thuộc lớp c là $-\ln p_{ic}=-s_{ic}+\ln\sum_j e^{s_{ij}}$, nên đạo hàm theo điểm số s_{ik} là $p_{ik}-\mathbb{1}[k=c]$, tức $p_{ik}-y_{ik}$. Ví dụ ở trên, nếu nhãn thật là lớp 2 thì mất mát là $-\ln 0{,}245\approx 1{,}408$ và gradient theo các điểm số là $[0{,}090 \quad -0{,}755 \quad 0{,}665]$.

Gradient và cập nhật. Cộng gộp cho cả tập dữ liệu, ta có công thức **cùng dạng** với trường hợp nhị phân:

$$\nabla_{\mathbf{W}}J=\frac{1}{n}\mathbf{X}^{T}(\mathbf{P}-\mathbf{Y}),\qquad \mathbf{W}\leftarrow\mathbf{W}-\eta\,\nabla_{\mathbf{W}}J$$

Kiểm tra kích thước: $\mathbf{X}^T$ là $(d+1) \times n$ và $\mathbf{P} - \mathbf{Y}$ là $n \times K$ nên tích là $(d+1) \times K$, khớp với $\mathbf{W}$. Mã nguồn chỉ khác Logistic Regression ở hai chỗ: sigmoid thay bằng softmax, và vector thành ma trận.

```
def softmax(S):
    """Softmax theo từng hàng; trừ giá trị lớn nhất của hàng để không tràn số."""
    S = S - S.max(axis=1, keepdims=True)
    E = np.exp(S)
    return E / E.sum(axis=1, keepdims=True)

def mat_mat_softmax(Xb, Y, W):
    """Cross-entropy trung bình: -(1/n) * sum(Y * ln P)."""
    P = np.clip(softmax(Xb @ W), 1e-12, 1.0)
    return -np.mean(np.sum(Y * np.log(P), axis=1))

def huan_luyen_softmax(Xb, Y, eta=0.5, so_epoch=2000):
    """Gradient Descent cho Softmax Regression: grad = (1/n) X^T (P - Y)."""
    n, d1 = Xb.shape
    W = np.zeros((d1, Y.shape[1]))
    lich_su = [mat_mat_softmax(Xb, Y, W)]
    for epoch in range(so_epoch):
        P = softmax(Xb @ W)
        W = W - eta * Xb.T @ (P - Y) / n
        lich_su.append(mat_mat_softmax(Xb, Y, W))
    return W, lich_su
```

Ta chạy thử trên bộ dữ liệu hoa Iris (150 mẫu, 4 đặc trưng, 3 loài hoa), chia 70% huấn luyện và 30% kiểm tra theo kiểu phân tầng, đặc trưng được chuẩn hóa:

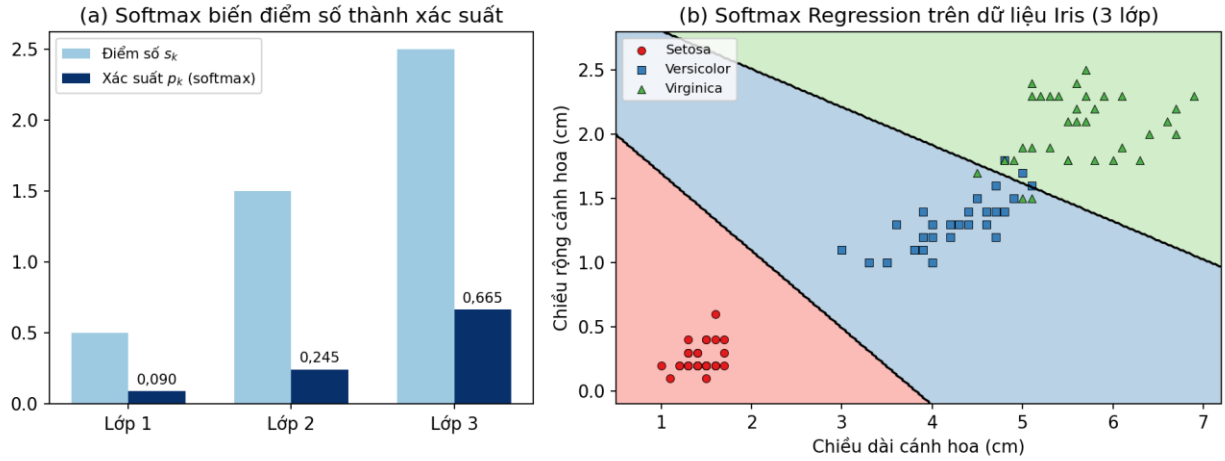
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix

d = load_iris(); X_iris, y_iris = d.data, d.target
X_tr, X_te, y_tr, y_te = train_test_split(X_iris, y_iris, test_size=0.3, rand
om_state=0, stratify=y_iris)
Z_tr, Z_te, mu, s = chuan_hoa(X_tr, X_te)
Zb_tr, Zb_te = them_bias(Z_tr), them_bias(Z_te)
Y_tr = np.eye(3)[y_tr]                                  # nhãn one-hot, kích
thước n x K
W, ls = huan_luyen_softmax(Zb_tr, Y_tr, eta=0.5, so_epoch=2000)
print("J ban dau =", round(ls[0], 4), "= ln 3 =", round(np.log(3), 4), "| J c
uoi =", round(ls[-1], 4))
print("W =")
print(W)
P_te = softmax(Zb_te @ W)
print("accuracy train =", np.mean(softmax(Zb_tr @ W).argmax(axis=1) == y_tr).
round(4),
```

```
      "| accuracy test =", np.mean(P_te.argmax(axis=1) == y_te).round(4))
print(confusion_matrix(y_te, P_te.argmax(axis=1)))
```

```
J ban dau = 1.0986 = ln 3 = 1.0986 | J cuoi = 0.0634
W =
[[-0.6158  4.6067 -3.9909]
 [-2.0832  1.3337  0.7494]
 [ 2.3429 -0.2745 -2.0684]
 [-4.079  -0.4941  4.5731]
 [-3.9368 -2.0567  5.9935]]
accuracy train = 0.9714 | accuracy test = 1.0
[[15  0  0]
 [ 0 15  0]
 [ 0  0 15]]
```

Khi $\mathbf{W} = \mathbf{0}$, mọi lớp có xác suất $1/K$ nên $J = \ln K$ (ở đây $\ln 3 \approx 1{,}0986$, tổng quát hóa của $\ln 2$ ở mục 3.2). Mô hình đạt 97,1% trên tập huấn luyện và phân loại đúng cả 45 mẫu kiểm tra. Ma trận nhầm lẫn bây giờ có kích thước $K \times K$ (ở đây 3×3). Ta cũng kiểm tra gradient giải tích $\frac{1}{n}\mathbf{X}^T(\mathbf{P} - \mathbf{Y})$ bằng đạo hàm số, sai khác chỉ cỡ 10^{-10}. Hình 22b vẽ kết quả khi chỉ dùng hai đặc trưng (chiều dài và chiều rộng cánh hoa): ba vùng quyết định được ngăn cách bởi các đường thẳng, đạt 94,3% trên tập huấn luyện và 97,8% trên tập kiểm tra.



Hình 22. (a) Softmax biến ba điểm số 0,5*;* 1,5*;* 2,5 *thành ba xác suất cộng lại bằng* 1. *(b) Softmax Regression trên dữ liệu Iris với hai đặc trưng (chiều dài và chiều rộng cánh hoa): ba vùng quyết định được ngăn cách bởi các đường thẳng.*

> **Lưu ý.** Loài hoa Setosa tách hoàn toàn khỏi hai loài còn lại, nên dữ liệu này gần như phân tách hoàn hảo, và ta lại gặp hiện tượng ở mục 4.4: nghiệm không duy nhất khi không điều chuẩn hóa. Ví dụ, `LogisticRegression` của scikit-learn với C rất lớn cho nhãn dự đoán **trùng** 100% với mô hình ở trên nhưng xác suất lệch nhau tới 0,25 ở một số mẫu, vì hai bộ giải dừng ở những $\mathbf{W}$ khác nhau theo hướng "đẩy ra vô cực".

Một khung chung: gradient $\mathbf{X}^T(\mu - \mathbf{y})$

Ta đã thấy cùng một dạng gradient ở ba mô hình. Đây không phải ngẫu nhiên: các mô hình này thuộc một họ gọi là **mô hình tuyến tính tổng quát**, trong đó trung bình μ của nhãn là một hàm của điểm số tuyến tính, và hàm mất mát là $-\ln$ của hợp lý tương ứng. Gradient luôn có dạng $\frac{1}{n}\mathbf{X}^T(\boldsymbol{\mu} - \mathbf{y})$:

Mô hình	Trung bình μ	Phân phối của y
Hồi quy tuyến tính (mất mát $\frac{1}{2}$MSE)	$\mu = \mathbf{xw}$	Chuẩn (Gauss)
Logistic Regression	$\mu = \sigma(\mathbf{xw})$	Bernoulli
Softmax Regression	$\boldsymbol{\mu} = \text{softmax}(\mathbf{xW})$	Phân loại nhiều lớp
Hồi quy Poisson (đếm số sự kiện)	$\mu = e^{\mathbf{xw}}$	Poisson

Nhờ đó, một khi đã hiểu Logistic Regression, việc học thêm các mô hình trong họ này chỉ là thay hàm μ và phân phối tương ứng.

Mạng nơ-ron: xếp nhiều lớp Logistic Regression

Một Logistic Regression chính là **một nơ-ron nhân tạo** với hàm kích hoạt sigmoid. Giới hạn của đường biên phẳng (mục 4.5) được vượt qua bằng cách **xếp nhiều nơ-ron thành các lớp**. Ta xét mạng một lớp ẩn gồm m nơ-ron. Với ma trận trọng số lớp ẩn $\mathbf{W}_1$ kích thước $(d+1) \times m$ (mỗi cột là một nơ-ron ẩn) và vector trọng số lớp ra $\mathbf{w}_2$ kích thước $(m+1) \times 1$:

$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}_1), \qquad \tilde{\mathbf{H}} = [\mathbf{1} \quad \mathbf{H}], \qquad \mathbf{p} = \sigma\big(\tilde{\mathbf{H}}\mathbf{w}_2\big)$$

Trong đó $\mathbf{H}$ $(n \times m)$ là đầu ra của các nơ-ron ẩn, và $\tilde{\mathbf{H}}$ là $\mathbf{H}$ thêm một cột 1 cho hệ số chệch. Có một nhận xét quan trọng: **lớp ra chính là Logistic Regression**, chạy trên các đặc trưng $\mathbf{H}$ mà lớp ẩn **tự học ra** thay vì các đặc trưng $\mathbf{X}$ ta đưa vào. Điều này giống việc ta thêm đặc trưng x_1^2, x_1x_2, x_2^2 ở mục 4.5, nhưng mạng tự tìm ra đặc trưng phù hợp thay vì ta phải đoán.

Huấn luyện bằng lan truyền ngược. Hàm mất mát vẫn là cross-entropy J, và ta cần gradient theo mọi trọng số. Ở lớp ra, gradient có dạng $\mathbf{y}$ **hệt** Logistic Regression, chỉ thay $\mathbf{X}$ bằng $\tilde{\mathbf{H}}$:

$$\nabla_{\mathbf{w}_2} J = \frac{1}{n}\,\tilde{\mathbf{H}}^T(\mathbf{p} - \mathbf{y})$$

Ở lớp ẩn, ta "lan truyền ngược" sai số $(\mathbf{p} - \mathbf{y})$ qua các trọng số $\mathbf{v}$ ($\mathbf{w}_2$ bỏ phần tử chệch đầu tiên) và đạo hàm sigmoid $\mathbf{H} \odot (1 - \mathbf{H})$ theo quy tắc đạo hàm hàm hợp:

$$\nabla_{\mathbf{W}_1} J = \frac{1}{n}\,\mathbf{X}^T\big[\big((\mathbf{p} - \mathbf{y})\,\mathbf{v}^T\big) \odot \mathbf{H} \odot (1 - \mathbf{H})\big]$$

trong đó $\odot$ là phép nhân từng phần tử và $1 - \mathbf{H}$ cũng tính theo từng phần tử. Công thức này suy ra trực tiếp từ đạo hàm sigmoid $\sigma' = \sigma(1 - \sigma)$ mà ta đã chứng minh ở mục 2.3, và đây là lý do hàm kích hoạt phải **khả vi** (ở Perceptron, hàm dấu có đạo hàm 0 nên không truyền được gradient về lớp ẩn, như đã nói ở mục 6.3 bài Perceptron). Mã nguồn:

```
def mang_hai_lop(X, y, so_an=3, eta=2.0, so_epoch=10000, seed=0):
    """Mạng nơ-ron một lớp ẩn (sigmoid), đầu ra sigmoid, huấn luyện bằng lan
truyền ngược."""
    rng = np.random.default_rng(seed)
    n = len(y)
    Xb = them_bias(X)
    W1 = rng.normal(0, 1, (Xb.shape[1], so_an))        # (d+1) x m: mỗi cột l
à một nơ-ron ẩn
    w2 = rng.normal(0, 1, so_an + 1)                   # (m+1): hệ số chệch +
trọng số đầu ra
    lich_su = []
    for ep in range(so_epoch):
        H = sigmoid(Xb @ W1)                           # lớp ẩn
        Hb = them_bias(H)
        p = sigmoid(Hb @ w2)                           # xác suất đầu ra
        lich_su.append(-np.mean(y * np.log(np.clip(p, 1e-12, 1)) + (1 - y) *
np.log(np.clip(1 - p, 1e-12, 1))))
        d2 = (p - y) / n                               # gradient theo điểm s
ố đầu ra
        g_w2 = Hb.T @ d2                               # lớp ra: dạng giống h
ệt Logistic Regression
        dH = np.outer(d2, w2[1:]) * H * (1 - H)        # lan truyền ngược về
lớp ẩn
        g_W1 = Xb.T @ dH
        w2 = w2 - eta * g_w2
        W1 = W1 - eta * g_W1
    return W1, w2, lich_su

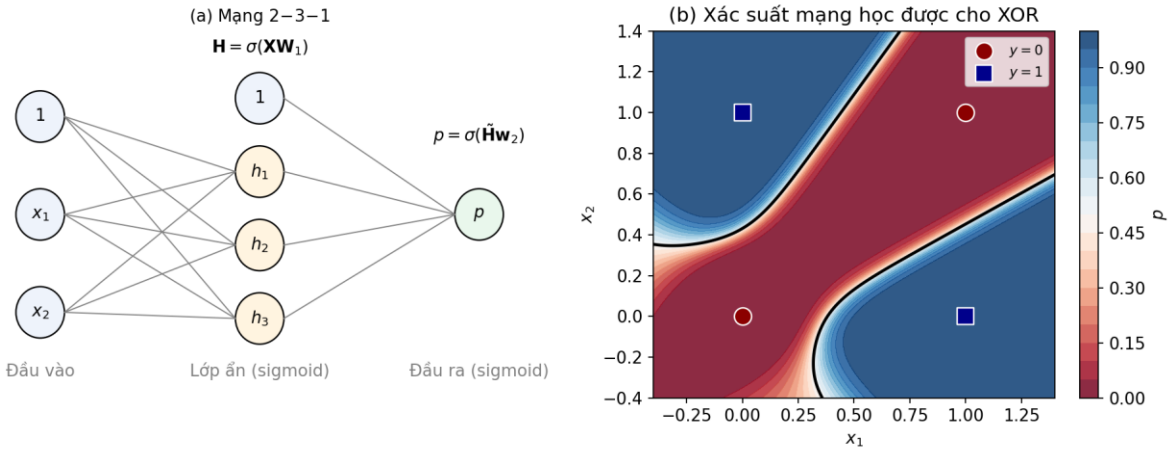
def du_doan_mang(X, W1, w2):
    Hb = them_bias(sigmoid(them_bias(X) @ W1))
    return sigmoid(Hb @ w2)
```

Mạng học XOR. Khác với mục 6 bài Perceptron (nơi ta tự chọn trọng số bằng tay), bây giờ mạng tự học từ khởi tạo ngẫu nhiên:

```
X4 = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y4 = np.array([0, 1, 1, 0], dtype=float)
for so_an in (2, 3):
    thanh_cong = 0
    for seed in range(10):
        W1, w2, ls = mang_hai_lop(X4, y4, so_an=so_an, eta=2.0, so_epoch=1000
0, seed=seed)
        thanh_cong += np.all((du_doan_mang(X4, W1, w2) >= 0.5) == y4)
    print(f"{so_an} no-ron an: hoc dung XOR o {thanh_cong}/10 lan khoi tao")
W1, w2, ls = mang_hai_lop(X4, y4, so_an=3, eta=2.0, so_epoch=10000, seed=0)
print("p =", du_doan_mang(X4, W1, w2).round(3), "| J cuoi =", round(ls[-1], 4
))
```

```
2 no-ron an: hoc dung XOR o 9/10 lan khoi tao
3 no-ron an: hoc dung XOR o 10/10 lan khoi tao
p = [0.    1.    0.999 0.001] | J cuoi = 0.0005
```

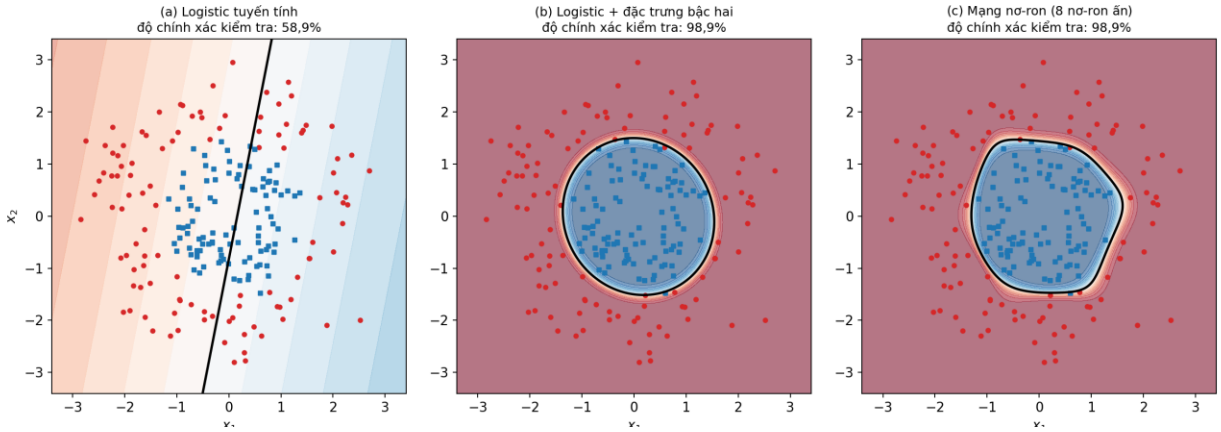
Mạng với 3 nơ-ron ẩn học đúng XOR ở **cả** 10 **lần** khởi tạo ngẫu nhiên, còn 2 nơ-ron ẩn thành công ở 9 trong 10 lần (một lần bị "kẹt", vì hàm mất mát của mạng nhiều lớp **không còn lồi**, đây là điểm khác căn bản so với Logistic Regression). Xác suất thu được là [0 1 0,999 0,001], đúng nhãn XOR [0 1 1 0]. Hình 23 vẽ cấu trúc mạng và bản đồ xác suất học được.



Hình 23. (a) Cấu trúc mạng $2 - 3 - 1$*: hai đặc trưng, ba nơ-ron ẩn và một nơ-ron ra, mỗi nơ-ron là một Logistic Regression. (b) Bản đồ xác suất mà mạng học được cho bài toán XOR: hai điểm lớp* 1 *nằm trong dải xanh, hai điểm lớp* 0 *nằm trong vùng đỏ, đường biên cong và gồm hai nhánh.*

Mạng học dữ liệu hình vòng tròn. Ở mục 4.5, ta phải tự thêm các đặc trưng bậc hai để Logistic Regression phân loại được dữ liệu hai vòng tròn đồng tâm. Mạng nơ-ron với 8 nơ-ron ẩn học điều đó **trực tiếp từ hai đặc trưng gốc**:

Mô hình	Số đặc trưng đầu vào	Accuracy huấn luyện	Accuracy kiểm tra
Logistic Regression tuyến tính	2	51,0%	58,9%
Logistic Regression + đặc trưng bậc hai (ta tự thêm)	5	99,0%	98,9%
Mạng nơ-ron 8 nơ-ron ẩn (tự học đặc trưng)	2	99,0%	98,9%



Hình 24. Dữ liệu hình vòng tròn. (a) Logistic Regression tuyến tính chỉ có đường thẳng. (b) Thêm đặc trưng bậc hai do ta tự chọn. (c) Mạng nơ-ron tự học đặc trưng từ hai đặc trưng gốc và cho đường biên cong gần như tròn.

Số nơ-ron ẩn cũng quan trọng: với cùng cách huấn luyện, 2 nơ-ron ẩn chỉ đạt khoảng 80% đến 86% trên tập kiểm tra (không đủ để tạo đường biên khép kín), 4 nơ-ron ẩn đạt khoảng 97% đến 99%, và 8 nơ-ron ẩn đạt khoảng 98% đến 99% (kết quả qua 5 lần khởi tạo khác nhau).

Lộ trình học tiếp

Có thể xem các mô hình trong chuỗi bài giảng này như các nấc thang, mỗi nấc thay đổi một thành phần của khung "mô hình, hàm mất mát, thuật toán tối ưu":

Mô hình	Điểm khác so với bước trước
Hồi quy tuyến tính	Dự đoán một số thực, mất mát MSE
Perceptron	Thêm hàm dấu để ra nhãn, mất mát $\max(0, -m)$
Logistic Regression	Làm mềm hàm dấu thành sigmoid, mất mát cross-entropy, cho xác suất
Softmax Regression	Nhiều lớp: ma trận $\mathbf{W}$, hàm softmax
Mạng nơ-ron nhiều lớp	Xếp nhiều Logistic Regression thành lớp, huấn luyện bằng lan truyền ngược, tự học đặc trưng
Mạng sâu (CNN, RNN, Transformer, …)	Cùng khung huấn luyện, thay đổi kiến trúc để xử lý ảnh, chuỗi, văn bản

6.4. Bài tập thực hành

Phần A. Tính tay và lý thuyết

A1. Một mô hình cho xác suất đậu của một sinh viên là $p = 0{,}8$. Tính odds và log-odds. Ngược lại, nếu log-odds là -2 thì xác suất bằng bao nhiêu?

A2. Cho 3 mẫu một đặc trưng: $\mathbf{X} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$ và $\mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$. Từ $\mathbf{w} = \mathbf{0}$, hãy tính J, gradient $\frac{1}{n}\mathbf{X}^T(\mathbf{p} - \mathbf{y})$ và $\mathbf{w}$ sau một bước Gradient Descent với $\eta = 1$. Tính lại J sau bước đó.

A3. Tung một đồng xu không cân đối 10 lần, được 7 lần ngửa. Dùng nguyên lý hợp lý cực đại với $L(p) = p^7(1-p)^3$ để tìm xác suất ngửa $\hat{p}$ (gợi ý: lấy $\ln L$ rồi đạo hàm).

A4. Tính softmax của $\mathbf{s} = [1 \quad 2 \quad 3]$ và so với ví dụ $[0{,}5 \quad 1{,}5 \quad 2{,}5]$ ở mục 6.3. Nhận xét. Nếu nhãn thật là lớp 3, mất mát là bao nhiêu?

A5. Chứng minh rằng Softmax với $K = 2$ trở về sigmoid (xác suất $p_1 = \sigma(s_1 - s_0)$).

A6. Một mô hình Logistic Regression có trọng số $w = 0{,}9$ cho đặc trưng "số năm kinh nghiệm". Tính odds ratio khi thêm 1 năm và 3 năm. Nếu một người có xác suất ban đầu $p = 0{,}2$ thì sau khi thêm 1 năm xác suất là bao nhiêu?

A7. Nếu bỏ sót một ca bệnh (FN) tốn kém gấp 5 lần một báo động nhầm (FP), ngưỡng tối ưu τ^* để dự đoán "bệnh" là bao nhiêu?

A8. Tính $\frac{1}{c}\ln(1 + e^{-c\,m})$ với $c = 10$ cho $m = -1$ và $m = 1$. So với $\max(0, -m)$ và nhận xét.

Phần B. Lập trình

B1. Cài đặt Logistic Regression từ đầu (sigmoid, cross-entropy, gradient, Gradient Descent) và kiểm tra gradient bằng đạo hàm số (sai khác nhỏ hơn 10^{-6}).

B2. So sánh batch, SGD và mini-batch trên dữ liệu ở mục 3.7 với nhiều tốc độ học. Vẽ J theo epoch và nhận xét.

B3. Với dữ liệu 6 sinh viên (phân tách hoàn hảo), vẽ $\|\,\tilde{\mathbf{w}}\,\|$ theo λ khi điều chuẩn hóa L2 và so với kết quả ở mục 4.4.

B4. Lặp lại thí nghiệm đa cộng tuyến ở mục 6.1 với các mức nhiễu của bản sao đặc trưng là $0{,}05$; $0{,}2$; $0{,}5$ và 1. Vẽ các trọng số theo mức nhiễu, có và không có điều chuẩn hóa.

B5. Cài đặt đường ROC và AUC từ đầu (không dùng thư viện), rồi so với `roc_auc_score`.

B6. Tải bộ dữ liệu `load_breast_cancer` của scikit-learn. Gán nhãn 1 cho lớp ác tính (`target == 0`). Chia train/test phân tầng $70/30$, chuẩn hóa bằng pipeline, huấn luyện Logistic Regression và Perceptron. Báo cáo accuracy, ma trận nhầm lẫn, log-loss, AUC. Loại sai nào nguy hiểm hơn và nên chỉnh ngưỡng theo hướng nào?

B7. Cài đặt Softmax Regression từ đầu và huấn luyện trên Iris (như mục 6.3). Kiểm tra gradient bằng đạo hàm số.

Phần C. Mở rộng

C1. So sánh điều chuẩn hóa L1 và L2 trên dữ liệu ung thư vú ở B6: đếm số trọng số khác 0 và so sánh accuracy khi C thay đổi.

C2. Cài đặt mạng nơ-ron hai lớp như ở mục 6.3, kiểm tra gradient bằng đạo hàm số, rồi huấn luyện trên dữ liệu hình vòng tròn với 2, 4 và 8 nơ-ron ẩn. Vẽ đường biên.

C3. Tạo dữ liệu mất cân bằng (95% lớp âm, 5% lớp dương). Huấn luyện Logistic Regression có và không có `class_weight="balanced"`, so sánh accuracy, recall, precision và AUC.

Gợi ý đáp án

A1. $p = 0{,}8$: odds $= \frac{0{,}8}{0{,}2} = 4$ và log-odds $= \ln 4 \approx 1{,}386$. Log-odds $= -2$: $p = \sigma(-2) = \frac{1}{1+e^2} \approx 0{,}119$.

A2. Với $\mathbf{w} = \mathbf{0}$, mọi $p_i = 0{,}5$ nên $J = \ln 2 \approx 0{,}6931$ và $\mathbf{p} - \mathbf{y} = [0{,}5 \quad 0{,}5 \quad -0{,}5]^T$. Khi đó $\mathbf{X}^T(\mathbf{p} - \mathbf{y}) = \begin{bmatrix} 0{,}5 + 0{,}5 - 0{,}5 \\ 0 + 0{,}5 - 1 \end{bmatrix} = \begin{bmatrix} 0{,}5 \\ -0{,}5 \end{bmatrix}$ và gradient $= \frac{1}{3}\begin{bmatrix} 0{,}5 \\ -0{,}5 \end{bmatrix} \approx \begin{bmatrix} 0{,}1667 \\ -0{,}1667 \end{bmatrix}$. Cập nhật: $\mathbf{w} = \mathbf{0} - 1 \cdot \begin{bmatrix} 0{,}1667 \\ -0{,}1667 \end{bmatrix} = \begin{bmatrix} -0{,}1667 \\ 0{,}1667 \end{bmatrix}$. Điểm số mới $s = -0{,}1667 + 0{,}1667x$ cho $\mathbf{s} = [-0{,}1667 \quad 0 \quad 0{,}1667]^T$, xác suất $\mathbf{p} \approx [0{,}4584 \quad 0{,}5 \quad 0{,}5416]^T$, và $J \approx \frac{0{,}6137 + 0{,}6931 + 0{,}6130}{3} \approx 0{,}6399$, giảm so với $0{,}6931$.

A3. $\ln L = 7\ln p + 3\ln(1-p)$. Đạo hàm: $\frac{7}{p} - \frac{3}{1-p} = 0 \Rightarrow 7(1-p) = 3p \Rightarrow \hat{p} = 0{,}7$, đúng bằng tỉ lệ quan sát được. Đây là trường hợp đơn giản nhất của hợp lý cực đại.

A4. $e^1 = 2{,}718$; $e^2 = 7{,}389$; $e^3 = 20{,}086$; tổng $= 30{,}193$ nên $\mathbf{p} \approx [0{,}090 \quad 0{,}245 \quad 0{,}665]$, **giống hệt** ví dụ ở mục 6.3. Lý do: $[1 \quad 2 \quad 3]$ chỉ hơn $[0{,}5 \quad 1{,}5 \quad 2{,}5]$ một hằng số $0{,}5$, và softmax không đổi khi cộng cùng một hằng số vào mọi điểm số. Nếu nhãn thật là lớp 3 thì mất mát là $-\ln 0{,}665 \approx 0{,}408$.

A5. $p_1 = \frac{e^{s_1}}{e^{s_0} + e^{s_1}}$. Chia cả tử và mẫu cho e^{s_1}: $p_1 = \frac{1}{e^{s_0 - s_1} + 1} = \frac{1}{1 + e^{-(s_1 - s_0)}} = \sigma(s_1 - s_0)$. Vậy Softmax hai lớp là Logistic Regression với điểm số $s_1 - s_0 = \mathbf{x}(\mathbf{w}_1 - \mathbf{w}_0)$.

A6. Odds ratio khi thêm 1 năm: $e^{0{,}9} \approx 2{,}46$. Khi thêm 3 năm: $e^{2{,}7} \approx 14{,}88$ (bằng $2{,}46^3$, vì các tác động nhân lên nhau). Với $p = 0{,}2$: odds ban đầu $= \frac{0{,}2}{0{,}8} = 0{,}25$; sau 1 năm odds $= 0{,}25 \times 2{,}46 \approx 0{,}615$ nên $p = \frac{0{,}615}{1 + 0{,}615} \approx 0{,}381$.

A7. Lớp dương là "bệnh": $c_{\text{FN}} = 5$ và $c_{\text{FP}} = 1$, nên $\tau^* = \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = \frac{1}{6} \approx 0{,}167$. Ngưỡng thấp hơn nhiều so với $0{,}5$: vì bỏ sót nguy hiểm hơn, ta dự đoán "bệnh" ngay cả khi xác suất bệnh chỉ trên $16{,}7\%$.

A8. Với $m = -1$: $\frac{1}{10}\ln(1 + e^{10}) \approx \frac{10{,}0000454}{10} \approx 1{,}0000045$, rất gần $\max(0,1) = 1$. Với $m = 1$: $\frac{1}{10}\ln(1 + e^{-10}) \approx 4{,}5 \cdot 10^{-6}$, rất gần $\max(0, -1) = 0$. Khi c lớn, mất mát logistic đã gần như trùng với mất mát Perceptron.

B1. Kết quả mong đợi: gradient giải tích và gradient số trùng nhau đến ít nhất 6 chữ số thập phân; $J(\mathbf{0}) = \ln 2$ (kiểm tra cài đặt); J giảm đều qua các epoch khi η phù hợp.

B2. Mong đợi: batch mượt nhưng chậm theo epoch, SGD nhanh nhưng dao động, mini-batch ở giữa (xem Hình 10 ở mục 3.7). Với đặc trưng chưa chuẩn hóa, $\eta \gtrsim 0{,}6$ bắt đầu dao động ở chế độ batch.

B3. Mong đợi: $\| \tilde{\mathbf{w}} \|$ giảm đơn điệu khi λ tăng (ví dụ ở mục 4.4: $\approx 3{,}04;\ 2{,}05;\ 1{,}18$ với $\lambda = 0{,}001;\ 0{,}01;\ 0{,}1$).

B4. Mong đợi: khi mức nhiễu nhỏ (đặc trưng gần trùng nhau), trọng số không điều chuẩn hóa có độ lớn lớn và dấu ngược nhau (như $-3{,}11$ và $4{,}27$ với nhiễu $0{,}05$), còn với điều chuẩn hóa thì ổn định và gần bằng nhau. Khi mức nhiễu tăng, hai đặc trưng bớt tương quan và trọng số không điều chuẩn hóa trở nên ổn định hơn. Tổng $w_1 + w_1'$ luôn xấp xỉ $1{,}15$.

B5. Với mô hình ở mục 5, AUC phải bằng $0{,}9534$ trên tập kiểm tra (tính bằng đếm cặp hoặc bằng diện tích dưới đường ROC).

B6. Kết quả tham khảo (nhãn 1 là ác tính, `test_size=0.3`, `random_state=0`, phân tầng, 171 mẫu kiểm tra): Logistic Regression ($C = 1$, có chuẩn hóa) đạt accuracy khoảng 95,3%, AUC khoảng $0{,}992$, log-loss khoảng $0{,}107$, ma trận nhầm lẫn $\begin{bmatrix} 103 & 4 \\ 4 & 60 \end{bmatrix}$ (TN, FP, FN, TP); Perceptron (có chuẩn hóa) đạt accuracy khoảng 96,5% và AUC của điểm số khoảng $0{,}992$. Chênh lệch giữa hai mô hình nằm trong dao động ngẫu nhiên của tập kiểm tra 171 mẫu. Với chẩn đoán ung thư, **FN** (bỏ sót ca ác tính) nguy hiểm hơn FP, nên nên **hạ** ngưỡng τ xuống dưới $0{,}5$ để tăng recall, chấp nhận thêm một số báo động nhầm.

B7. Kết quả tham khảo: J ban đầu $= \ln 3 \approx 1{,}0986$, J cuối $\approx 0{,}0634$ sau 2000 epoch với $\eta = 0{,}5$, accuracy huấn luyện $\approx 97{,}1\%$, accuracy kiểm tra 100% trên 45 mẫu. Gradient giải tích và gradient số sai khác cỡ 10^{-10}.

C1. Kết quả tham khảo (bộ giải `liblinear`, có chuẩn hóa, 30 đặc trưng): với L2, cả 30 trọng số đều khác 0 ở mọi C. Với L1, số trọng số khác 0 là $7;\ 9;\ 11;\ 13$ ứng với $C = 0{,}05;\ 0{,}1;\ 0{,}5;\ 1$, và accuracy kiểm tra khoảng 94,7%. L1 tự chọn một tập con nhỏ các đặc trưng với accuracy gần tương đương.

C2. Kết quả tham khảo qua 5 lần khởi tạo: 2 nơ-ron ẩn cho accuracy kiểm tra khoảng 80% đến 86%; 4 nơ-ron ẩn khoảng 97% đến 99%; 8 nơ-ron ẩn khoảng 98% đến 99%. Gradient của mạng nên được kiểm tra bằng đạo hàm số trước khi huấn luyện.

C3. Mong đợi: không dùng `class_weight`, mô hình thiên về lớp âm, accuracy cao (gần 95%) nhưng recall của lớp dương thấp. Với `class_weight="balanced"`, recall tăng, precision giảm, accuracy giảm nhẹ, còn AUC (chỉ phụ thuộc xếp hạng) gần như không đổi. Điều này minh họa vì sao với dữ liệu lệch lớp ta cần xem recall, precision và AUC thay vì chỉ accuracy.

Lời kết

Logistic Regression là một mô hình nhỏ nhưng chứa trong nó những ý tưởng lớn: **mô hình hóa bằng xác suất**, **hợp lý cực đại** như nguyên lý sinh ra hàm mất mát, **gradient có dạng** $\mathbf{X}^T(\text{dự đoán} - \text{thực tế})$, và sự cân bằng giữa **khớp dữ liệu** với **độ lớn của tham số** qua điều chuẩn hóa. Đặt cạnh Perceptron, ta thấy nó chính là phiên bản "mềm" của Perceptron, và chính sự "mềm" đó mở ra cánh cửa sang Softmax Regression, các mô hình tuyến tính tổng quát và

mạng nơ-ron. Nắm vững Logistic Regression, bạn đã có một nền tảng chắc chắn để bước vào các mô hình học sâu.